FPT UNIVERSITY

Capstone Project Report

Report 3 – Software Requirement Specification

– Can Tho, October 2026 –

Table of Contents

I. Record of Changes 4

II. Software Requirement Specification 5

1. Product Overview 5

2. User Requirements 6

2.1 Actors 6

2.2 Use Cases 9

2.2.1 Diagram(s) 9

2.2.2 Descriptions 29

3. Functional Requirements 40

3.1 System Functional Overview 40

3.1.1 Screens Flow 41

3.1.2 Screen Descriptions 43

3.1.3 Screen Authorization 47

3.1.4 Non-Screen Functions 50

3.1.5 Entity Relationship Diagram 64

3.2 Manage Forum 85

3.2.1 Function: View Forum Category List 85

3.2.2 Function: View Forum Category Detail and Post List 90

3.2.3 Function: View Post Detail 95

3.2.4 Function: Create Post 100

3.2.5 Function: Edit Post 105

3.2.6 Function: Delete Post 110

3.2.7 Function: Like a Post 114

3.2.8 Function: View Post Likes 117

3.2.9 Function: Create Comment 121

3.2.10 Function: Update Comment 124

3.2.11 Function: Delete Comment 128

3.2.12 Function: Like a Comment 132

3.2.13 Function: Report Content 135

3.3 Manage Conversation 139

3.3.1 Start Conversation 139

3.3.2 View Conversation List 143

3.3.3 View Conversation Detail 145

3.3.4 Send Message 147

3.3.5 Send File 149

3.3.6 Mark Conversation as Read . . . 151
3.3.7 Pin Conversation . . . 153
3.3.8 Delete Message . . . 155
3.4 Moderation . . . 157
3.4.1 View Moderation Queue . . . 157
3.4.2 View Moderation Item Detail . . . 159
3.4.3 Resolve Moderation Item . . . 161
3.5 ID Verification . . . 163
3.5.1 View Verification Request List . . . 164
3.5.2 View ID Verification Request Detail . . . 167
3.5.3 Approve or Reject ID Verification . . . 169
3.6 Trash . . . 172
3.6.1 View Deleted Post List . . . 172
3.6.2 View Deleted Post Detail . . . 174
3.6.3 Restore Deleted Post . . . 176
3.6.4 View Deleted Comment List . . . 178
3.6.5 View Deleted Comment Detail . . . 180
3.6.6 Restore Deleted Comment . . . 182
4. Non-Functional Requirements . . . 185
4.1 External Interfaces . . . 185
4.2 Quality Attributes . . . 185
4.2.1 Usability . . . 186
4.2.2 Reliability . . . 186
4.2.3 Performance . . . 186
4.2.4 Security & Privacy . . . 187
5. Requirement Appendix . . . 187
5.1 Business Rules . . . 187
5.2 Common Requirements . . . 193
5.3 Application Messages List . . . 194

I. Record of Changes

Date	A* M, D	In charge	Change Description

**A - Added M - Modified D - Deleted*

II. Software Requirement Specification

1. Product Overview

The Frevia project is a comprehensive freelancing platform that enables clients, freelancers, and administrators to build trusted professional relationships and efficiently manage the entire project lifecycle. Designed to connect businesses with skilled talent worldwide, Frevia provides intelligent job matching, structured contract workflows, and secure payment solutions to facilitate successful collaborations.

Through AI-powered recommendations, real-time communication, and advanced moderation mechanisms, the platform helps users discover suitable opportunities, deliver high-quality work, and maintain transparent interactions.

Frevia serves three primary stakeholder groups: clients seeking specialized expertise, freelancers showcasing their skills and services, and administrators responsible for maintaining platform integrity and operational efficiency. The platform offers an intuitive user experience, dynamic communication channels, and comprehensive management capabilities to support collaboration, professional growth, and secure transactions.

Key features of Frevia include:

- **Smart Job Matching and AI Recommendations:** The platform analyzes freelancer skills, experience, and preferences to recommend relevant job opportunities. AI-powered profile scoring provides quality assessments, while job alerts and saved searches help users stay informed about new opportunities.
- **End-to-End Contract and Milestone Management:** Clients and freelancers can negotiate project terms, define milestones and tasks, monitor progress, and securely manage payments. Integrated contract signing, activity logs, and file-sharing capabilities ensure transparency throughout the project lifecycle.
- **Community and Forum Engagement:** Frevia includes a community forum where users can discuss industry topics, exchange knowledge, and seek professional advice. Features such as categories, posts, comments, reactions, and reporting mechanisms encourage active participation and self-moderation.
- **Identity Verification and Content Moderation:** Freelancers can verify their identities through document submission, while administrators utilize moderation tools, banned keyword management, and AI-assisted violation detection to maintain a secure and trustworthy environment.

- **Wallet, Payments, and Gas Fee Subsidies:** Users can manage account balances, withdraw earnings, and review transaction histories. The platform also supports gas fee subsidy mechanisms for blockchain-based transactions, improving payment accessibility and reducing operational costs.
- **Administrative Management Tools:** Administrators have access to comprehensive management functionalities, including user administration, role and permission control, system configuration, advertisement management, activity monitoring, and dispute resolution, ensuring fair and reliable platform operations.

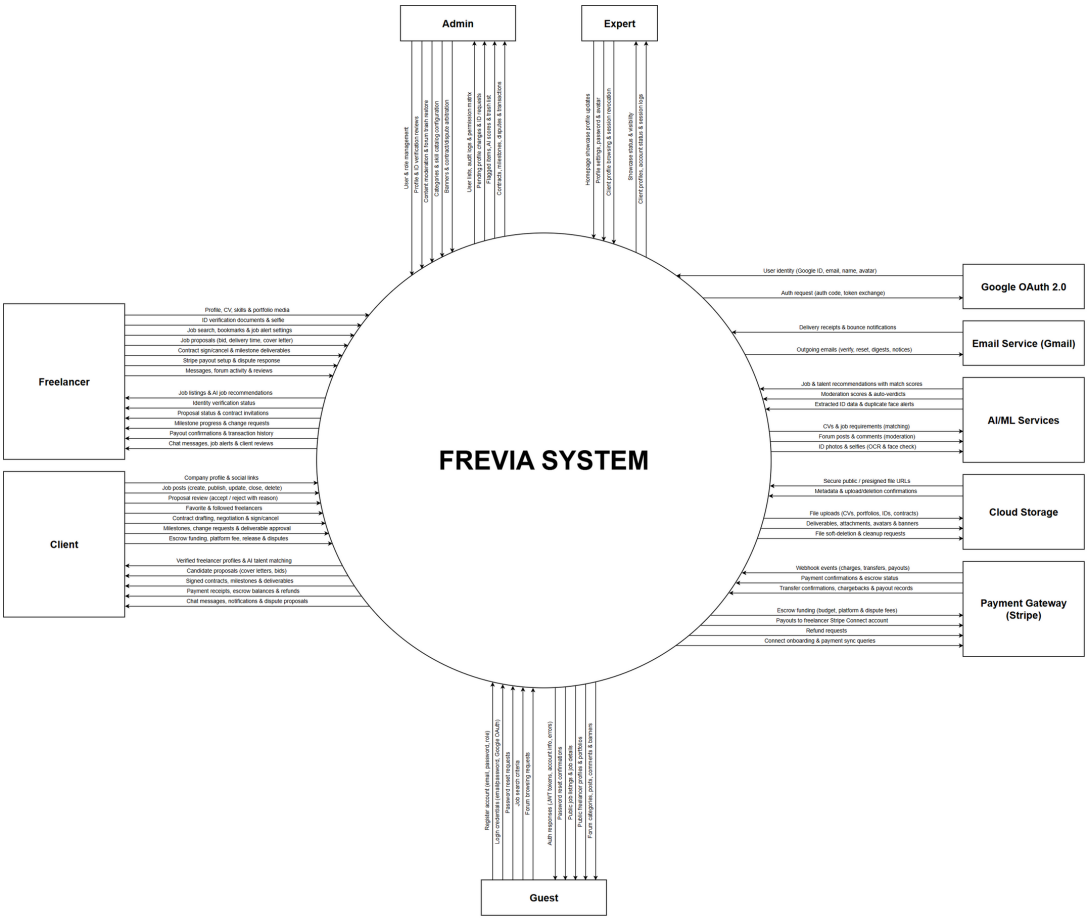


Figure 1: Context diagram

2. User Requirements

2.1 Actors

Table 1: Actors of the Frevia Platform

No	Actor	Description
1	Guest	Guests are unregistered visitors who have not yet created an account on the platform. They can browse publicly available information such as job listings, freelancer profiles, forum content, and platform features. Guests may also register a new account (via email/password or Google OAuth) or authenticate using supported login methods to access platform services.
2	Client	Clients are individuals or organizations seeking professional services from freelancers. They can create and manage job postings, review proposals, hire freelancers, establish contracts, manage escrow payments, monitor project progress, initiate disputes, and provide reviews upon project completion.
3	Freelancer	Freelancers are service providers who offer professional skills and expertise through the platform. They can create and manage profiles, portfolios, and skills; search and apply for jobs; submit proposals; collaborate with clients; complete project tasks and milestones; and receive payments after successful project delivery.
4	Admin	Administrators are responsible for managing and maintaining the platform's operations, security, and compliance. Their responsibilities include user management, role and permission management, identity verification, content moderation, dispute resolution, system configuration, and monitoring platform activities to ensure a secure and reliable environment.
5	Expert	Experts are verified, high-ranking domain specialists featured in the platform's public expert showcase. They possess verified credentials, manage specialized public showcase profiles, consult on high-value client projects, and manage their security sessions and credentials.

Continued on next page

Table 1 continued from previous page

No	Actor	Description
6	System	The internal automated execution engine and background scheduler of the platform. The System autonomously executes scheduled background jobs, manages deadline expirations, triggers automatic state transitions for contracts and proposals, processes automated dispute resolution payouts upon administrative decisions, and dispatches automated notifications.
7	AI Service	The artificial intelligence and machine learning microservice providing automated cognitive capabilities. It analyzes resumes to compute CV-to-job matching scores, evaluates forum posts and comments for toxicity and spam to assist content moderation, and performs optical character recognition (OCR) and face-matching validation on identity verification documents.
8	Payment Gateway (Stripe)	The third-party financial processor (Stripe Checkout and Stripe Connect) powering platform monetary operations. It securely processes client escrow funding, holds funds in escrow during project execution, handles automated payouts to freelancer bank accounts, processes refunds and dispute arbitration fees, and transmits asynchronous webhook events to synchronize payment statuses.
9	Email Service (Gmail / SMTP)	The external transactional email delivery provider (SMTP / Nodemailer). It delivers transactional and security communications to users, including email verification links, multi-factor authentication one-time passwords (OTP), password reset instructions, contract status updates, and dispute notifications.
10	Google OAuth 2.0	Third-party federated identity provider. Enables seamless and secure single sign-on (SSO) authentication, allowing guests to register and log in to the Frevia platform using their Google accounts without requiring separate password management.

Continued on next page

Table 1 continued from previous page

No	Actor	Description
11	Cloud Storage (Cloudinary)	Third-party media storage and content delivery network (CDN) service. Securely stores, optimizes, and delivers user-uploaded digital assets, including user avatars, portfolio media, project deliverable files, and identity verification documents.

2.2 Use Cases

2.2.1 Diagram(s)

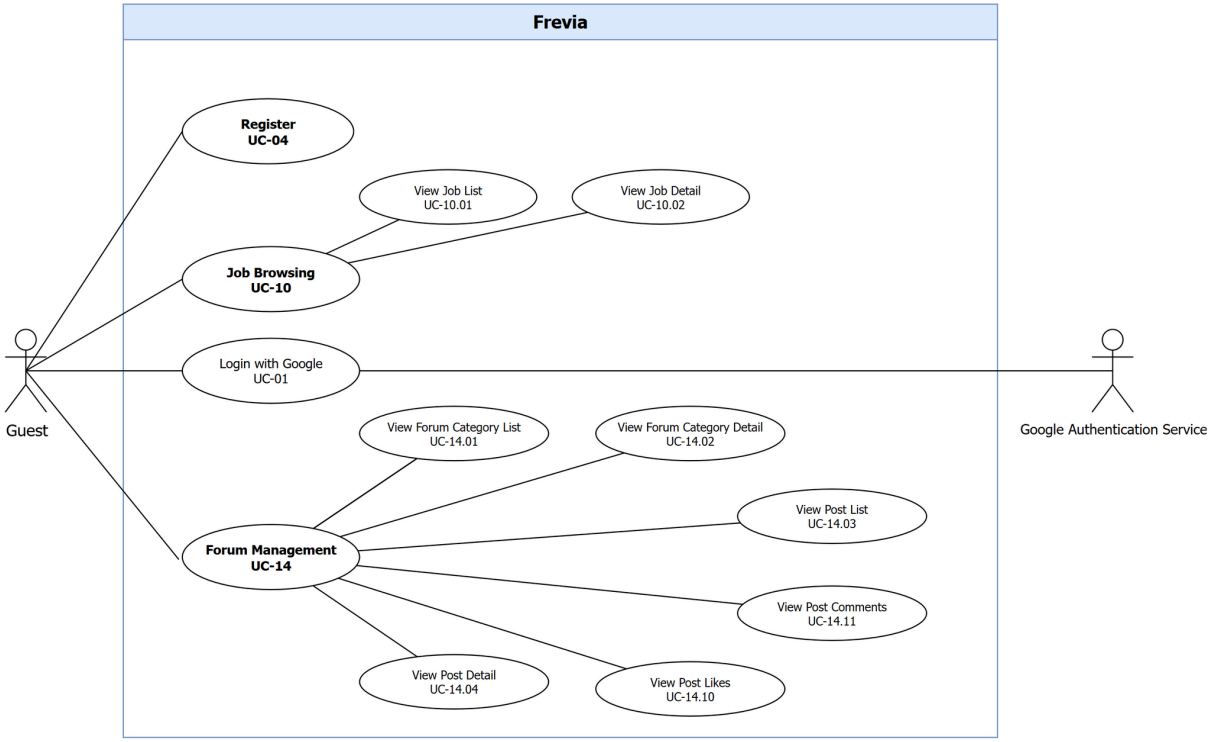


Figure 2: Guest Usecase

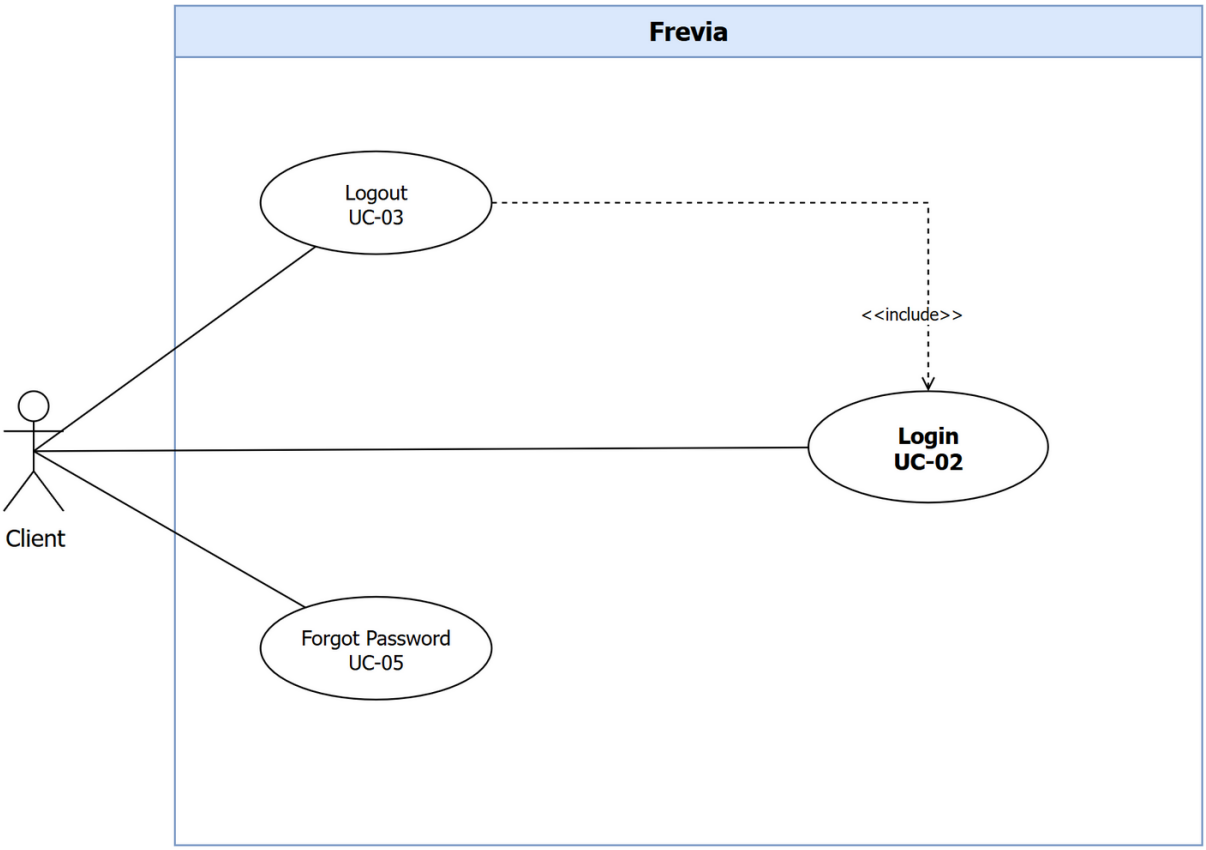


Figure 3: Client Usecase 1/6

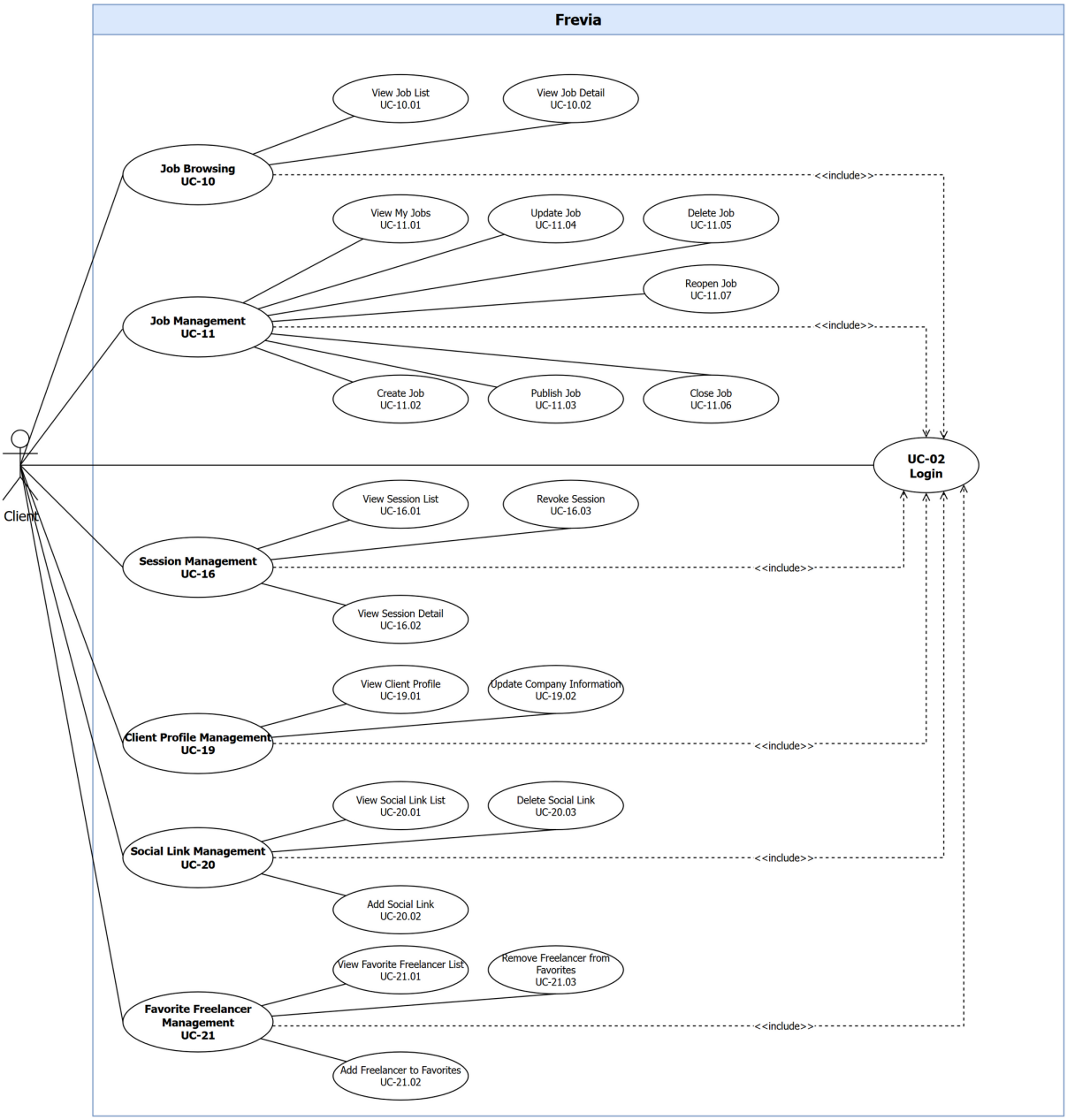


Figure 4: Client Usecase 2/6

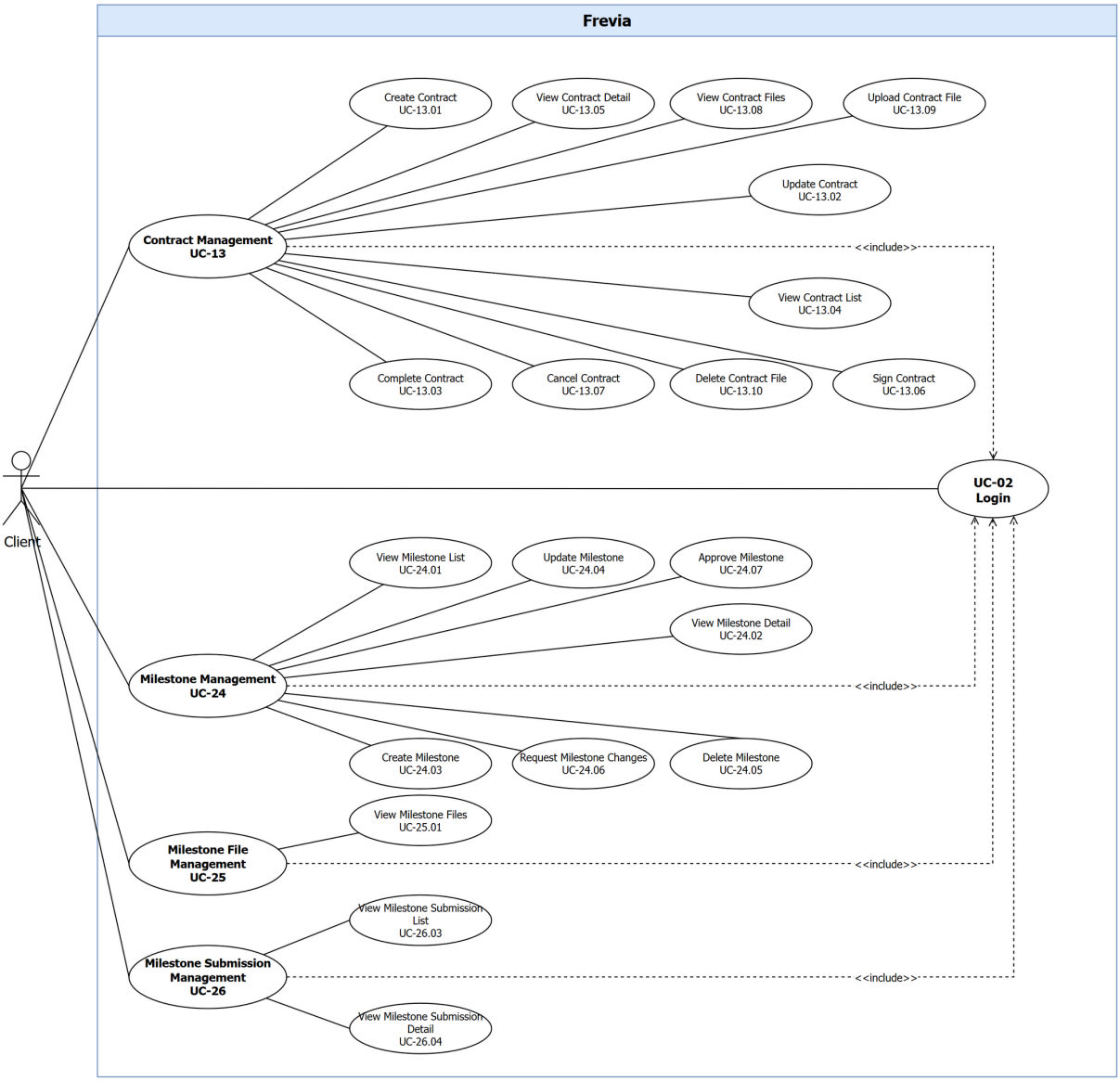


Figure 5: Client Usecase 3/6

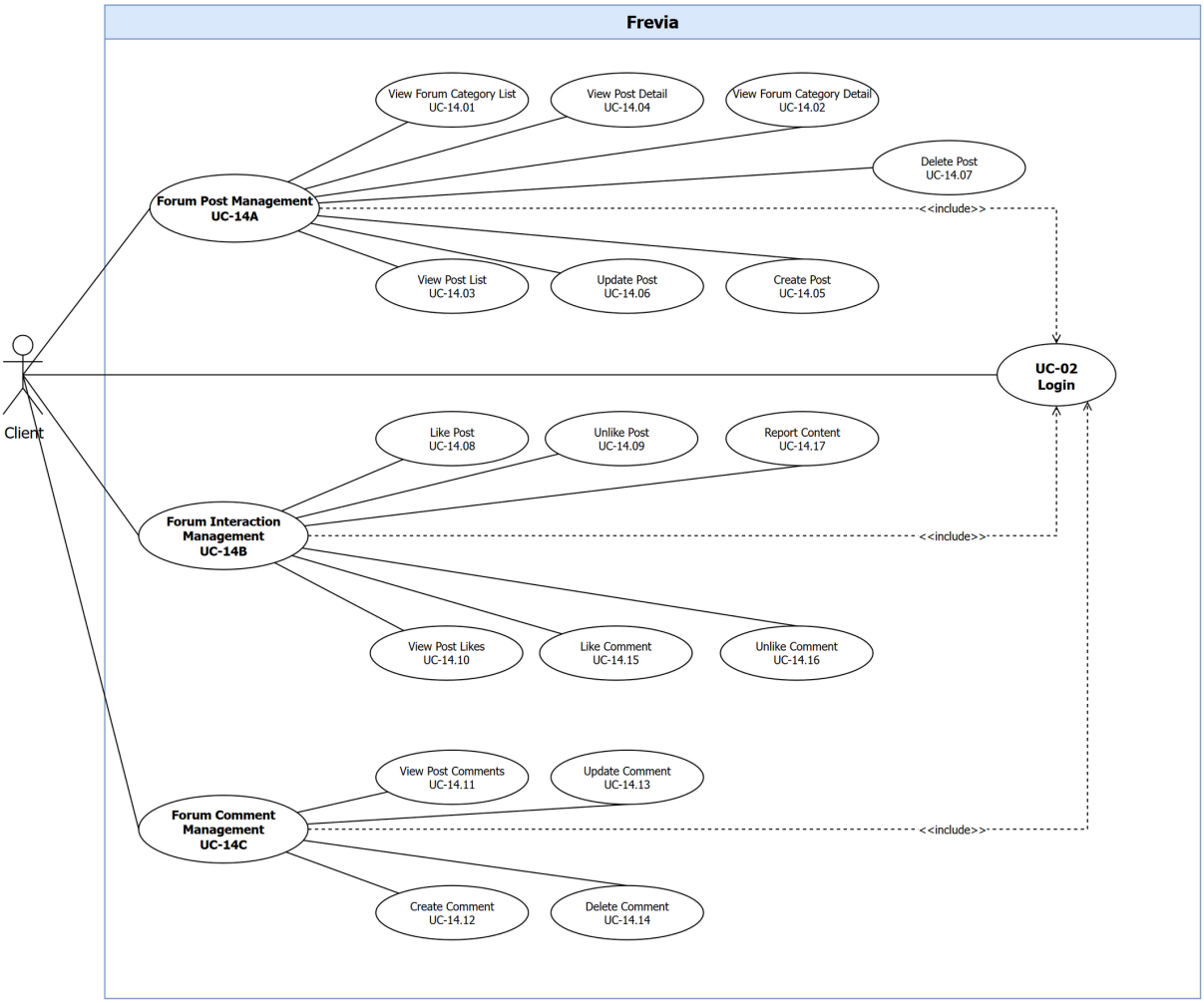


Figure 6: Client Usecase 4/6

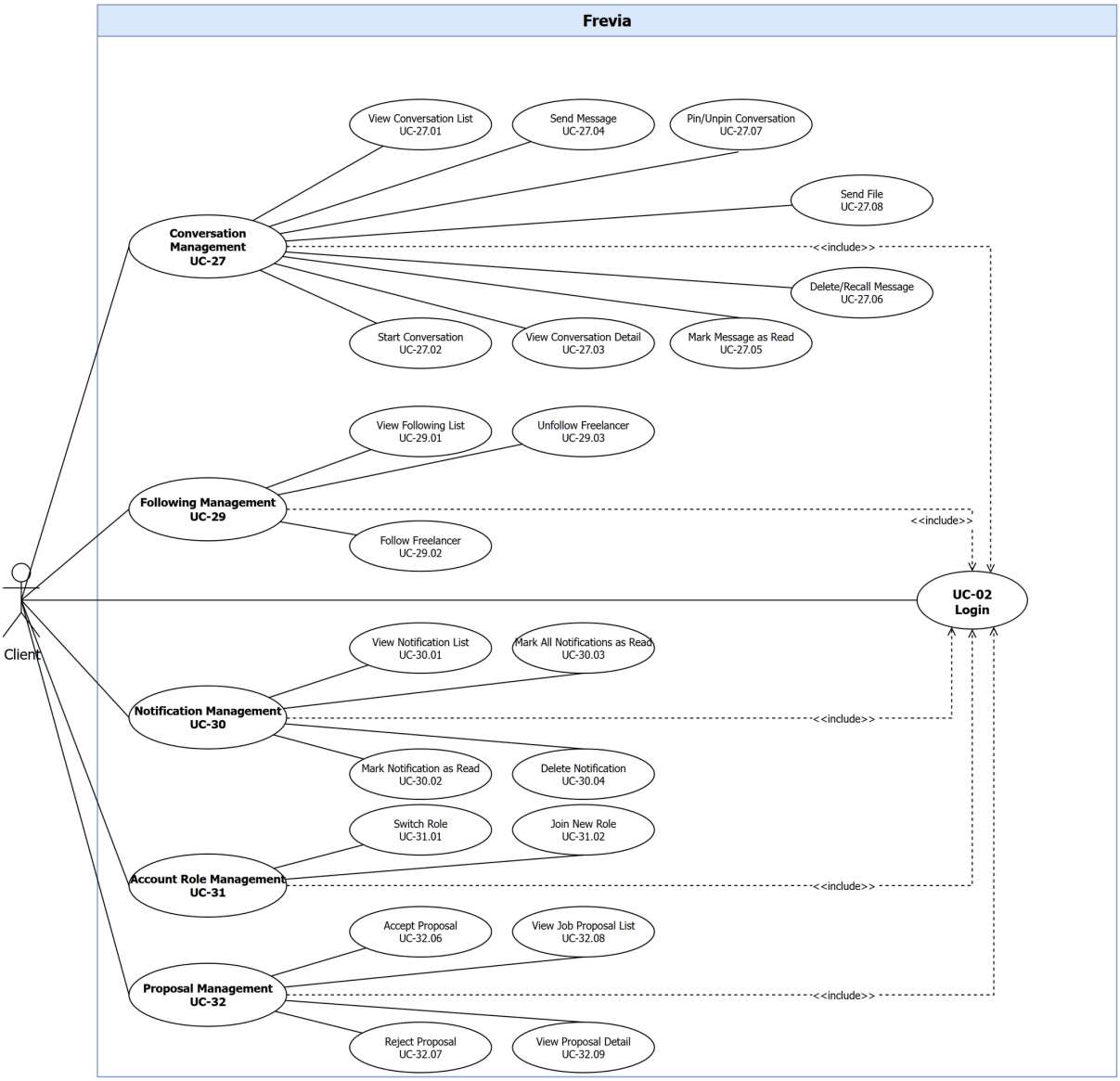


Figure 7: Client Usecase 5/6

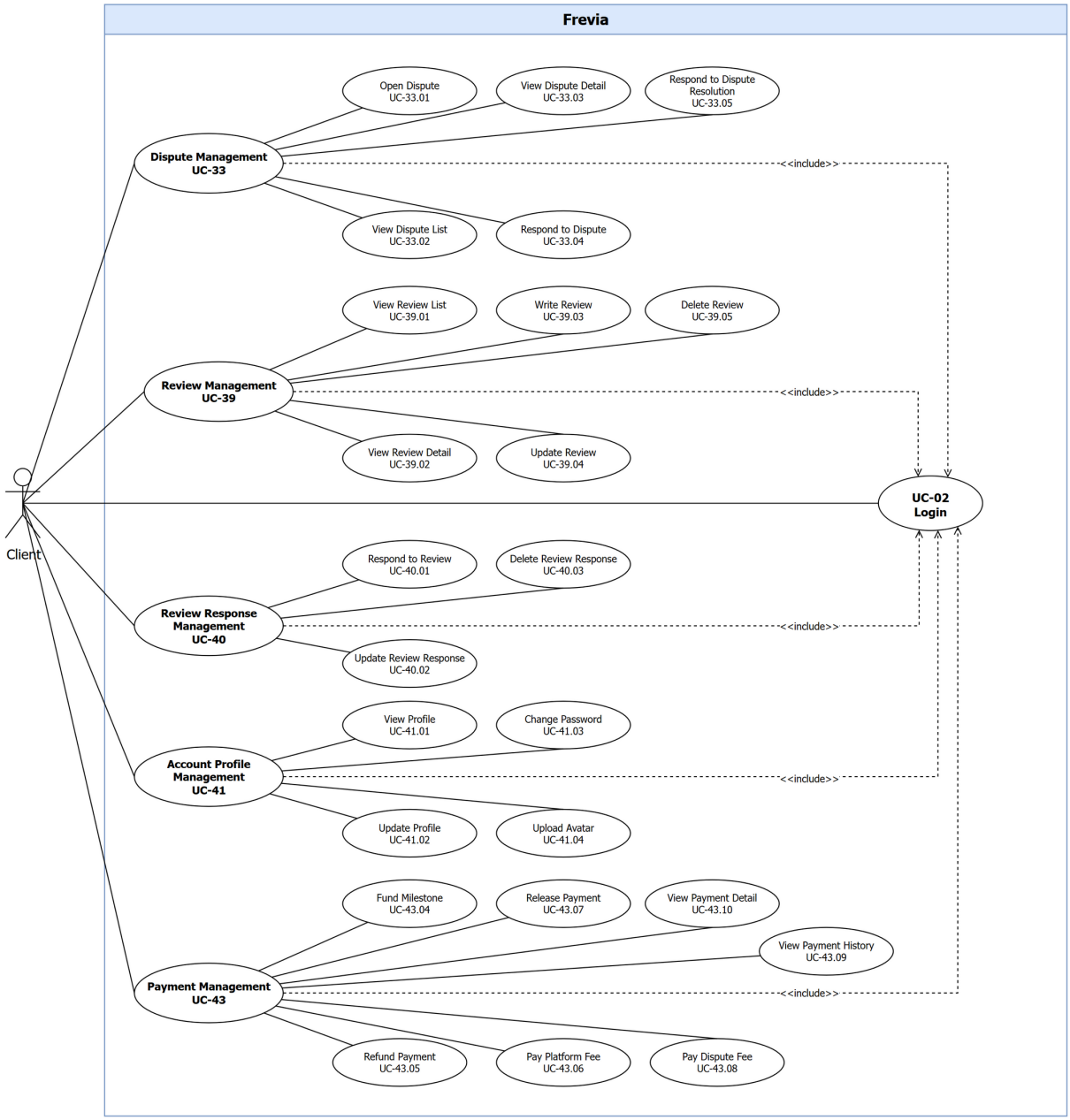


Figure 8: Client Usecase 6/6

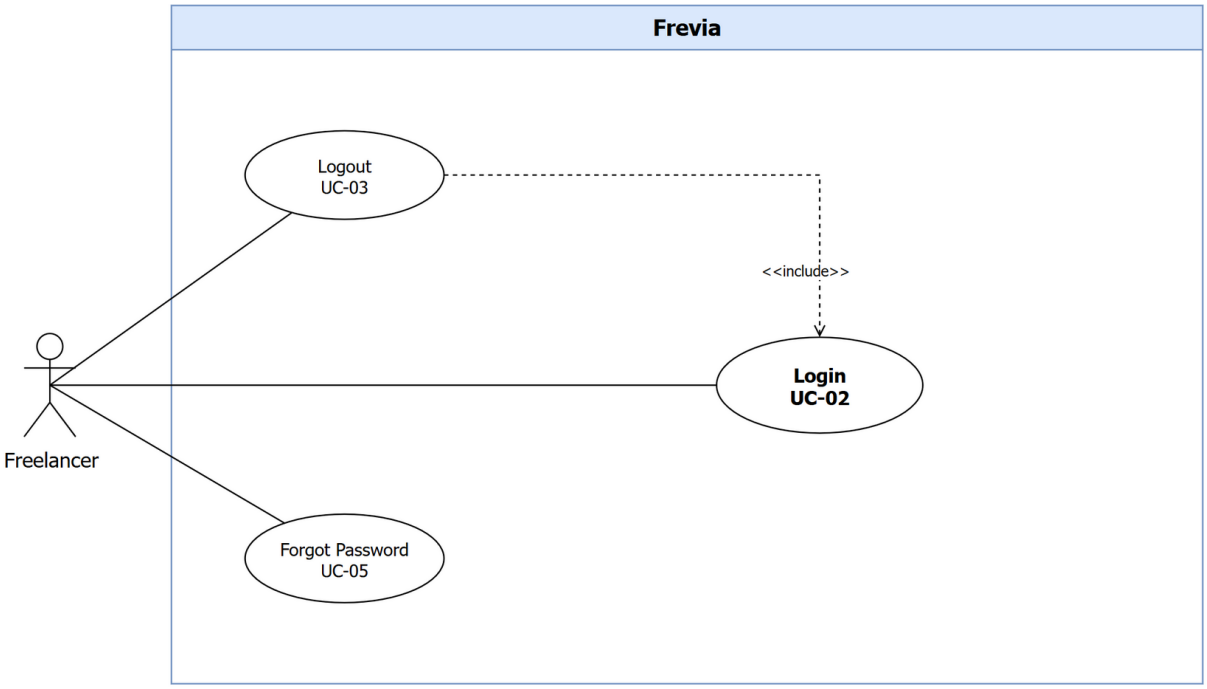


Figure 9: Freelancer Usecase 1/7

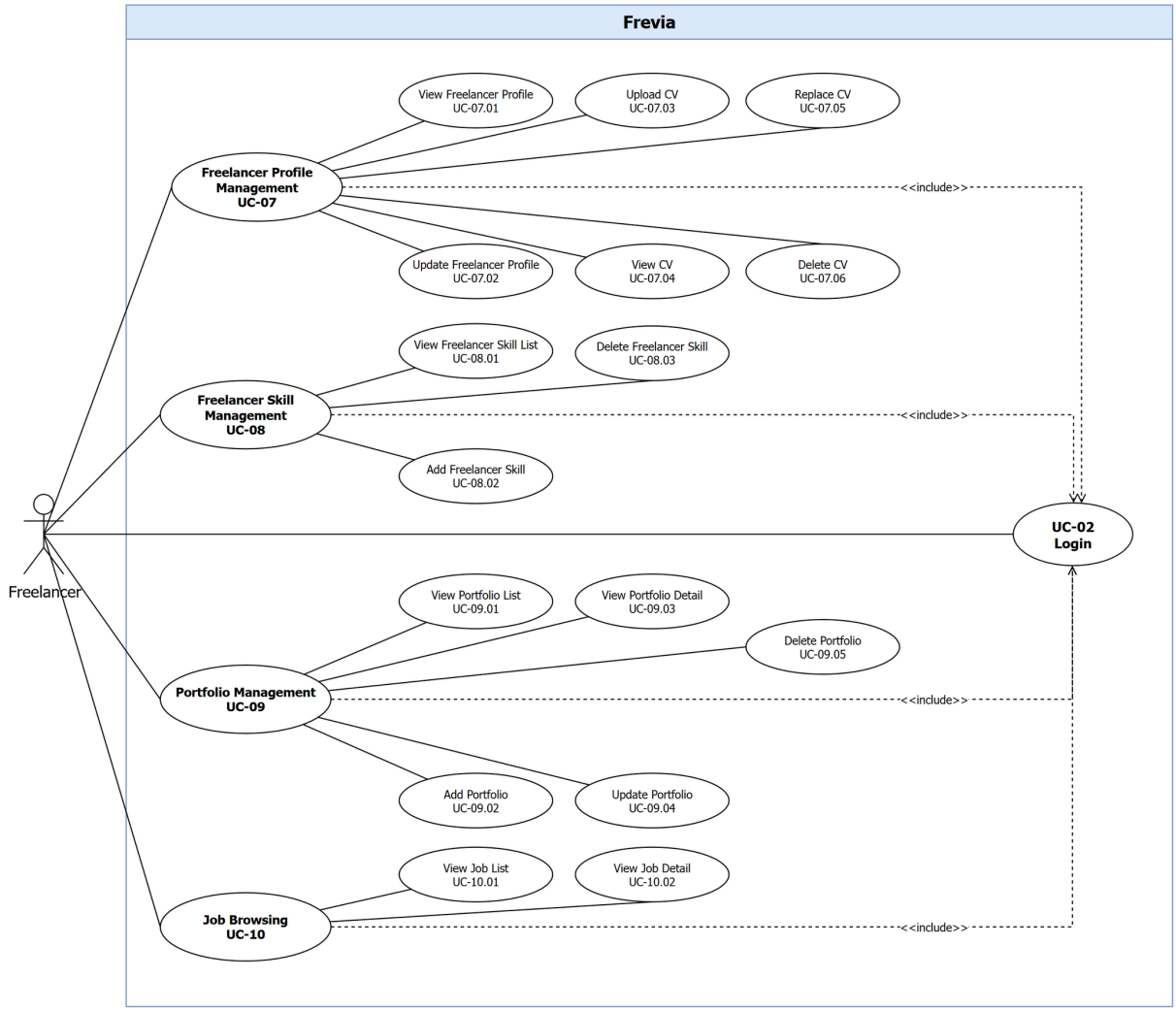


Figure 10: Freelancer Usecase 2/7

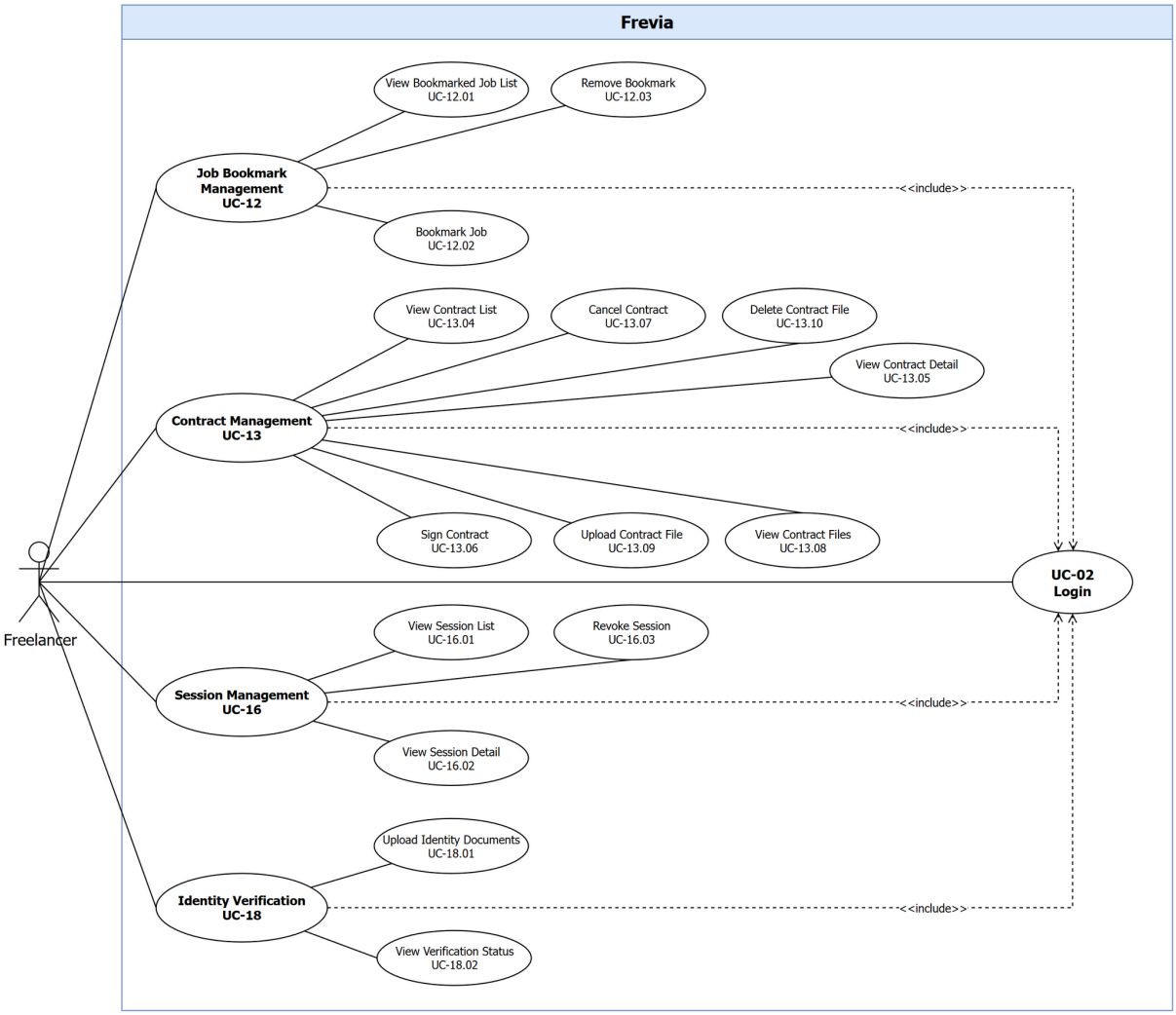


Figure 11: Freelancer Usecase 3/7

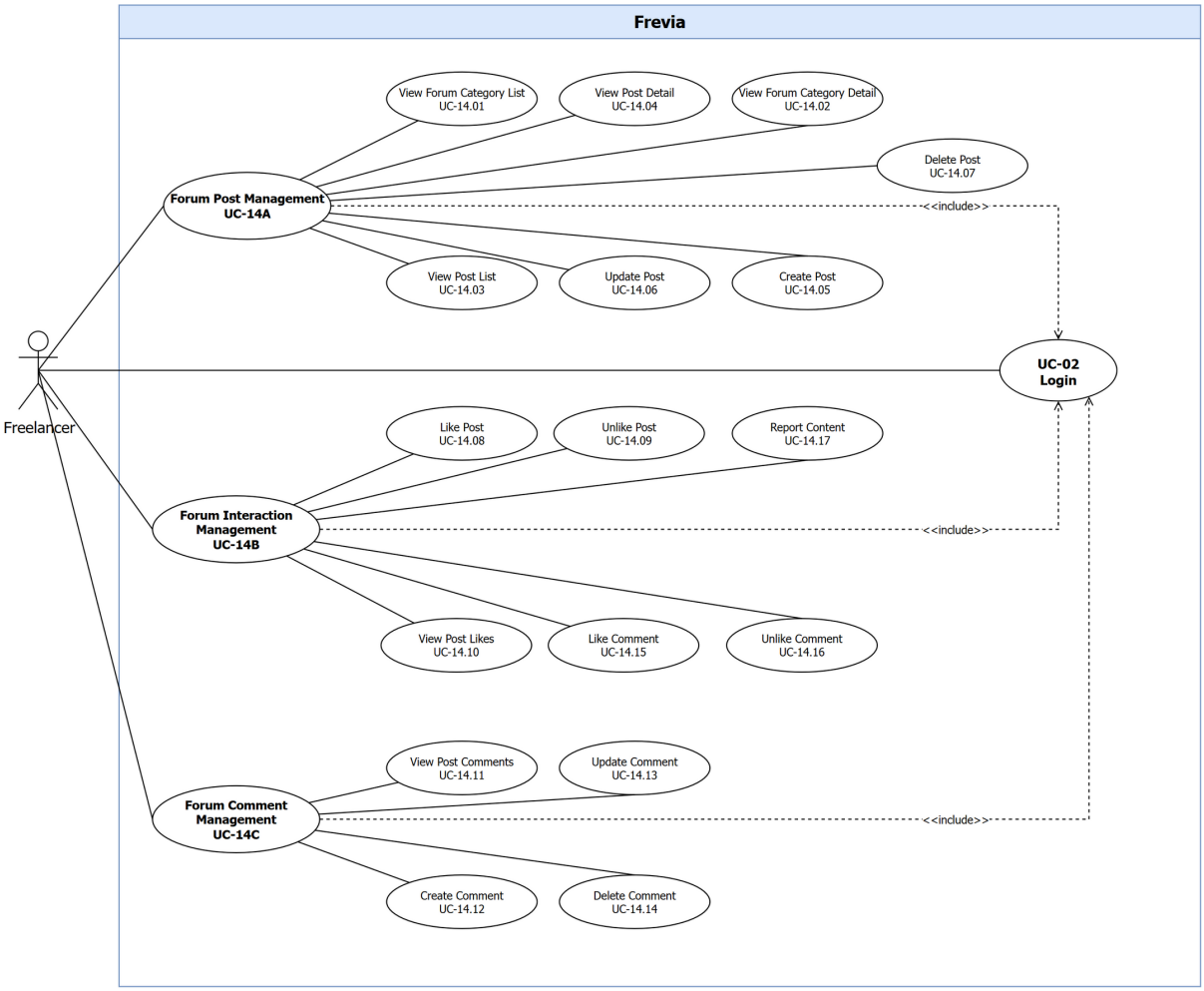


Figure 12: Freelancer Usecase 4/7

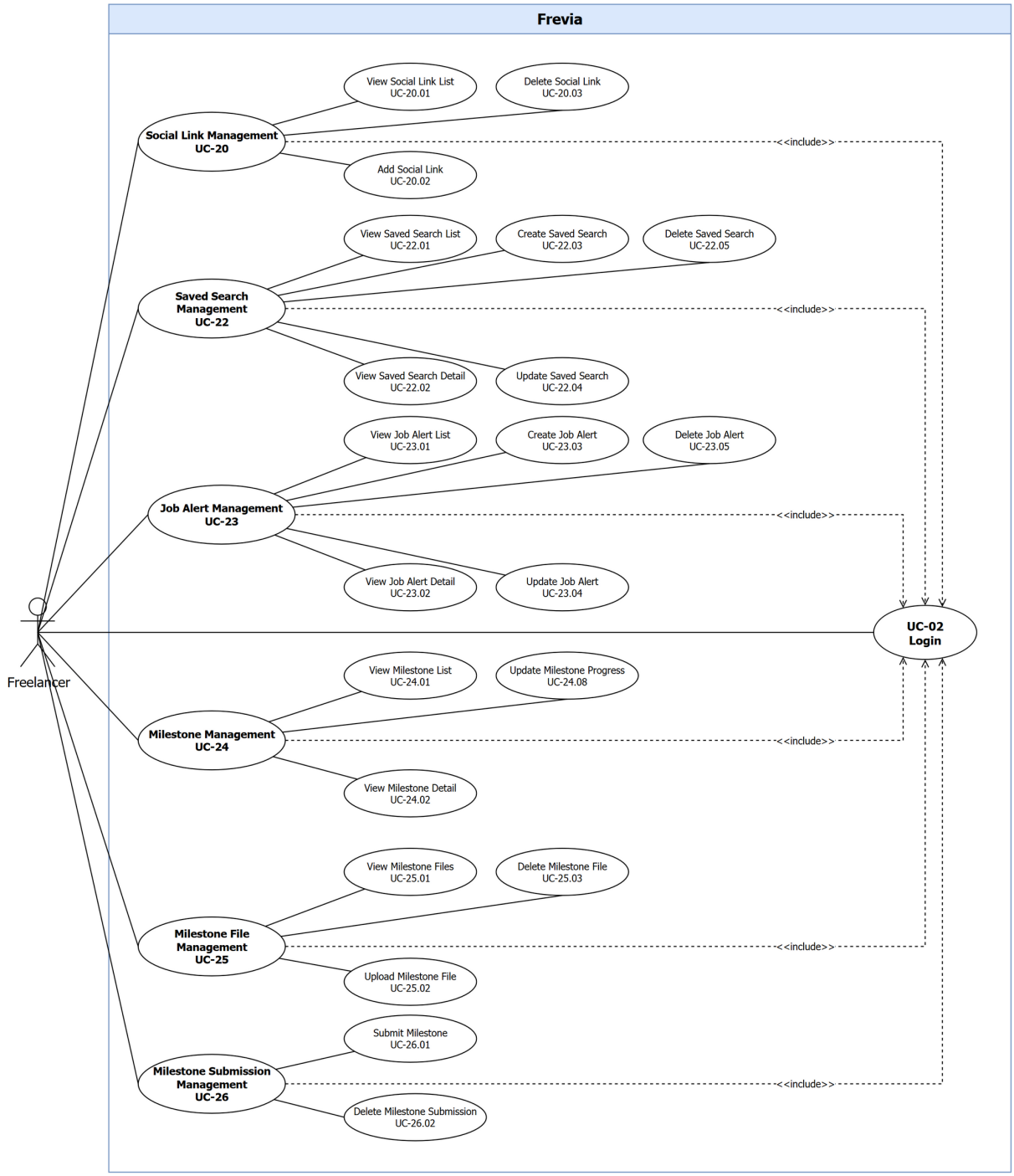


Figure 13: Freelancer Usecase 5/7

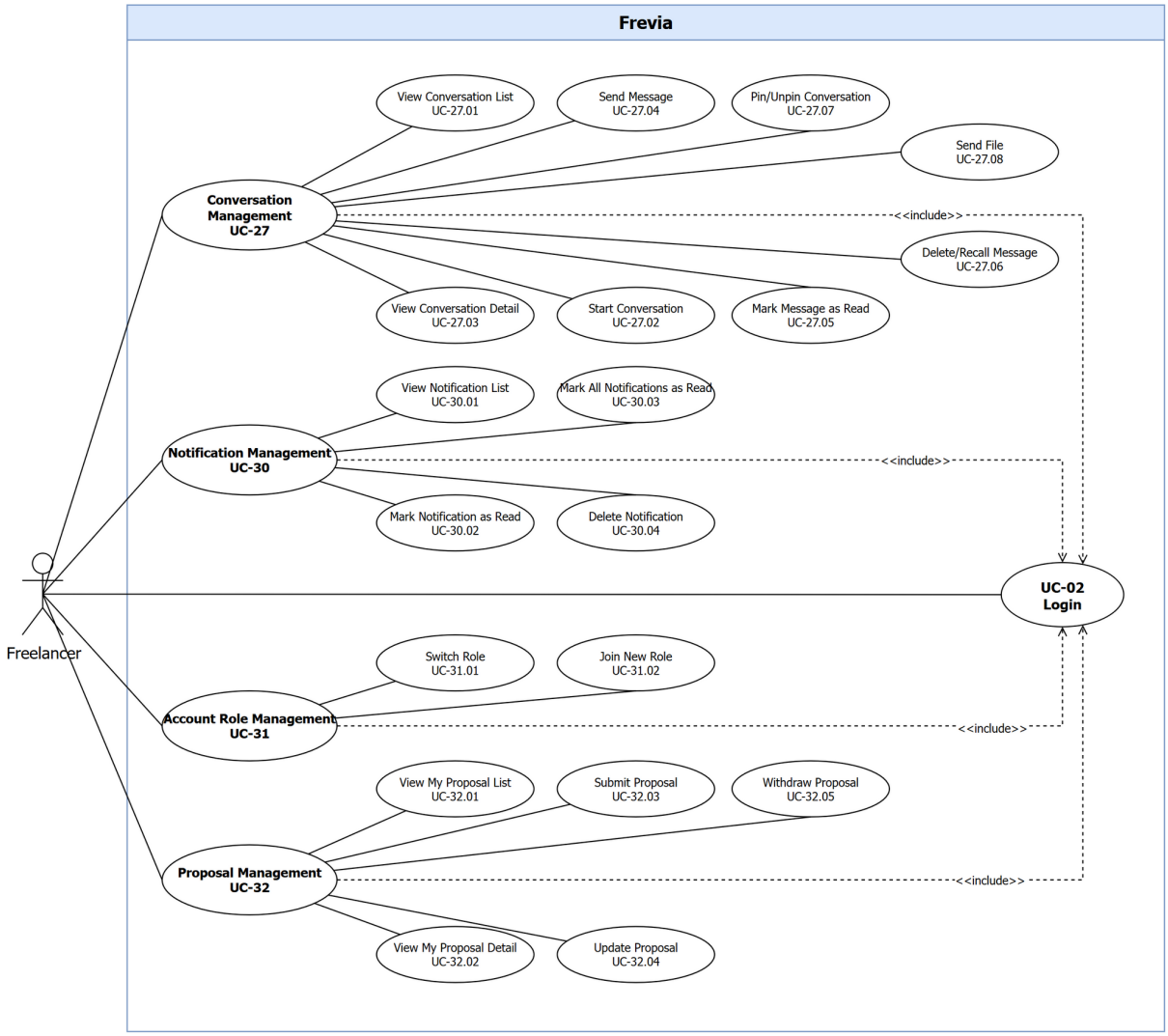


Figure 14: Freelancer Usecase 6/7

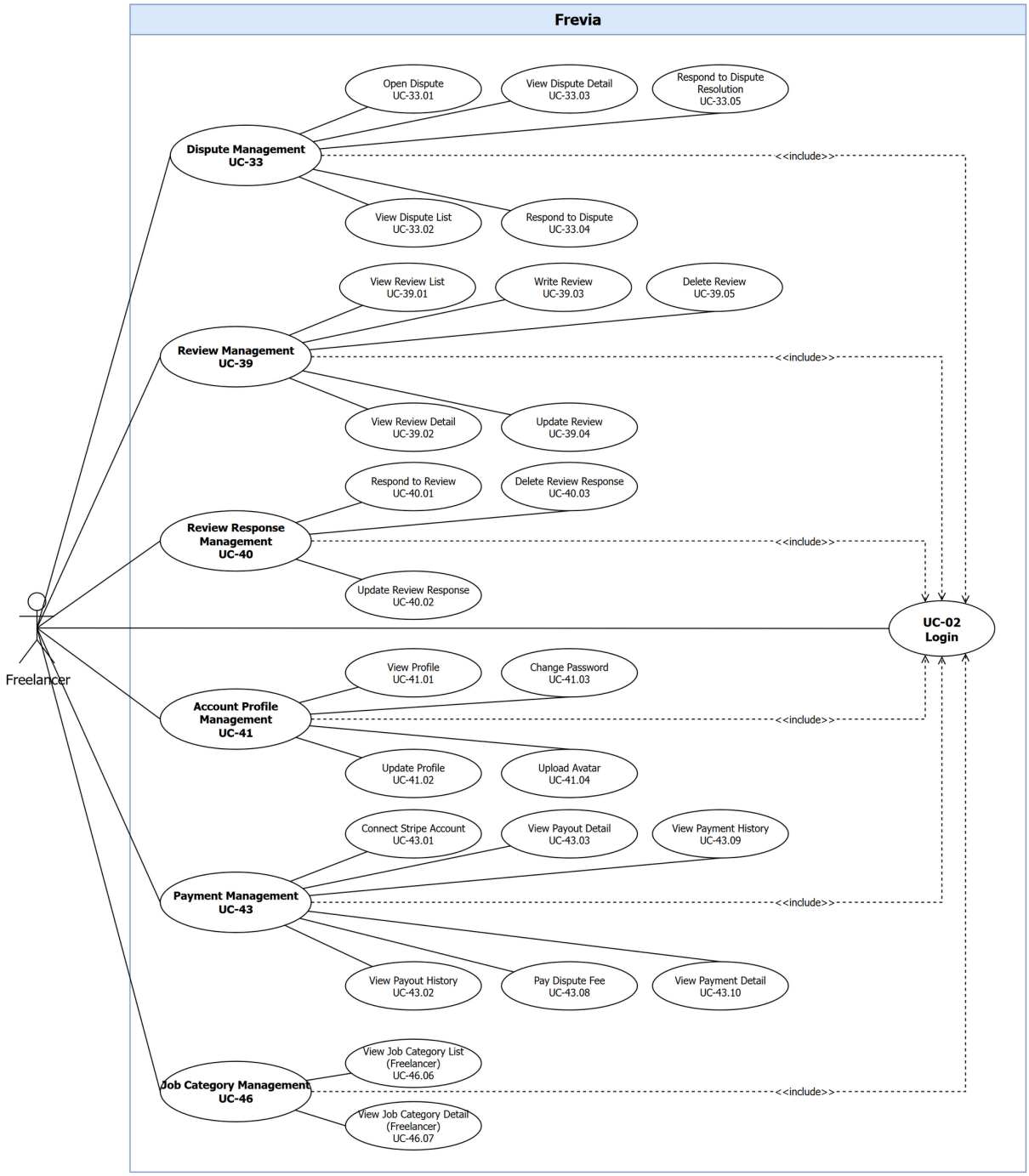


Figure 15: Freelancer Usecase 7/7

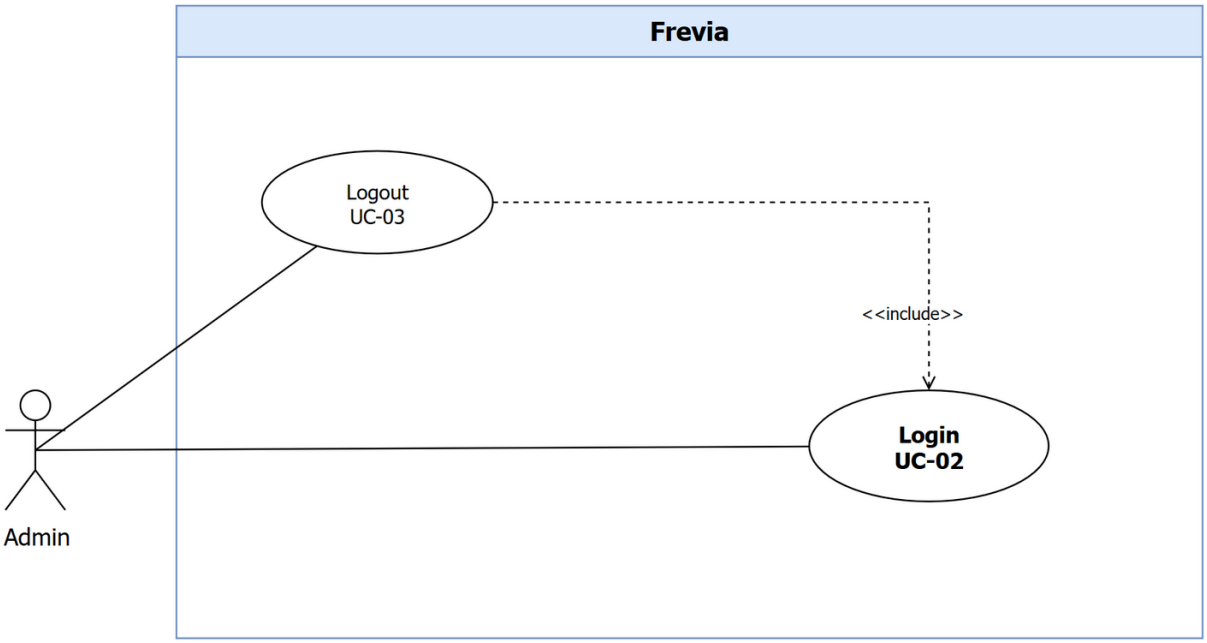


Figure 16: Admin Usecase 1/4



Figure 17: Admin Usecase 2/4

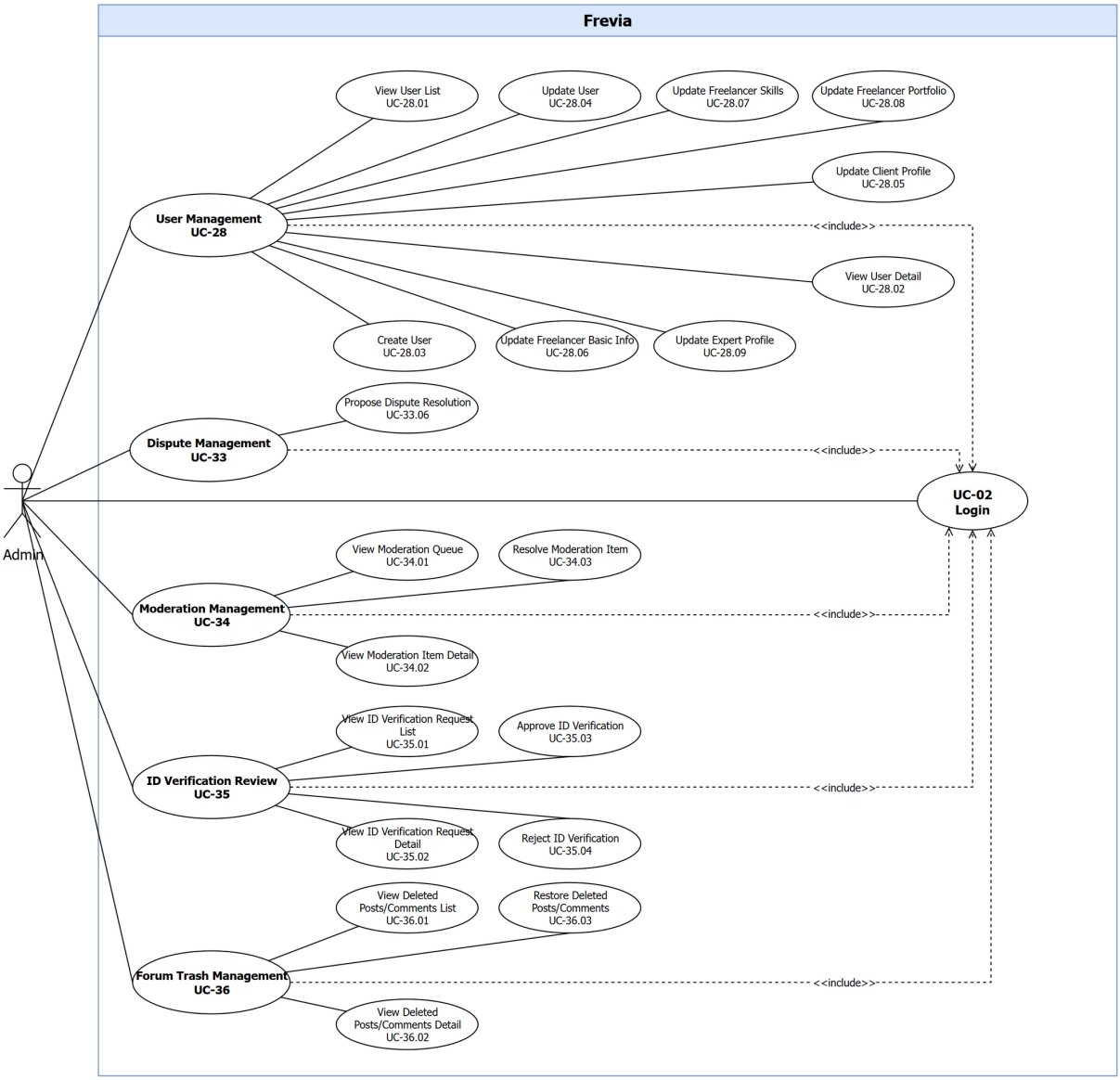


Figure 18: Admin Usecase 3/4

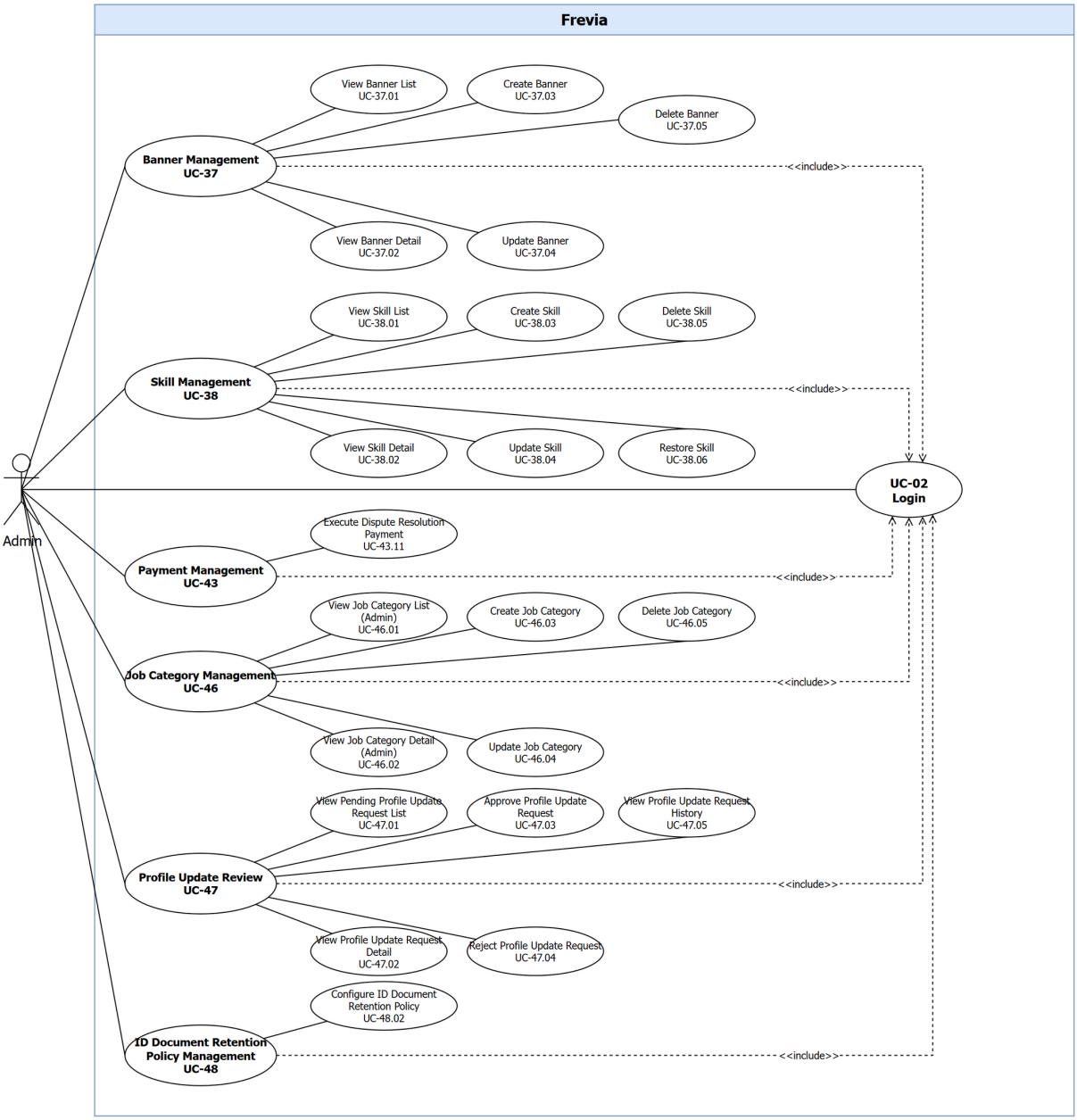


Figure 19: Admin Usecase 4/4

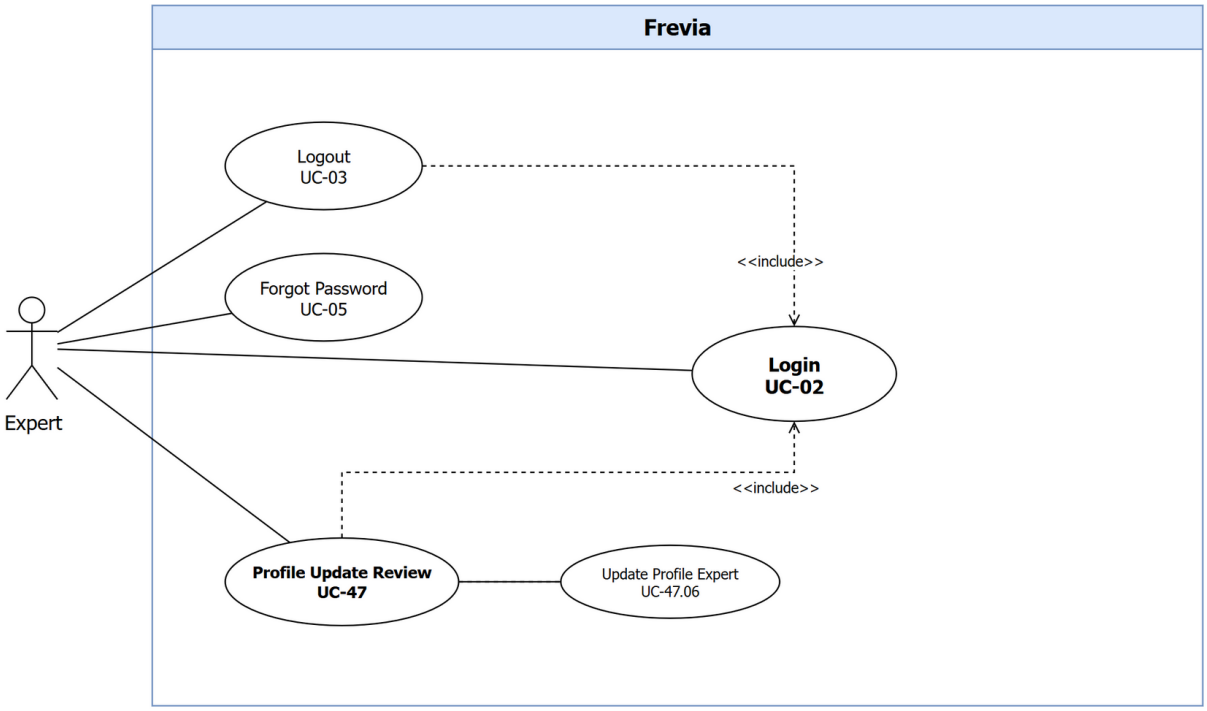


Figure 20: Expert Usecase

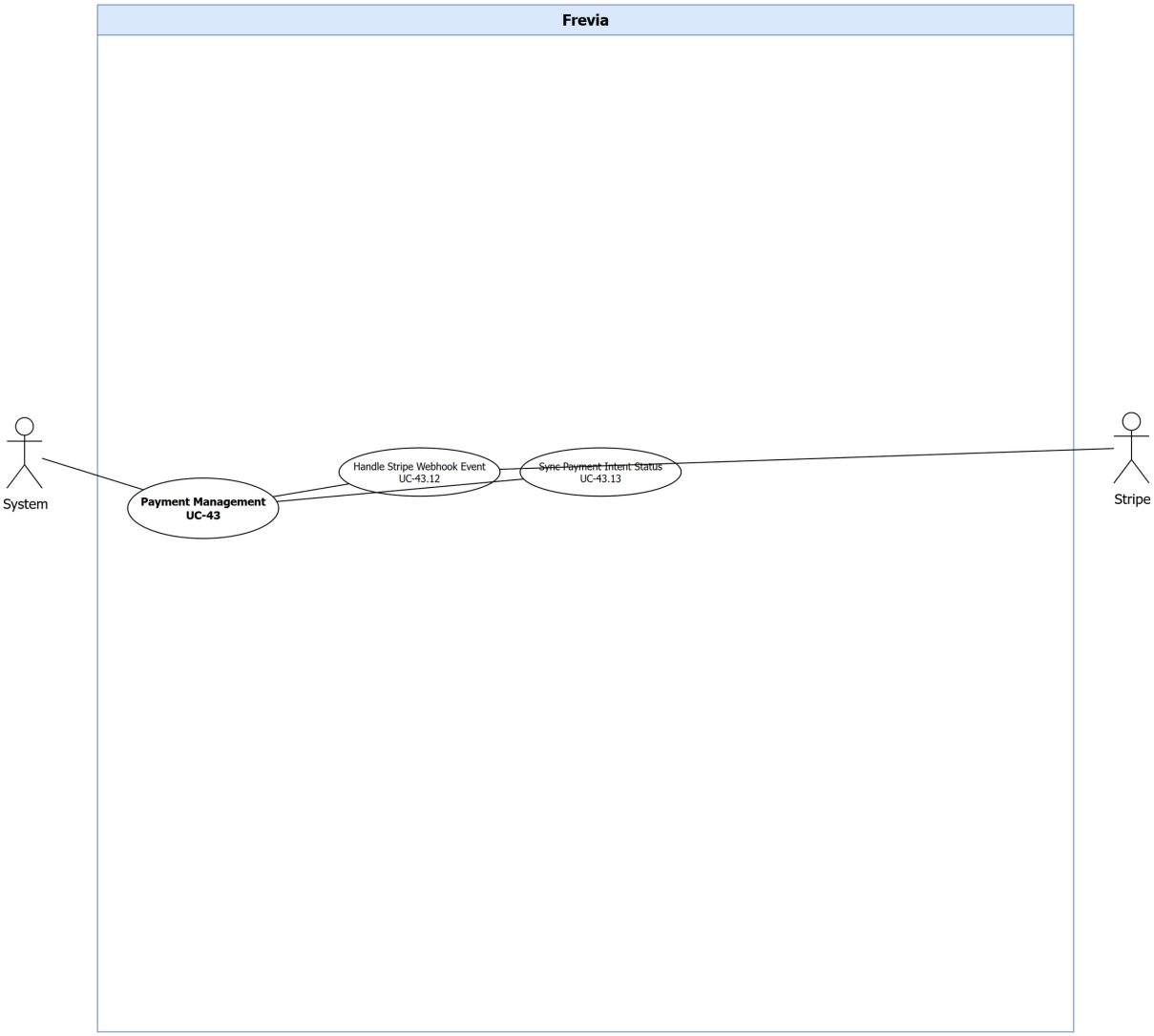


Figure 21: System Usecase

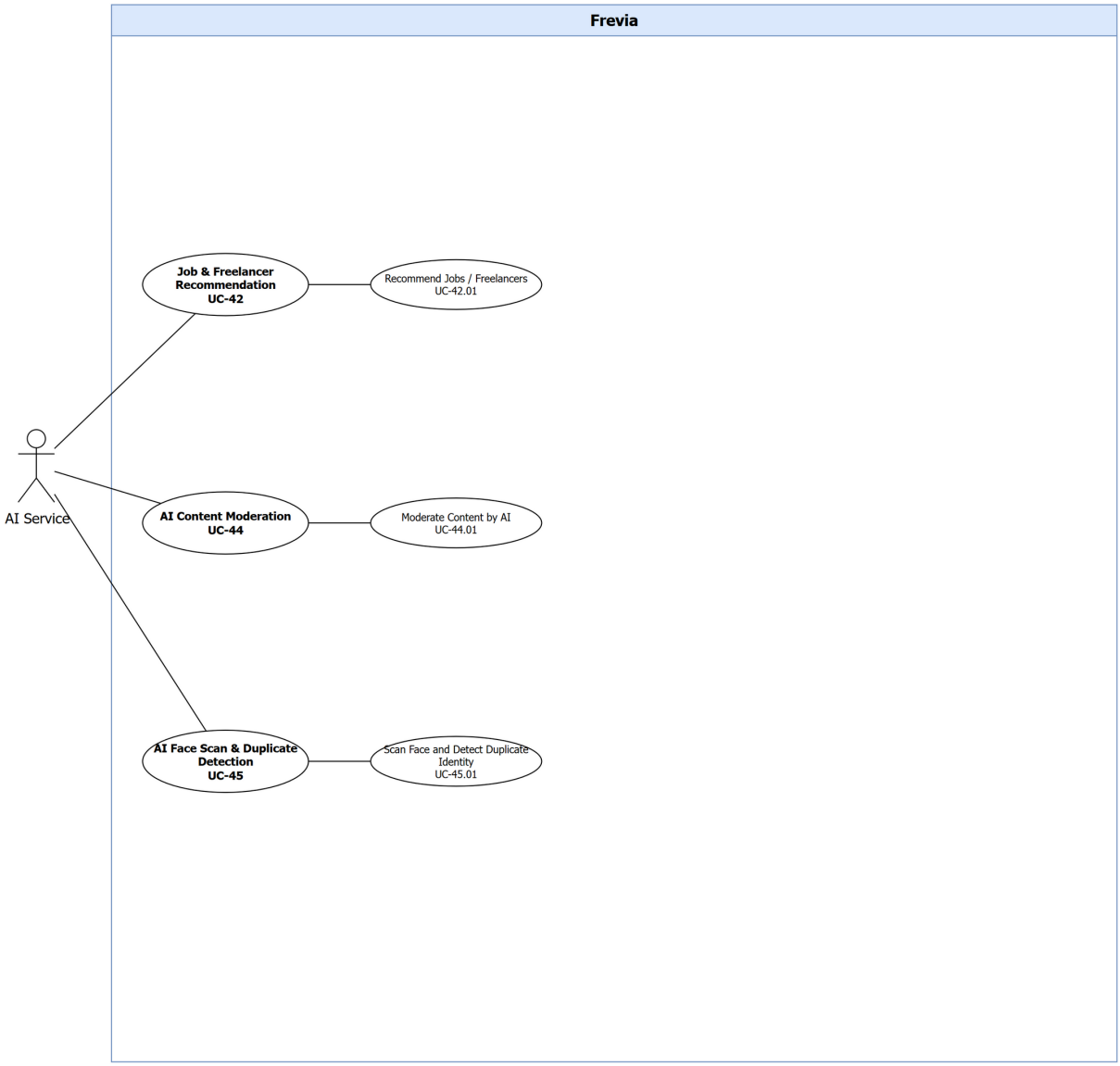


Figure 22: AI Service Usecase

2.2.2 Descriptions

ID	Use Case	Actors	Description
UC-01	Login with Google	Guest	Allows users to login/register using Google OAuth.
UC-02	Login	Guest	Allows users to log in using email/password and JWT authentication.
UC-03	Logout	Client, Expert, Freelancer, Admin	Allows users to logout and terminate authentication session.

Continued on next page

ID	Use Case	Actors	Description
UC-04	Register	Guest	Allows users to create new accounts using email and password.
UC-05	Forgot Password	Guest	Allows users to request password reset via email verification link.
UC-06.01	View Role List	Admin	Retrieve list of all roles along with their assigned permissions.
UC-06.02	View Role Detail	Admin	View detailed information of a specific role including its permissions.
UC-06.03	Create Role	Admin	Create a new role with name, description, and a list of permission IDs.
UC-06.04	Update Role	Admin	Update a role's name, description, and replace all assigned permissions.
UC-06.05	Delete Role	Admin	Delete a role from the system.
UC-06.06	Update Role Permissions	Admin	Assign route access rights to roles.
UC-15.01	View Permission List	Admin	Retrieve all permissions (id, name, path, method, module) for admin to assign to roles.
UC-15.02	View Permission Detail	Admin	View detailed information of a specific permission.
UC-16.01	View Session List	Client, Expert, Freelancer, Admin	View a list of active login sessions.
UC-16.02	View Session Detail	Client, Expert, Freelancer, Admin	View login session information (device, IP address, login time, approximate location, etc.).
UC-16.03	Revoke Session	Client, Expert, Freelancer, Admin	Log out of a specific device remotely.
UC-17.01	Create Forum Category	Admin	Allows admins to create a new forum category.
UC-17.02	Update Forum Category	Admin	Allows admins to update an existing forum category.
UC-17.03	Delete Forum Category	Admin	Allows admins to soft-delete a forum category.

Continued on next page

ID	Use Case	Actors	Description
UC-14.01	View Forum Category List	Admin	Allows admins to view the list of forum categories.
UC-14.02	View Forum Category Detail	Admin	Allows admins to view the details of a specific forum category.
UC-17.06	Restore Forum Category	Admin	Restore a deleted Category forum by setting deletedAt back to null.
UC-03	Job Browsing	Guest, Client, Freelancer	Enables guests to browse and search for available job listings.
UC-03.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters (category, budget, skills, status, keywords) and sorting options.
UC-03.2	View Job Detail	Guest, Client, Freelancer	Get full job details: title, description, budget, skills, client info, proposals count, attachments, timeline.
UC-04	Manage Forum	Guest, Client, Freelancer, Admin	Manages forum operations including viewing, creating, editing, and moderating posts and comments.
UC-04.1	View Forum Categories	Guest, Client, Freelancer, Admin	Displays all available forum categories for browsing.
UC-04.2	View Forum Category Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific forum category and its sub-forums.
UC-04.3	View Post List	Guest, Client, Freelancer, Admin	Retrieves paginated list of posts within a forum category with filters and sorting.
UC-04.4	View Post Detail	Guest, Client, Freelancer, Admin	Displays full post content, author info, creation date, and associated tags.
UC-04.5	View Comments	Guest, Client, Freelancer, Admin	Retrieves all comments related to a specific forum post.
UC-04.6	View Comment Detail	Guest, Client, Freelancer, Admin	Shows detailed information of a specific comment, including content, author, and timestamps.

Continued on next page

ID	Use Case	Actors	Description
UC-04.7	Create Post	Client, Freelancer	Allows freelancers to create new discussion posts or questions in the forum.
UC-04.8	Edit Post	Client, Freelancer	Allows freelancers to edit their own forum posts.
UC-04.9	Delete Post	Client, Freelancer, Admin	Allows post owners or admins to delete a forum post.
UC-04.10	Like Post	Client, Freelancer	Allows freelancers to like a forum post.
UC-04.11	Unlike Post	Client, Freelancer	Allows freelancers to remove their like from a forum post.
UC-04.12	Add Comment	Client, Freelancer	Allows freelancers to add comments to a forum post.
UC-04.13	Edit Comment	Client, Freelancer	Allows freelancers to edit their own comments.
UC-04.14	Delete Comment	Client, Freelancer	Allows comment owners or admins to delete a comment.
UC-04.15	Like Comment	Client, Freelancer	Allows freelancers to like a comment.
UC-04.16	Unlike Comment	Client, Freelancer	Allows freelancers to remove their like from a comment.
UC-04.17	Report Content	Client, Freelancer	Allows freelancers to report inappropriate content in forum posts or comments.
UC-06	Manage Job	Client, Freelancer	Manages job postings including creation, editing, deletion, and status changes.
UC-06.1	View List Job	Guest, Client, Freelancer	Retrieve paginated job list with filters and sorting options.
UC-06.2	Post New Job	Client	Allows clients to create and publish new job postings.
UC-06.3	Edit Job	Client	Allows clients to edit their own job postings.
UC-06.4	Delete Job	Client	Allows clients to delete their own job postings.
UC-06.5	Change Job Status	Client	Allows clients to update job status (Open, In Progress, Closed, Cancelled).
UC-06.6	View List Bookmarked Job	Freelancer	Retrieves list of jobs bookmarked by the freelancer.

Continued on next page

ID	Use Case	Actors	Description
UC-06.7	Bookmark Job	Freelancer	Allows freelancers to save/bookmark a job for later reference.
UC-06.8	Remove Bookmark	Freelancer	Allows freelancers to remove a bookmarked job from their list.
UC-13	Manage Contract	Client, Freelancer, Admin	Manages contractual agreements, milestones, deliverables, and file exchanges.
UC-13.01	Create Contract	Client	Allows clients to create and initiate a new contract with a freelancer.
UC-13.02	Update Contract	Client	Allows clients to update contract terms, budget, milestones, or deadlines before signing.
UC-13.03	Complete Contract	Client, Freelancer	Marks an active contract as completed after all milestones are approved and settled.
UC-13.04	View Contract List	Client, Freelancer, Admin	Retrieves paginated list of contracts with status filters and sorting.
UC-13.05	View Contract Detail	Client, Freelancer, Admin	Displays full contract details including terms, parties, status, and milestones.
UC-13.06	Sign Contract	Freelancer, Client	Allows users to digitally accept and sign a contract to activate it.
UC-13.07	Cancel Contract	Client, Freelancer	Allows either party to cancel or terminate a contract prior to completion.
UC-13.08	View Contract Files	Client, Freelancer, Admin	Manages and accesses all files and documentation associated with a contract.
UC-13.09	Upload Contract File	Client, Freelancer	Allows users to upload documents and deliverables to the contract workspace.
UC-13.10	Delete Contract File	Client, Freelancer, Admin	Allows the uploader or admin to delete files from a contract workspace.
UC-10	Manage Profile	Freelancer	Manages freelancer profile information and settings.

Continued on next page

ID	Use Case	Actors	Description
UC-10.1	View Detailed Freelancer Profile	Freelancer	Displays full freelancer profile including skills, portfolio, experience, and ratings.
UC-10.2	Update Your Detailed Freelancer Profile	Freelancer	Allows freelancers to edit and update their profile information.
UC-11	Manage Skills	Freelancer	Manages freelancer skill set and proficiency levels.
UC-11.1	View List Freelancer Skill	Freelancer	Retrieve the list of skills added by the freelancer.
UC-11.2	Add Freelancer Skill	Freelancer	Allows freelancers to add new skills to their profile.
UC-11.3	Delete Freelancer Skill	Freelancer	Allows freelancers to remove skills from their profile.
UC-12	Manage Portfolio	Freelancer	Manages freelancer portfolio projects and work samples.
UC-12.1	View Portfolio List	Freelancer	Retrieve the list of portfolios created by the freelancer.
UC-12.2	Add New Portfolio	Freelancer	Allows freelancers to add new portfolio items with images and descriptions.
UC-12.3	View Portfolio Detail	Freelancer	Displays full details of a portfolio item including media, description, and technologies used.
UC-12.4	Update Portfolio	Freelancer	Allows freelancers to edit and update their portfolio items.
UC-12.5	Delete Portfolio	Freelancer	Allows freelancers to remove portfolio items.
UC-15	Identity Verification	Freelancer	Manages freelancer identity verification document submission and review.
UC-15.1	Upload Identity Documents	Freelancer	Allows freelancers to upload ID card, passport, or license documents.
UC-15.2	View Verification Status	Freelancer	Displays current identity verification status (Pending, Approved, Rejected).
UC-16	View Client Profile	Guest, Client, Freelancer	Manages client company profile details and public showcase.

Continued on next page

ID	Use Case	Actors	Description
UC-16.1	View Client Profile Detail	Guest, Client, Freelancer	Displays client company details, bio, website, and job history.
UC-16.2	Update Company Information	Client	Allows clients to update company name, bio, website URL, and industry sector.
UC-17	Social Links	Guest, Client, Freelancer	Manages external social and portfolio links on user profiles.
UC-17.1	View Social Links	Guest, Client, Freelancer	Retrieves social links associated with a user profile.
UC-17.2	Add Social Link	Client, Freelancer	Allows users to add external social links to their profile.
UC-17.3	Delete Social Link	Client, Freelancer	Allows users to remove social links from their profile.
UC-18	Favorites	Client	Manages saved favorite freelancer profiles for clients.
UC-18.1	View Favorite Freelancers	Client	Retrieves paginated list of client's favorite freelancers.
UC-18.2	Add to Favorites	Client	Allows clients to save a freelancer profile to their favorites.
UC-18.3	Remove from Favorites	Client	Allows clients to remove a freelancer from their favorites list.
UC-19	Following	Client, Freelancer	Manages followed freelancer network connections.
UC-19.1	View Following List	Client, Freelancer	Retrieves list of freelancers currently followed by the user.
UC-19.2	Follow Freelancer	Client, Freelancer	Allows users to follow a freelancer for activity updates.
UC-19.3	Unfollow Freelancer	Client, Freelancer	Allows users to stop following a freelancer.
UC-20	Manage Notifications	Client, Freelancer, Admin	Manages user in-app notification center and read states.
UC-20.1	View Notification List	Client, Freelancer, Admin	Retrieves user notifications list with unread counter.

Continued on next page

ID	Use Case	Actors	Description
UC-20.2	Mark Notification as Read	Client, Freelancer, Admin	Sets read status of a specific notification to true.
UC-20.3	Mark All Notifications as Read	Client, Freelancer, Admin	Bulk marks all user notifications as read.
UC-20.4	Delete Notification	Client, Freelancer, Admin	Soft-deletes a notification from user's notification list.
UC-21	Account Management	Client, Freelancer	Manages user role switching and complementary role activation.
UC-21.1	Switch Role	Client, Freelancer	Allows dual-role users to switch working context between Client and Freelancer.
UC-21.2	Join New Role	Client, Freelancer	Allows single-role users to activate complementary secondary role.
UC-22	Reviews	Guest, Client, Freelancer, Admin	Manages post-contract ratings and detailed feedback reviews.
UC-22.1	View Reviews List	Guest, Client, Freelancer	Retrieves reviews list for a user profile or contract.
UC-22.2	View Review Detail	Guest, Client, Freelancer	Displays rating breakdown scores and detailed comment text.
UC-22.3	Write Review	Client, Freelancer	Allows contract participants to review each other upon project completion.
UC-22.4	Edit Review	Client, Freelancer	Allows review authors to update ratings or feedback text.
UC-22.5	Delete Review	Client, Freelancer, Admin	Soft-deletes a review record.
UC-23	Manage Response Review	Client, Freelancer, Admin	Manages recipient responses to reviews.
UC-23.1	Respond to Review	Client, Freelancer	Allows review recipients to post a public response.
UC-23.2	Edit Response	Client, Freelancer	Allows response authors to modify response text.

Continued on next page

ID	Use Case	Actors	Description
UC-23.3	Delete Response	Client, Freelancer, Admin	Soft-deletes a review response.
UC-24	Manage Profile	Client, Freelancer, Admin	Manages account credentials, profile details, and avatar.
UC-24.1	Get Profile	Client, Freelancer, Admin	Retrieves general account profile information.
UC-24.2	Update Profile	Client, Freelancer, Admin	Updates general profile details (name, phone, bio, location).
UC-24.3	Change Password	Client, Freelancer, Admin	Allows authenticated users to change password securely.
UC-24.4	Upload Avatar	Client, Freelancer, Admin	Allows users to upload and crop profile avatar image.
UC-25	Profile Update Review	Admin	Manages manual admin review of profile modification requests.
UC-25.1	View Pending Profile Update Requests	Admin	Retrieves list of profile update requests awaiting review.
UC-25.2	View Profile Update Request Detail	Admin	Displays side-by-side diff comparison for profile changes.
UC-25.3	Approve Profile Update Request	Admin	Approves profile modification request and updates live profile.
UC-25.4	Reject Profile Update Request	Admin	Rejects profile request with mandatory reason note.
UC-25.5	View Profile Update Request History	Admin	Views history of processed profile update requests.
UC-25.6	Update Profile Expert	Expert, Admin	Updates expert consultant profile and homepage showcase.

Continued on next page

ID	Use Case	Actors	Description
UC-47.01	View Pending Profile Update Request List	Admin	Retrieves profile update requests awaiting manual review.
UC-47.02	View Profile Update Request Detail	Admin	Displays a request and compares current profile data with proposed changes.
UC-47.03	Approve Profile Update Request	Admin	Approves a request and applies its proposed changes to the live profile.
UC-47.04	Reject Profile Update Request	Admin	Rejects a request with a reason while preserving the current profile data.
UC-47.05	View Profile Update Request History	Admin	Displays previously approved and rejected profile update requests.
UC-47.06	Update Profile Expert	Expert	Allows an Expert to edit the profile used in the public expert experience.
UC-07.03	Upload CV	Freelancer	Uploads a supported CV file and associates it with the Freelancer profile.
UC-07.04	View CV	Freelancer	Views or downloads the CV currently associated with the Freelancer profile.
UC-07.05	Replace CV	Freelancer	Replaces the existing CV with a newly uploaded supported file.
UC-07.06	Delete CV	Freelancer	Removes the CV associated with the Freelancer profile.
UC-33	Manage Dispute	Client, Freelancer, Admin	Manages formal dispute arbitration on funded contract milestones.
UC-33.01	Open Dispute	Client, Freelancer	Allows a client or freelancer to initiate a formal dispute on a funded milestone.
UC-33.02	View Dispute List	Client, Freelancer, Admin	Retrieves paginated overview of disputes with status filtering and deadline badges.
UC-33.03	View Dispute Detail	Client, Freelancer, Admin	Displays full dispute case records, claims, evidence, fees, and resolutions.
UC-33.04	Respond to Dispute	Client, Freelancer	Allows the respondent to submit a rebuttal statement and counter-evidence.

Continued on next page

ID	Use Case	Actors	Description
UC-33.05	Respond to Dispute Resolution	Client, Freelancer	Allows involved parties to accept or reject the arbitrator's proposed decision.
UC-33.06	Propose Dispute Resolution	Admin	Allows an administrator/arbitrator to evaluate claims and propose a fund split decision.
UC-43	Manage Payment	Client, Freelancer, Admin	Handles platform financial transactions, escrow deposits, payouts, and fees via Stripe.
UC-43.01	Connect Stripe Account	Freelancer	Connects a freelancer's bank account via Stripe Connect Express to receive payouts.
UC-43.02	View Payout History	Freelancer, Admin	Retrieves paginated history of all payout transfers sent to the freelancer's bank.
UC-43.03	View Payout Detail	Freelancer, Admin	Displays detailed ledger breakdown, fees, and arrival estimates for a payout transfer.
UC-43.04	Fund Milestone	Client	Deposits milestone funds into platform escrow using credit card or saved payment methods.
UC-43.05	Refund Payment	Client, Admin	Returns escrowed milestone funds to the client upon contract cancellation or dispute.
UC-43.06	Pay Platform Fee	Client	Processes the platform engagement fee upon contract initiation via Stripe.
UC-43.07	Release Payment	Client	Authorizes the release of escrowed milestone funds to the freelancer upon work approval.
UC-43.08	Pay Dispute Fee	Client, Freelancer	Processes arbitration fee payment within the 24-hour dispute window via Stripe.
UC-43.09	View Payment History	Client, Freelancer, Admin	Displays a comprehensive, paginated ledger of all account financial transactions.
UC-43.10	View Payment Detail	Client, Freelancer, Admin	Shows itemized receipt details, fees, and downloadable invoice for a transaction.
UC-43.11	Execute Dispute Resolution Payment	Admin, System	Executes split payouts (client refund and freelancer transfer) upon dispute finalization.

Continued on next page

ID	Use Case	Actors	Description
UC-43.12	Handle Stripe Webhook Event	Stripe, System	Listens for and verifies asynchronous Stripe webhook events to reconcile database state.
UC-43.13	Sync Payment Intent Status	Client, System	Synchronizes real-time payment intent and checkout status directly with the Stripe API.
UC-46	Job Category Management	Admin, Freelancer	Provides administrative category management and freelancer categorization for jobs.
UC-46.01	View Job Category List	Admin	Allows an administrator to view, search, and filter job categories with pagination.
UC-46.02	View Job Category Detail	Admin	Displays detailed information of a specific job category including associated jobs.
UC-46.03	Create Job Category	Admin	Allows an administrator to create a new job category with name, description, and status.
UC-46.04	Update Job Category	Admin	Allows an administrator to modify existing job category details and status.
UC-46.05	Delete Job Category	Admin	Allows an administrator to soft-delete a job category after verifying active job dependencies.
UC-46.06	Restore Job Category	Admin	Allows an administrator to restore a soft-deleted job category.
UC-46.07	View Job Category List	Freelancer	Allows freelancers to browse active job categories when filtering available jobs.
UC-46.08	View Job Category Detail	Freelancer	Allows freelancers to view category information and open jobs under that category.

3. Functional Requirements

3.1 System Functional Overview

3.1.1 Screens Flow

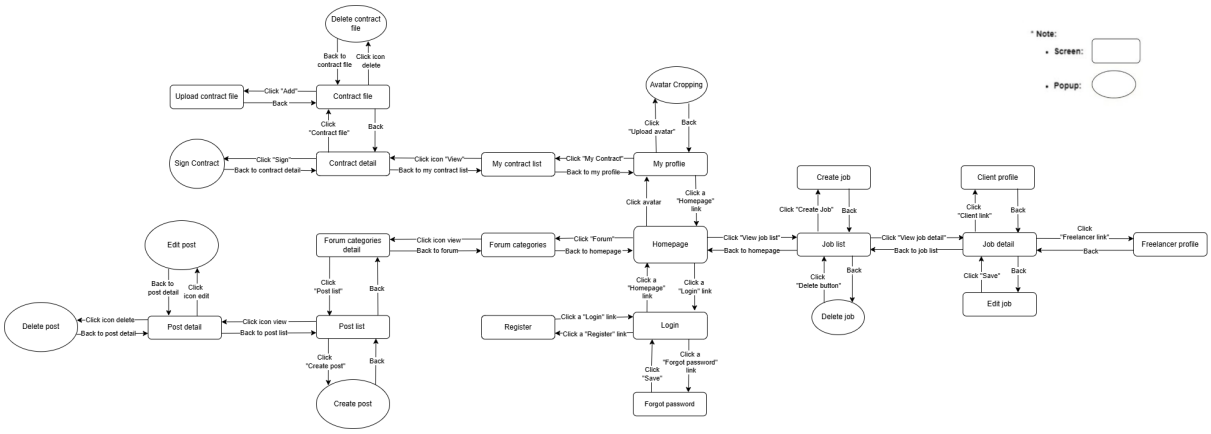


Figure 23: Screen Flow of Client

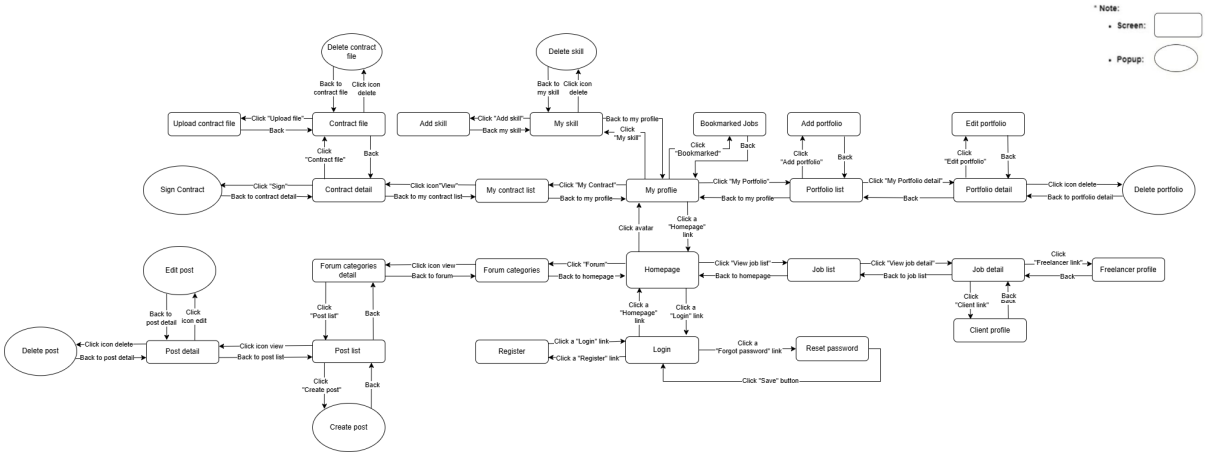


Figure 24: Screen Flow of Freelancer

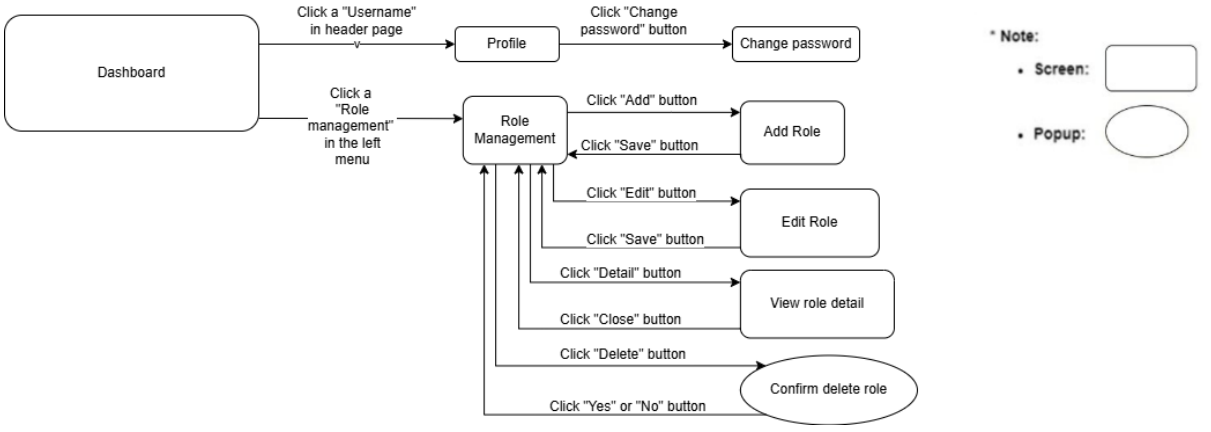


Figure 25: Screen Flow of Admin

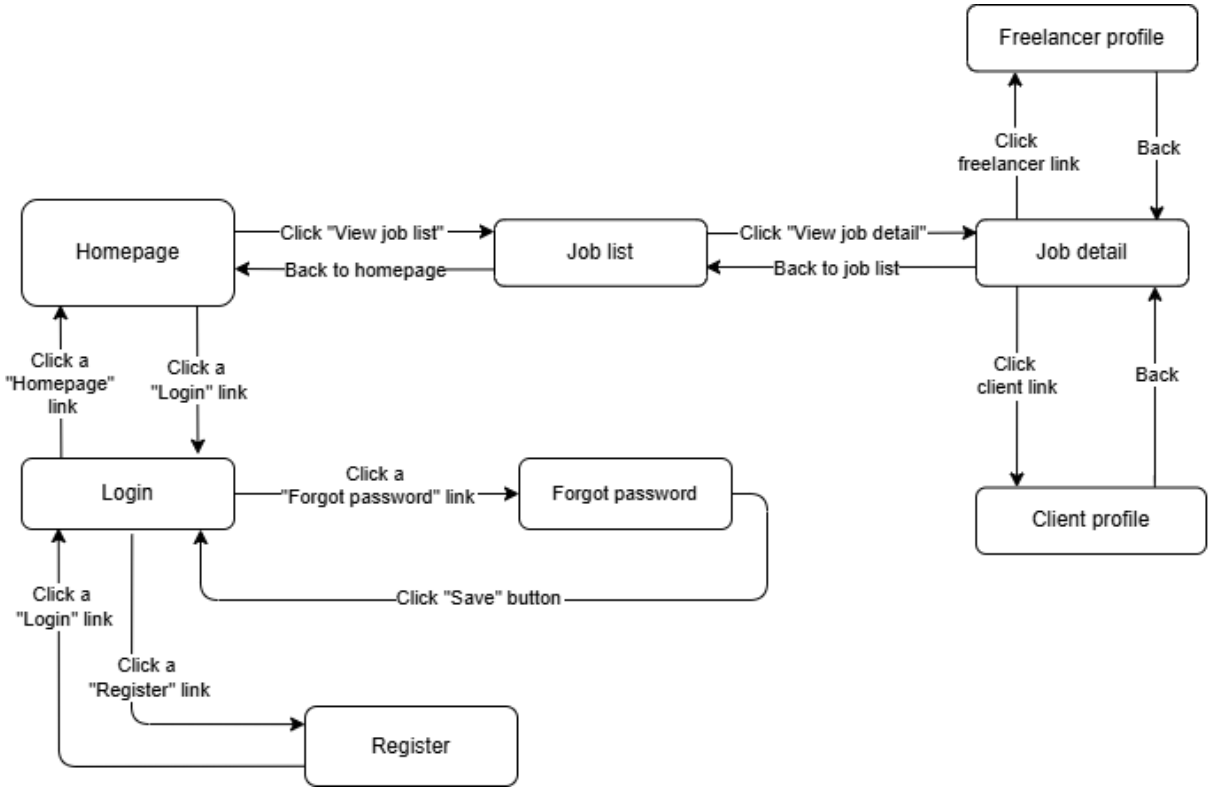


Figure 26: Screen Flow of Guest

3.1.2 Screen Descriptions

No	Actor	Screen Name	Description
1	Guest	Homepage	The main landing page introducing the platform and its features.
2		Login	Allows users to log in using email/password and JWT authentication.
3		Forgot Password	Allows users to request a password reset via email verification link.
4		Register	Allows users to create new accounts using email and password.
5		Job List	Displays a list of available jobs with search and filter capabilities.
6		Job Detail	Displays full details of a specific job posting.
7		Freelancer Profile	Displays a freelancer's public profile including skills and portfolio.
8		Client Profile	Displays a client's public profile including posted jobs and reviews.
9	Freelancer	Google Login	Allows users to login/register using Google OAuth.
10		Login	Allows users to log in using email/password and JWT authentication.
11		Logout	Allows users to logout and terminate authentication session.
12		Register	Allows users to create new accounts using email and password.
13		Forgot Password	Allows users to request password reset via email verification link.
14		My Profile	Main dashboard for freelancer to manage their skills, contracts, and portfolio.

No	Actor	Screen Name	Description
15		My Skill	Displays the list of skills added by the freelancer.
16		Add Skill	Form screen to add a new skill to the profile.
17		Delete Skill	Popup dialog to confirm the deletion of a skill.
18		Bookmarked Jobs	Displays a list of jobs that the freelancer has bookmarked or saved.
19		My Contract List	Displays all active and past contracts associated with the freelancer.
20		Contract Detail	Displays detailed information and terms of a specific contract.
21		Sign Contract	Popup/Action screen allowing the freelancer to digitally sign a contract.
22		Contract File	Displays documents and files attached to a specific contract.
23		Upload Contract File	Form screen to upload new files or documents to a contract.
24		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
25		Portfolio List	Displays a list of projects and works in the freelancer's portfolio.
26		Add Portfolio	Form screen to create and add a new project to the portfolio.
27		Portfolio Detail	Displays full details, descriptions, and media of a specific portfolio project.
28		Edit Portfolio	Form screen to update or modify an existing portfolio project.
29		Delete Portfolio	Popup dialog to confirm the deletion of a portfolio project.

No	Actor	Screen Name	Description
30		Forum Categories	Displays a list of available discussion categories in the forum.
31		Forum Categories Detail	Displays sub-topics and general information of a selected forum category.
32		Post List	Displays a list of discussion threads or posts within a category.
33		Create Post	Form screen allowing the freelancer to create a new forum post.
34		Post Detail	Displays the full content of a forum post along with its replies.
35		Edit Post	Form/Popup to edit the content of an existing forum post.
36		Delete Post	Popup dialog to confirm the deletion of a forum post.
37	Client	Google Login	Allows users to login/register using Google OAuth.
38		Login	Allows users to log in using email/password and JWT authentication.
39		Logout	Allows users to logout and terminate authentication session.
40		Register	Allows users to create new accounts using email and password.
41		Forgot Password	Allows users to request password reset via email verification link.
42		Client Dashboard	Main dashboard for authenticated clients, showing active jobs, contract summaries, and notifications.
43		Create Job Form	Form screen allowing clients to create and publish a new job posting with details (title, description, budget, category, skills).

No	Actor	Screen Name	Description
44		Edit Job	Form screen allowing clients to modify details of an existing job posting.
45		Delete Job	Popup dialog to confirm the soft deletion of a job posting.
46		Change Job Status	Form/Dropdown action to change the status of a job posting (Open, In Progress, Closed, Cancelled).
47		My Contract List	Displays all active and past contracts associated with the client.
48		Contract Detail	Work interface for client to manage contract terms, milestones, and release escrow payments.
49		Create Contract	Form screen allowing the client to initiate and create a new contract with a freelancer.
50		Update Contract Terms	Form/Edit screen to update terms, milestones, and payment settings of a pending contract.
51		Sign Contract	Popup/Action screen allowing the client to digitally sign and fund the contract escrow (Web3).
52		Cancel Contract	Popup/Action screen allowing the client to request contract cancellation and handle escrow refunds.
53		Upload Contract File	Form screen allowing the client to upload files or deliverables to a contract.
54		Delete Contract File	Popup dialog to confirm the deletion of a contract file.
55		View Progress Updates	Screen displaying progress updates and attached proof of work submitted by the freelancer.

No	Actor	Screen Name	Description
56		Hired Freelancers	Screen displaying the list of freelancers currently hired by the client.
57	Admin	Dashboard	Main admin screen displaying system overview and key statistics.
58		Profile	Allows admin to view and update personal account information.
59		Change Password	Allows admin to change their account password.
60		Role Management	Displays a list of all roles along with their assigned permissions.
61		Add Role	Form screen for creating a new role with name, description, and permissions.
62		Edit Role	Form screen for modifying an existing role's name, description, and permissions.
63		View Role Detail	Displays detailed information of a specific role including its permissions.
64		Confirm Delete Role	Popup dialog prompting admin to confirm before permanently deleting a role.

3.1.3 Screen Authorization

No	Screen / Page	Guest	Client	Freelancer	Admin
1	Forum Categories	✓	✓	✓	✓
2	Forum Category Detail	✓	✓	✓	✓
3	Post List	✓	✓	✓	✓
4	Post Detail	✓	✓	✓	✓
5	View Comments	✓	✓	✓	✓
6	View Comment Detail	✓	✓	✓	✓

No	Screen / Page	Guest	Client	Freelancer	Admin
7	Create Post		✓	✓	
8	Edit Post		✓	✓	
9	Delete Post		✓	✓	✓
10	Like Post		✓	✓	
11	Unlike Post		✓	✓	
12	Add Comment		✓	✓	
13	Edit Comment		✓	✓	
14	Delete Comment		✓	✓	✓
15	Like Comment		✓	✓	
16	Unlike Comment		✓	✓	
17	Report Content		✓	✓	
18	View Detailed Freelancer Profile	✓	✓	✓	
19	Update Detailed Freelancer Profile			✓	
20	Get List Freelancer Skill	✓	✓	✓	
21	Add Freelancer Skill			✓	
22	Delete Freelancer Skill			✓	
23	Get Portfolio List			✓	
24	Add New Portfolio			✓	
25	Get Portfolio Detail			✓	
26	Update Portfolio			✓	
27	Delete Portfolio			✓	
28	Create Contract		✓		
29	Update Contract Terms		✓		
30	Complete Contract		✓	✓	
31	Get Contract List		✓	✓	✓
32	Get Contract Detail		✓	✓	✓

No	Screen / Page	Guest	Client	Freelancer	Admin
33	Sign Contract		✓	✓	
34	Cancel Contract		✓	✓	✓
35	Contract Activities		✓	✓	✓
36	Contract Files		✓	✓	✓
37	Upload Contract File		✓	✓	
38	Delete Contract File		✓	✓	✓
39	Progress Update			✓	
40	View Progress Updates		✓	✓	✓
41	Google Login	✓			
42	Login	✓			
43	Logout		✓	✓	✓
44	Register	✓			
45	Forgot Password	✓	✓	✓	✓
46	View Role List				✓
47	View Role Detail				✓
48	Create New Role				✓
49	Update Role				✓
50	Delete Role				✓
51	View List Job	✓	✓	✓	
52	View Job Detail	✓	✓	✓	
53	Post New Job		✓		
54	Edit Job		✓		
55	Delete Job		✓		
56	Change Job Status		✓		
57	View List Bookmarked Job			✓	
58	Bookmark Job			✓	
59	Remove Bookmark			✓	

3.1.4 Non-Screen Functions

#	Feature	System Function	Description
1	**Manage Authentication**	[POST] /api/auth/register [POST] /api/auth/otp [POST] /api/auth/login [POST] /api/auth/refresh-token [POST] /api/auth/logout [POST] /api/auth/forgot-password [GET] /api/auth/google-link [GET] /api/auth/google/callback [GET] /api/auth/me [POST] /api/auth/switch-role [POST] /api/auth/join-role	The Authentication feature handles user identity verification and session establishment. It supports account registration with OTP verification, email/password login with JWT access and refresh tokens, token renewal, logout, and password reset through an email verification link. Users can also sign in with Google OAuth, retrieve the currently authenticated account, and switch or join a role (Client or Freelancer) on the same account.
2	**Manage Account Profile**	[GET] /api/account-profile [PUT] /api/account-profile [PUT] /api/account-profile/password [POST] /api/account-profile/avatar [GET] /api/account-profile/avatar/{userId}/file [GET] /api/clients/{userId} [PUT] /api/clients/me/profile [GET] /api/social-links [POST] /api/social-links [DELETE] /api/social-links/{id} [GET] /api/favorites/freelancers [POST] /api/favorites/freelancers/{freelancerId} [DELETE] /api/favorites/freelancers/{freelancerId} [GET] /api/following/freelancers/discover [GET] /api/following/freelancers [POST] /api/following/freelancers/{freelancerId} [DELETE] /api/following/freelancers/{freelancerId}	The Account Profile feature lets authenticated users manage their account-level information and social network. Users can view and update their account profile, change their password, upload and retrieve their avatar, and manage social links. Clients can view another client's public profile and update their own client profile. Users can also save favorite freelancers, discover and follow freelancers, and unfollow or remove them from the favorites list.

#	Feature	System Function	Description
3	**Manage Freelancer Profile**	[GET] /api/profiles/{id} [PUT] /api/profiles/{id} [POST] /api/profiles/{id}/cv [GET] /api/profiles/{id}/cv/file [POST] /api/profiles/{id}/cv/analyze [DELETE] /api/profiles/{id}/cv [GET] /api/expert-profile/me [PUT] /api/expert-profile/me [PATCH] /api/users/{id}/expert-profile [GET] /api/experts [GET] /api/experts/{id}	The Freelancer Profile feature enables freelancers to build and maintain their professional identity, including bio, professional title, education, availability status and profile strength. Freelancers can upload, download, delete and AI-analyze their CV. The feature also exposes the expert (verified freelancer) profile endpoints, the public expert directory, and allows a freelancer to view and update their own expert profile while administrators can update it on behalf of a user.
4	**Manage Freelancer Skills**	[GET] /api/profiles/{id}/skills [GET] /api/profiles/skills/suggestions [POST] /api/profiles/{id}/skills [DELETE] /api/profiles/skills/{id}	The Freelancer Skills feature allows freelancers to define and showcase their professional capabilities. Users can view the skills attached to a profile, search the skill catalog for suggestions while typing, add a skill together with its proficiency level, and remove a skill that is no longer relevant. This keeps each freelancer's skill set accurate for the client's talent evaluation process.

#	Feature	System Function	Description
5	**Manage Freelancer Portfolio**	[GET] /api/profiles/{id}/portfolios [POST] /api/profiles/{id}/portfolios [GET] /api/profiles/portfolios/{id} [PUT] /api/profiles/portfolios/{id} [DELETE] /api/profiles/portfolios/{id}	The Freelancer Portfolio feature enables freelancers to demonstrate their past work visually. Freelancers can list every portfolio item of a profile, create a new item with project details and media, view the detail of a specific item, update its content or media, and soft-delete an item so that it is no longer displayed publicly. A well-organized portfolio helps freelancers build credibility with prospective clients.
6	**Manage Job**	[GET] /api/jobs [GET] /api/jobs/{slug} [GET] /api/job-categories [GET] /api/job-categories/{slug} [GET] /api/manage-jobs/skills [GET] /api/manage-jobs [GET] /api/manage-jobs/{id} [POST] /api/manage-jobs [PATCH] /api/manage-jobs/{id} [DELETE] /api/manage-jobs/{id} [PATCH] /api/manage-jobs/{id}/status [GET] /api/manage-jobs/bookmarks [POST] /api/manage-jobs/jobs/{slug}/bookmark [GET] /api/manage-jobs/jobs/{slug}/bookmark [DELETE] /api/manage-jobs/bookmarks/{slug}	The Job Management feature is the core marketplace functionality that connects clients with freelancers. Guests and freelancers can browse and search the paginated job list, view a job detail by slug, and browse job categories. Clients can create a job posting with description, budget, required skills and timeline, list their own jobs, view a job detail, update or delete a job, and change its status (Open, In Progress, Closed, Cancelled); they can also look up skills while composing a job. Freelancers can list, add and remove bookmarked jobs and check whether a job is bookmarked.

#	Feature	System Function	Description
7	**Manage Job Categories (Admin)**	[GET] /api/admin/job-categories [GET] /api/admin/job-categories/{id} [POST] /api/admin/job-categories [PATCH] /api/admin/job-categories/{id} [PATCH] /api/admin/job-categories/{id}/ restore [DELETE] /api/admin/job-categories/{id}	The Job Category Management feature allows administrators to maintain the taxonomy used to classify job postings. Administrators can list all categories, view the detail of a category, create a new category, update its name or description, soft-delete a category, and restore a previously deleted category when it is needed again.
8	**Manage Saved Searches**	[GET] /api/saved-searches [GET] /api/saved-searches/{id} [POST] /api/saved-searches [PATCH] /api/saved-searches/{id} [DELETE] /api/saved-searches/{id}	The Saved Search feature allows authenticated users to store job search criteria for later reuse. Users can list their saved searches, view the detail of one saved search, create a new saved search from the current filters, update its name or filter conditions, and delete a saved search that is no longer needed.

#	Feature	System Function	Description
9	**Manage Proposals**	[GET] /api/proposals/my [GET] /api/proposals/jobs/{jobId}/mine [GET] /api/proposals/jobs/{jobId}/submitted [GET] /api/jobs/{jobId}/proposals [GET] /api/proposals/{id} [GET] /api/proposals/{id}/client [POST] /api/proposals/jobs/{jobId} [POST] /api/proposals/jobs/{jobId}/drafts [PATCH] /api/proposals/{id}/draft [PATCH] /api/proposals/{id}/submit [PATCH] /api/proposals/{id}/withdraw [PATCH] /api/proposals/{id}/interview [PATCH] /api/proposals/{id}/reject [PATCH] /api/proposals/{id}/accept	The Proposal Management feature supports the bidding process between freelancers and clients. Freelancers can submit a proposal to a job, save it as a draft, update and submit a draft, withdraw a submitted proposal, and view their own proposals for a job. Clients can list all proposals submitted to their job, view proposal details, and move a proposal through interview, acceptance or rejection, while acceptance can later be used to create a contract.
10	**Manage Contract**	[GET] /api/contracts [GET] /api/contracts/{id} [POST] /api/contracts [PATCH] /api/contracts/{id} [PATCH] /api/contracts/{id}/sign [PATCH] /api/contracts/{id}/complete [PATCH] /api/contracts/{id}/cancel [GET] /api/contracts/{contractId}/files [POST] /api/contracts/{contractId}/files [DELETE] /api/contracts/{contractId}/files/{fileId}	The Contract Management feature manages the agreement between a client and a freelancer throughout the project lifecycle. Users can create a contract, list their contracts, view contract details, update contract terms, sign the contract to confirm participation, complete the contract and request cancellation. Contract participants can also list, upload and delete contract-level files that document the agreement.

#	Feature	System Function	Description
11	**Manage Milestone**	[GET] /api/contracts/{contractId}/milestones [GET] /api/contracts/{contractId}/milestones/{id} [POST] /api/contracts/{contractId}/milestones [PATCH] /api/contracts/{contractId}/milestones/{id} [PATCH] /api/contracts/{contractId}/milestones/{id}/progress [DELETE] /api/contracts/{contractId}/milestones/{id}	The Milestone Management feature breaks a contract into payable deliverables. Contract participants can list and view the milestones of a contract, create a new milestone with its amount and due date, update milestone information, report progress on a milestone, and delete a milestone that is no longer part of the agreement.
12	**Manage Milestone Files**	[GET] /api/contracts/{contractId}/milestones/{milestoneId}/files [POST] /api/contracts/{contractId}/milestones/{milestoneId}/files [DELETE] /api/contracts/{contractId}/milestones/{milestoneId}/files/{fileId}	The Milestone File feature allows contract participants to attach supporting documents and media to a specific milestone. Users can list every file of a milestone, upload a new file, and delete a file that was uploaded by mistake or is no longer relevant.
13	**Manage Milestone Submissions**	[POST] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions [GET] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id} [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/request-changes [PATCH] /api/contracts/{contractId}/milestones/{milestoneId}/submissions/{id}/approve	The Milestone Submission feature supports the deliverable review workflow. A freelancer can submit work for a milestone, and both parties can list the submissions of a milestone and view the detail of one submission. The client can request changes so that the freelancer revises and resubmits the work, or approve the submission to complete the milestone and release the corresponding payment.

#	Feature	System Function	Description
14	**Manage Payments**	[POST] /api/payments/connect/onboarding-link [GET] /api/payments/connect/status [GET] /api/payments/payment-methods [POST] /api/payments/customer-portal [POST] /api/payments/contracts/{contractId}/platform-fee/intent [POST] /api/payments/contracts/{contractId}/platform-fee/checkout-session [POST] /api/payments/contracts/{contractId}/platform-fee/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/intent [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/checkout-session [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/fund/saved-card [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/release [POST] /api/payments/contracts/{contractId}/milestones/{milestoneId}/refund [POST] /api/payments/disputes/{disputeId}/fee/intent [POST] /api/payments/disputes/{disputeId}/fee/checkout-session [POST] /api/payments/disputes/{disputeId}/settle [GET] /api/payments/transactions [POST] /api/payments/intents/{paymentIntentId}/sync [POST] /api/payments/checkout-sessions/{sessionId}/sync [POST] /api/payments/webhook	The Payment Management feature integrates Stripe to provide secure escrow-style transactions. Users can connect a Stripe account and check its onboarding status, list saved payment methods, open the Stripe customer portal, and pay the platform fee or fund a milestone either through a PaymentIntent, a checkout session, or a saved card. Clients can release or refund milestone funds, pay a dispute fee and settle a dispute, and view their transaction history. The system also synchronizes PaymentIntent and CheckoutSession status and receives asynchronous Stripe webhook events.

#	Feature	System Function	Description
15	**Manage Dispute**	[GET] /api/disputes [POST] /api/disputes [GET] /api/disputes/{id} [POST] /api/disputes/{id}/fee [POST] /api/disputes/{id}/response [GET] /api/disputes/milestone/{milestoneId} [POST] /api/disputes/{id}/review [GET] /api/admin/disputes [GET] /api/admin/disputes/{id} [POST] /api/admin/disputes/{id}/decision [POST] /api/admin/disputes/{id}/ final-decision	The Dispute Management feature handles conflicts that arise during a milestone. Participants can open a dispute for a milestone, list and view their disputes, pay the dispute fee, submit a response to the other party's claim, and review the proposed resolution. The dispute of a specific milestone can be looked up. Administrators can list all disputes, view dispute details, issue a decision, and record the final decision that settles the conflict.
16	**Manage Review**	[GET] /api/reviews/contracts/{contractId} [POST] /api/reviews/contracts/{contractId} [GET] /api/reviews/{reviewId} [PATCH] /api/reviews/{reviewId} [DELETE] /api/reviews/{reviewId} [POST] /api/reviews/{reviewId}/response [PATCH] /api/reviews/responses/{responseId} [DELETE] /api/reviews/responses/ {responseId}	The Review Management feature lets contract participants rate and comment on each other after a contract is completed. Users can list the reviews of a contract, create a review, view, update and delete their own review, and respond to a review they received. Responses can also be updated or deleted by their author, helping both parties maintain a trustworthy reputation on the platform.

#	Feature	System Function	Description
17	**Manage Forum**	[GET] /api/forums/categories [GET] /api/forums/categories/top [GET] /api/forums/categories/{id} [GET] /api/forums/categories/by-slug/{slug} [GET] /api/forums/posts [GET] /api/forums/posts/top [GET] /api/forums/posts/{id} [GET] /api/forums/posts/by-slug/{slug} [GET] /api/forums/users/top [POST] /api/forums/posts [DELETE] /api/forums/posts/{id} [GET] /api/forums/posts/{postId}/comments [POST] /api/forums/posts/{postId}/comments [PATCH] /api/forums/posts/{postId}/comments/{id} [DELETE] /api/forums/posts/{postId}/comments/{id} [POST] /api/forums/posts/{postId}/comments/{commentId}/like [POST] /api/forums/posts/{id}/like [GET] /api/forums/posts/{id}/like [POST] /api/forums/posts/{postId}/reports [POST] /api/forums/posts/{postId}/comments/{commentId}/reports [GET] /api/forums/posts/{postId}/is-reported [GET] /api/forums/posts/{postId}/comments/{commentId}/is-reported	The Forum Management feature enables community discussion and knowledge sharing within the platform. Guests and authenticated users can browse forum categories (including the most active categories), browse and search posts (including the most interacted posts and the most active users), and open a post by id or slug. Registered users can create a post, delete their own post, and manage comments (list, add, edit and delete). Posts and comments support a like/unlike toggle and a liker list, and users can report inappropriate posts or comments and check whether they have already reported them.

#	Feature	System Function	Description
18	**Manage Forum Moderation**	[GET] /api/forums/admin/stats [GET] /api/forums/admin/categories [GET] /api/forums/admin/categories/{id} [POST] /api/forums/admin/categories [PATCH] /api/forums/admin/categories/{id} [DELETE] /api/forums/admin/categories/{id} [PATCH] /api/forums/admin/categories/{id}/restore [GET] /api/forums/admin/comments [GET] /api/forums/admin/pending-posts [GET] /api/forums/admin/trash/posts [GET] /api/forums/admin/trash/comments [PATCH] /api/forums/admin/trash/posts/{id}/restore [PATCH] /api/forums/admin/trash/comments/{id}/restore [PATCH] /api/forums/admin/posts/{id}/approve [PATCH] /api/forums/admin/posts/{id}/reject [GET] /api/forums/reports [PATCH] /api/forums/reports/{reportId}/status	The Forum Moderation feature gives administrators the tools to keep the community healthy. Administrators can view forum statistics, list, view, create, update, soft-delete and restore forum categories, and list all comments. New posts can be reviewed through the pending-post queue and either approved or rejected, while the trash lists allow deleted posts and comments to be reviewed and restored. The report queue lists content reported by users, and each report's status can be updated once it has been handled.
19	**Manage Messaging**	[GET] /api/conversations [POST] /api/conversations [GET] /api/conversations/{id}/messages [POST] /api/conversations/{id}/messages [POST] /api/conversations/{id}/files [DELETE] /api/conversations/{id}/messages/{messageId} [PATCH] /api/conversations/{id}/read [PATCH] /api/conversations/{id}/hide [PATCH] /api/conversations/{id}/pin	The Messaging feature provides real-time communication between clients and freelancers. Users can list their conversations, start a new conversation, list and send messages, and send files within a conversation. A message can be deleted or recalled, a conversation can be marked as read to reset its unread counter, hidden from the conversation list, and pinned or unpinned. Messages are delivered in real time through a WebSocket gateway.

#	Feature	System Function	Description
20	**Manage Notification**	[GET] /api/notifications [PATCH] /api/notifications/read-all [PATCH] /api/notifications/{id}/read [DELETE] /api/notifications/{id}	The Notification feature delivers in-app notifications about relevant events such as proposals, contracts, milestones, payments and forum moderation. Users can list their notifications with their read state, mark a single notification or all notifications as read, and delete a notification they no longer want to keep.
21	**Manage Roles**	[GET] /api/roles [GET] /api/roles/{id} [POST] /api/roles [PATCH] /api/roles/{id} [PUT] /api/roles/{id}/permissions [DELETE] /api/roles/{id}	The Role Management feature allows administrators to define and control user permissions. Administrators can list all roles, view the detail of a role together with its permissions, create a new role, update its name or description, replace the permission set assigned to a role, and delete a role that is no longer needed. This provides fine-grained access control and proper segregation of duties across user types.
22	**Manage Permissions**	[GET] /api/permissions [GET] /api/permissions/{id}	The Permission Management feature exposes the catalog of permissions used by the role-based access control system. Administrators can list every available permission and view the details of a single permission so that permissions can be assigned to roles accurately.

#	Feature	System Function	Description
23	**Manage Sessions**	[GET] /api/sessions [GET] /api/sessions/{id} [DELETE] /api/sessions/{id}	The Session Management feature gives authenticated users visibility and control over their active login sessions. Users can list all sessions of their account, view the details of a specific session (such as device and login time), and revoke a session so that the corresponding refresh token can no longer be used.
24	**Manage Users (Admin)**	[POST] /api/users [GET] /api/users [GET] /api/users/{id} [PATCH] /api/users/{id} [PATCH] /api/users/{id}/client-profile [PATCH] /api/users/{id}/freelancer-profile [PUT] /api/users/{id}/freelancer-profile/skills [POST] /api/users/{id}/freelancer-profile/portfolio-items [PATCH] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [DELETE] /api/users/{id}/freelancer-profile/portfolio-items/{itemId} [GET] /api/users/skills-catalog	The User Management feature allows administrators to manage accounts from the administration side. Administrators can list users, view user details, create a new user, and update general user information. They can also edit a client profile, edit a freelancer's basic profile and replace the freelancer's skills, manage the freelancer's portfolio items, and read the skill catalog when performing these updates.

#	Feature	System Function	Description
25	**Manage Identity Verification**	[POST] /api/identity-verifications/documents [POST] /api/identity-verifications/ai-check [POST] /api/identity-verifications/liveness-pose [GET] /api/identity-verifications/status [GET] /api/identity-verifications/documents/{id}/file [GET] /api/admin/identity-verifications [GET] /api/admin/identity-verifications/{id} [GET] /api/admin/identity-verifications/{id}/file [PATCH] /api/admin/identity-verifications/{id}/approve [PATCH] /api/admin/identity-verifications/{id}/reject	The Identity Verification feature supports the eKYC process for user accounts. Users can upload identity documents, run an AI check on the submitted documents, capture liveness-pose images, check their verification status, and download an uploaded document file. Administrators can list verification requests, view request details and the attached file, then approve or reject the verification with a reason.
26	**Manage Profile Revisions**	[GET] /api/profile-revisions/me [GET] /api/admin/profile-revisions [GET] /api/admin/profile-revisions/{id} [PATCH] /api/admin/profile-revisions/{id}/approve [PATCH] /api/admin/profile-revisions/{id}/reject	The Profile Revision feature tracks changes made to freelancer profiles so that they can be moderated. A freelancer can view the latest revision of their own profile, while administrators can list all pending revisions, view the detail of a revision, and approve or reject it before the changes become publicly visible.

#	Feature	System Function	Description
27	**Manage Banners**	[GET] /api/banners [GET] /api/banners/image/{filename} [GET] /api/admin/banners [GET] /api/admin/banners/{id} [POST] /api/admin/banners [POST] /api/admin/banners/upload-image [PATCH] /api/admin/banners/{id} [DELETE] /api/admin/banners/{id}	The Advertisement Banner feature manages promotional banners displayed on the platform. The public endpoints return the active banners for a given position and serve the banner image files. Administrators can list and view all banners, create a new banner, upload its image, update its content, target position or schedule, and delete a banner.
28	**Manage Skills (Admin)**	[GET] /api/admin/skills [GET] /api/admin/skills/{id} [POST] /api/admin/skills [PATCH] /api/admin/skills/{id} [PATCH] /api/admin/skills/{id}/restore [DELETE] /api/admin/skills/{id}	The Skill Management feature allows administrators to maintain the master skill catalog used by freelancer profiles and job postings. Administrators can list skills, view the detail of a skill, create a new skill, update its name or category, soft-delete a skill, and restore a previously deleted skill.

3.1.5 Entity Relationship Diagram

3.1.5.1 Overall Entity Relationship Diagram

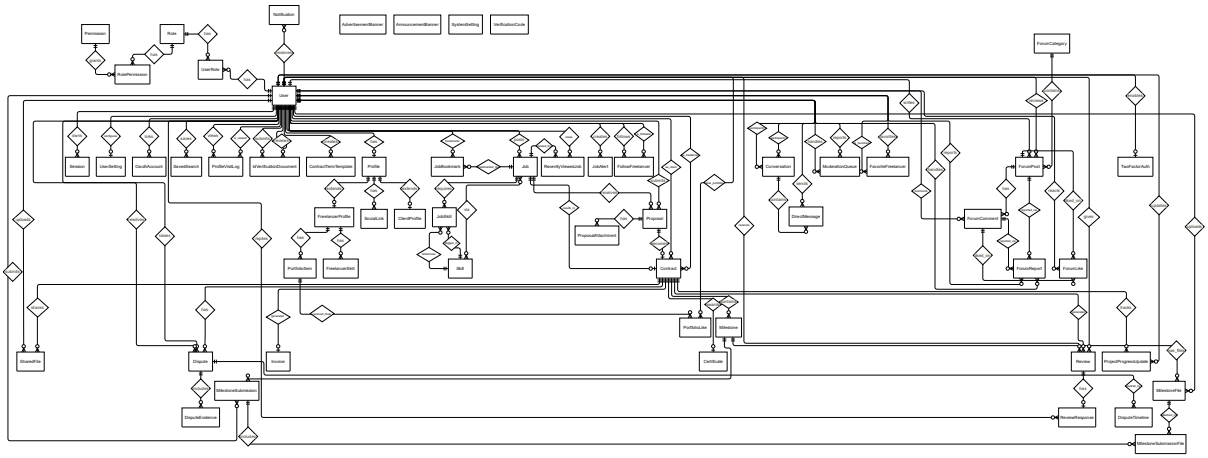


Figure 27: Overall Entity Relationship Diagram

3.1.5.2 Authentication & Sessions

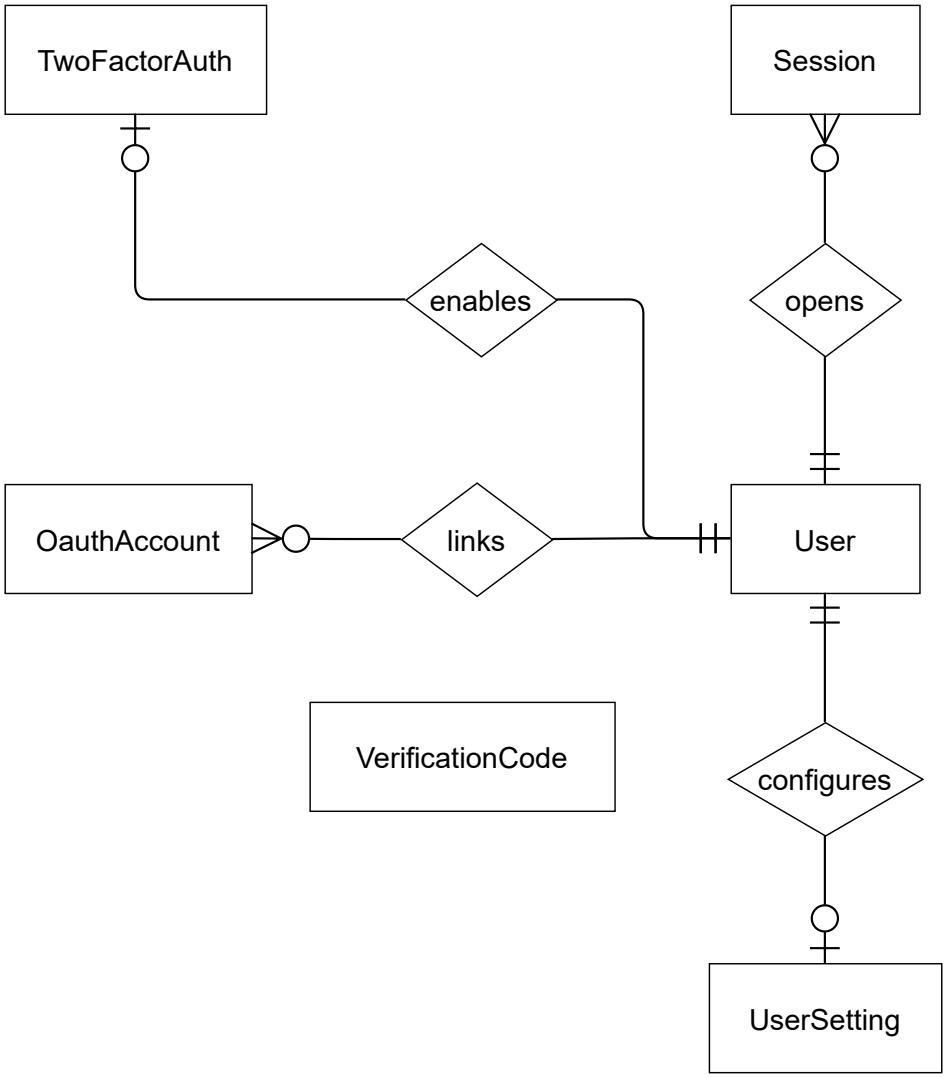


Figure 28: Authentication & Sessions ERD

3.1.5.3 Role-Based Access Control

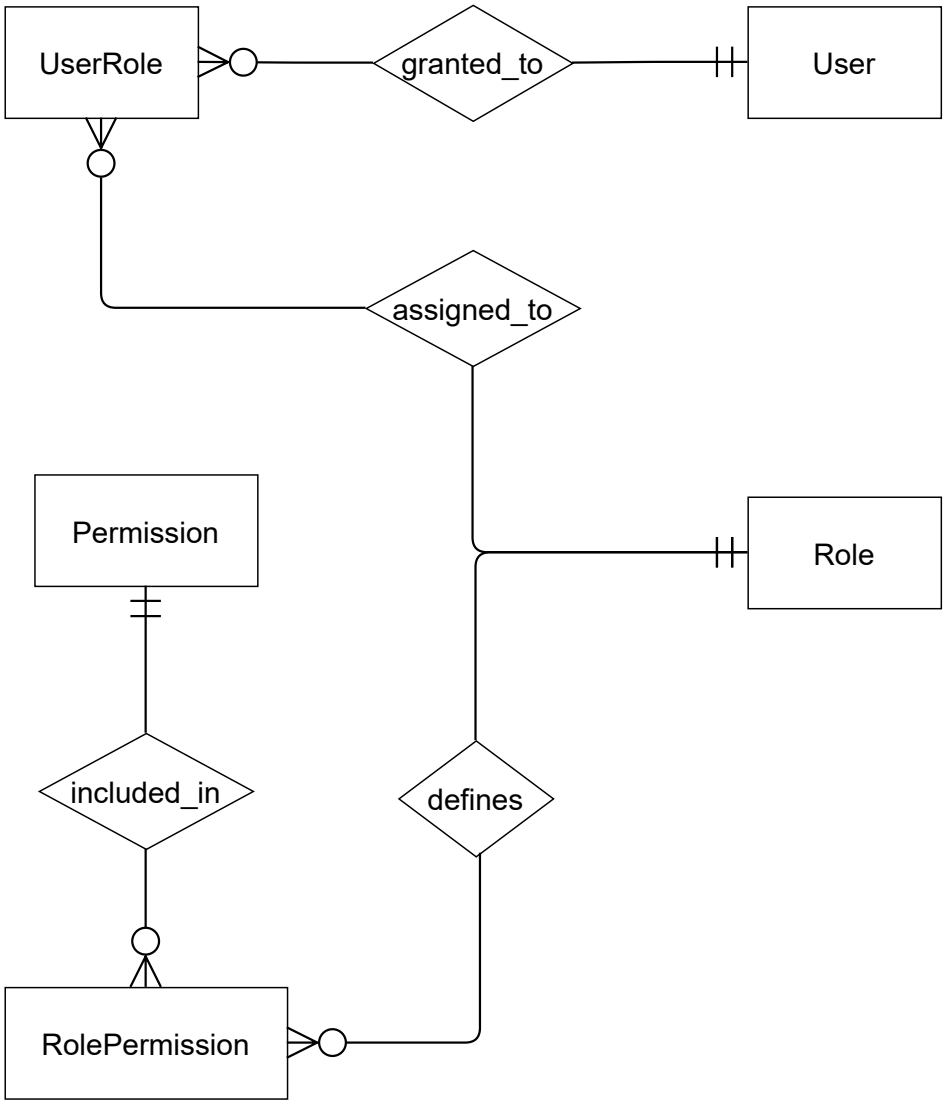


Figure 29: Role-Based Access Control ERD

3.1.5.4 Profile Core

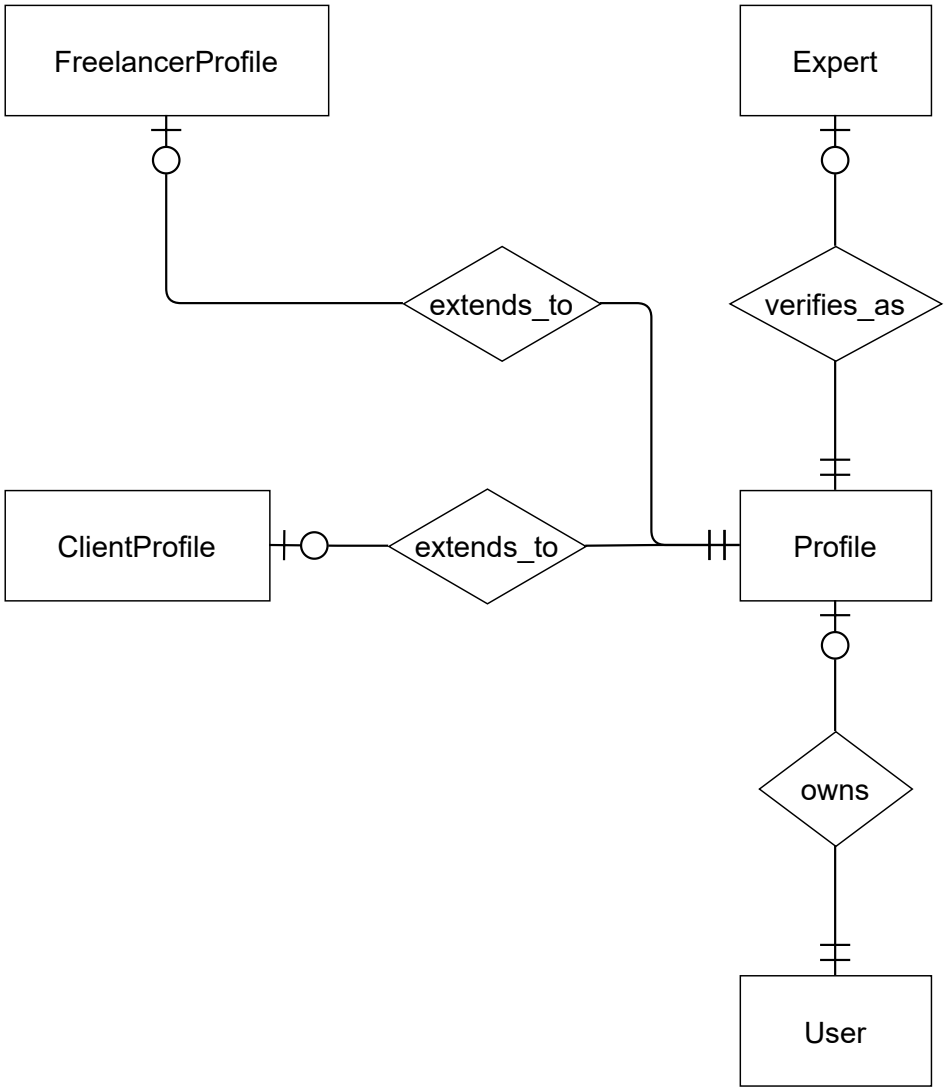


Figure 30: Profile Core ERD

3.1.5.5 Profile History & Social Links

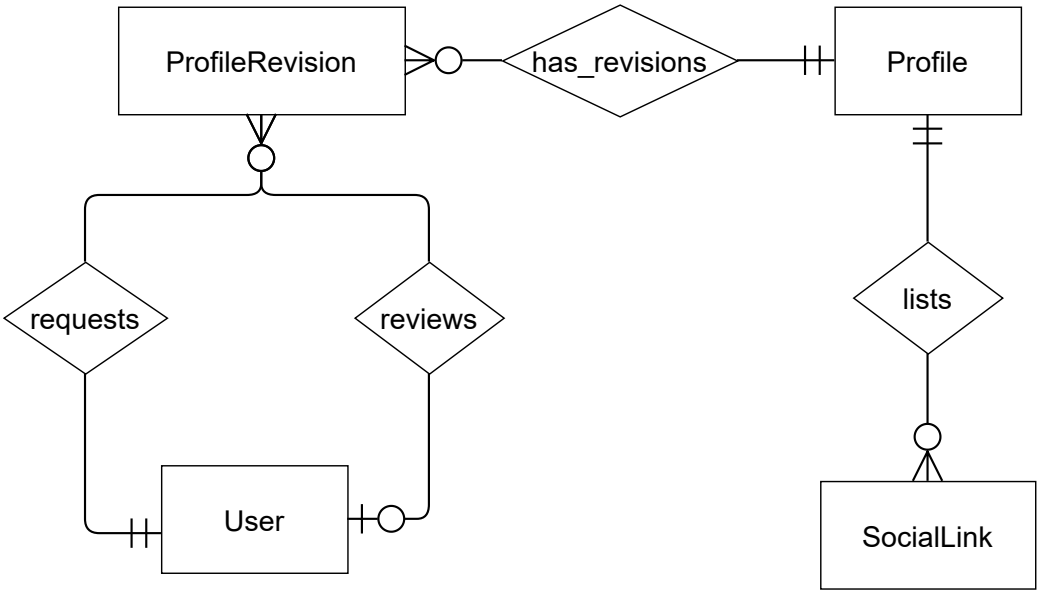


Figure 31: Profile History & Social Links ERD

3.1.5.6 Identity Verification & Profile Visits

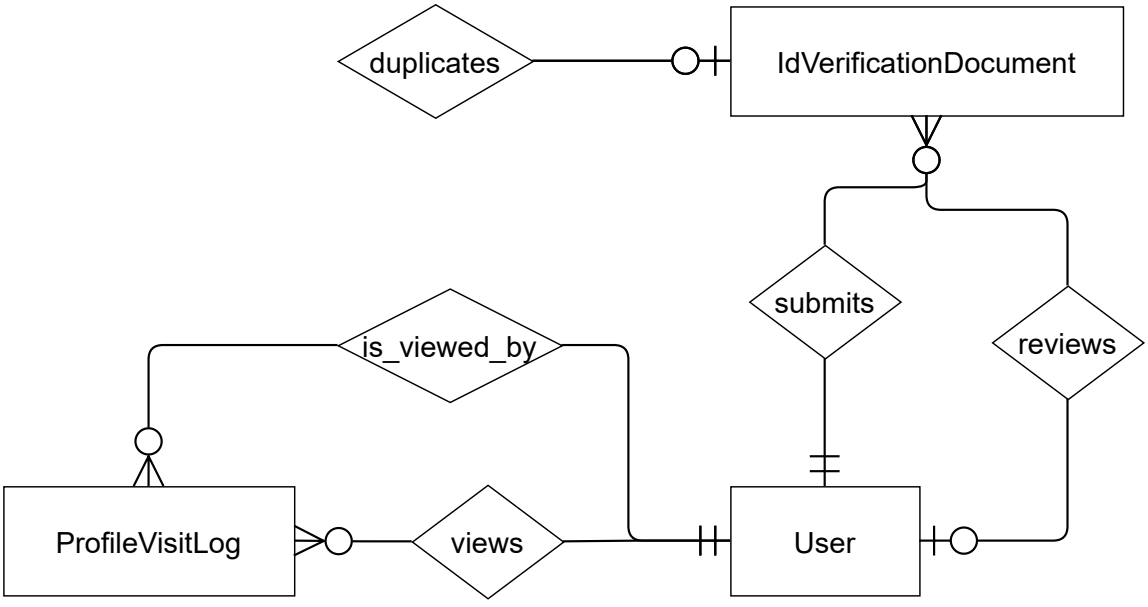


Figure 32: Identity Verification & Profile Visits ERD

3.1.5.7 Portfolio & Skills

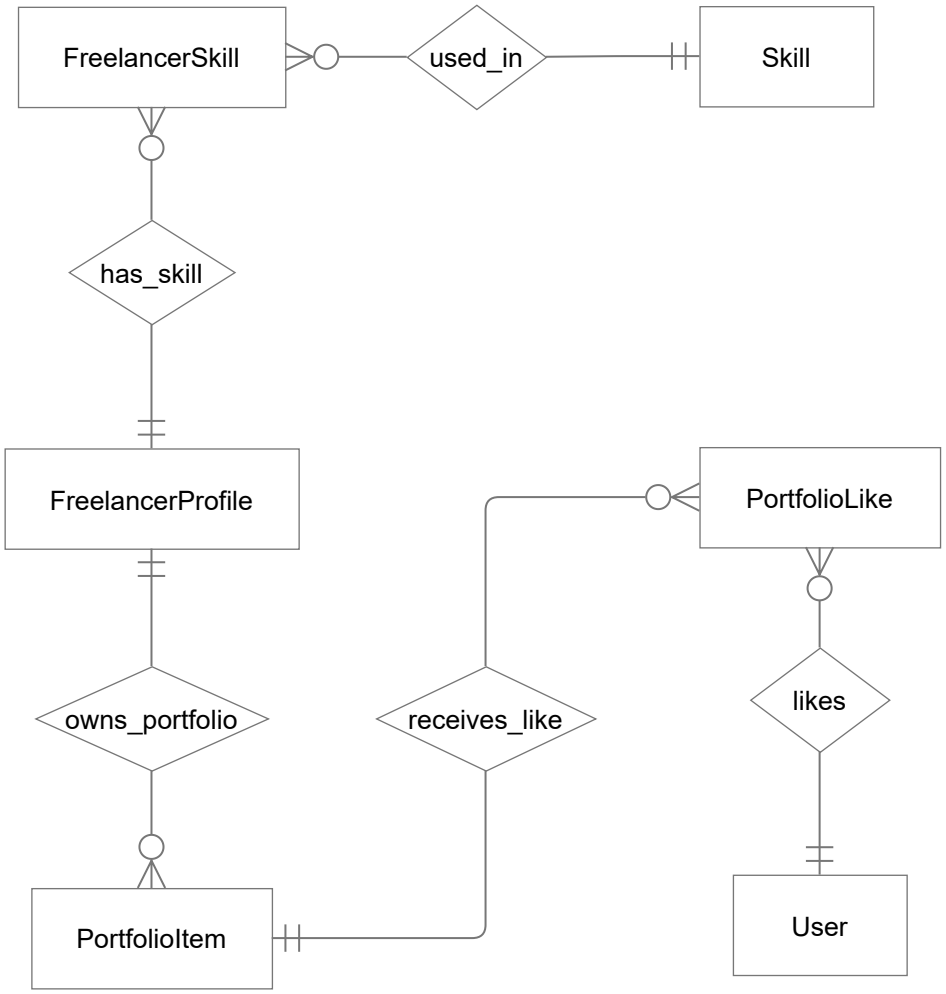


Figure 33: Portfolio & Skills ERD

3.1.5.8 Favorites & Follows

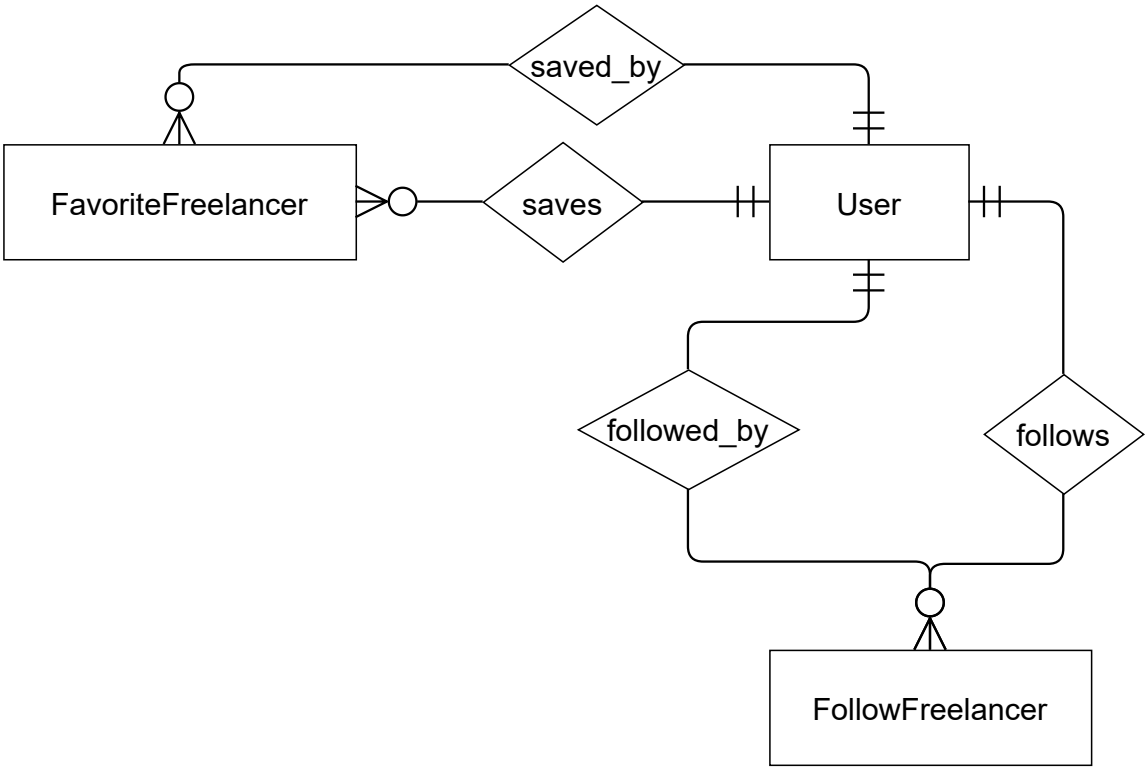


Figure 34: Favorites & Follows ERD

3.1.5.9 Job Posting

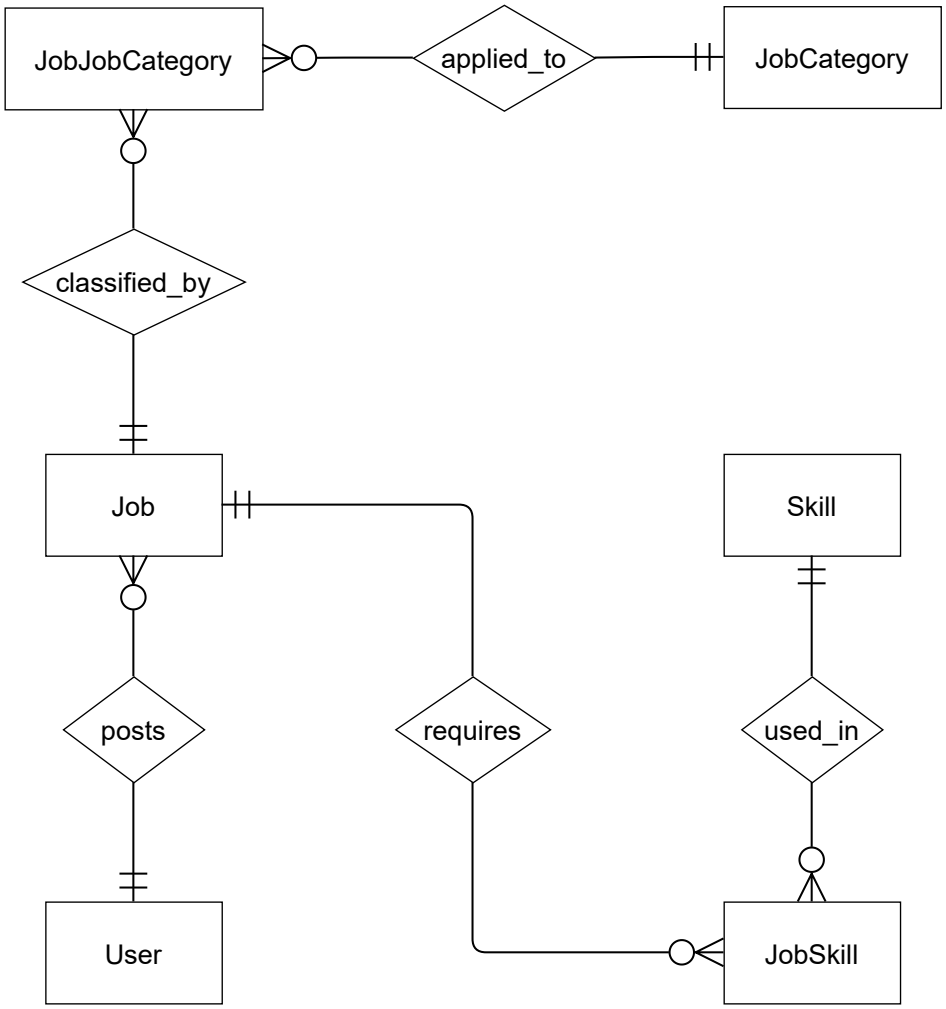


Figure 35: Job Posting ERD

3.1.5.10 Job Discovery & Alerts

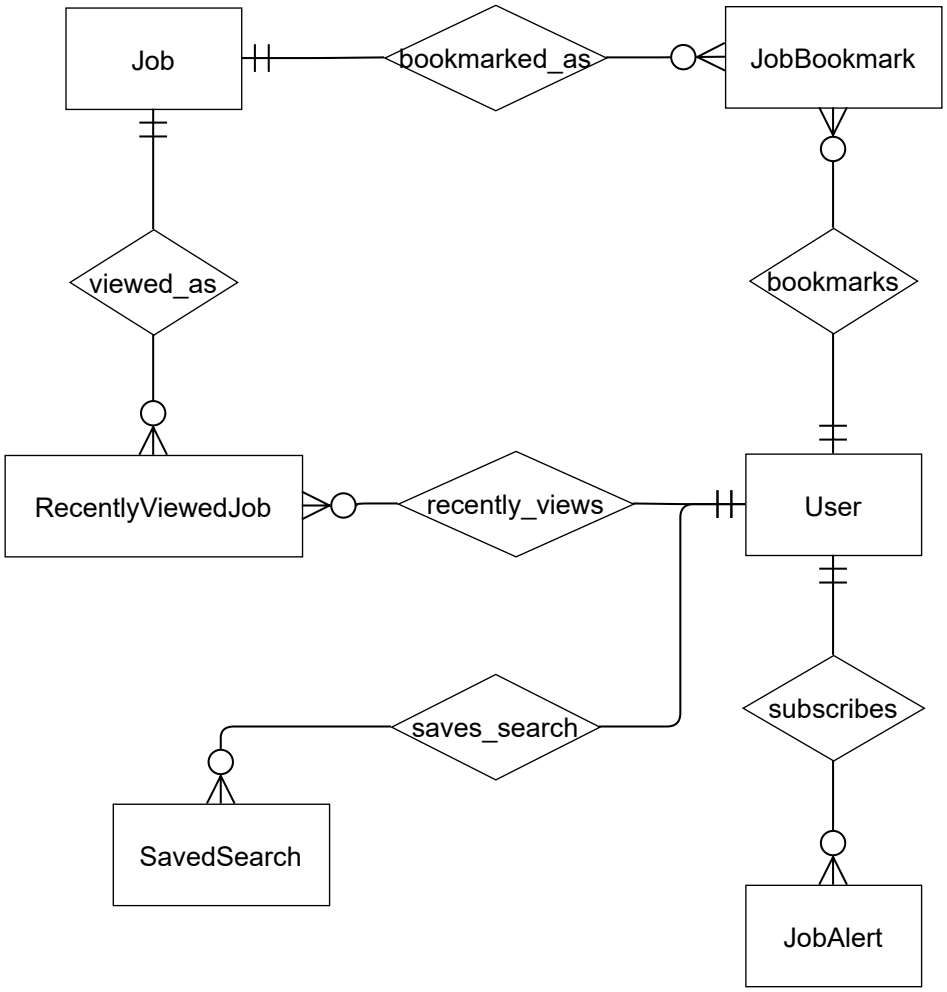


Figure 36: Job Discovery & Alerts ERD

3.1.5.11 Proposal

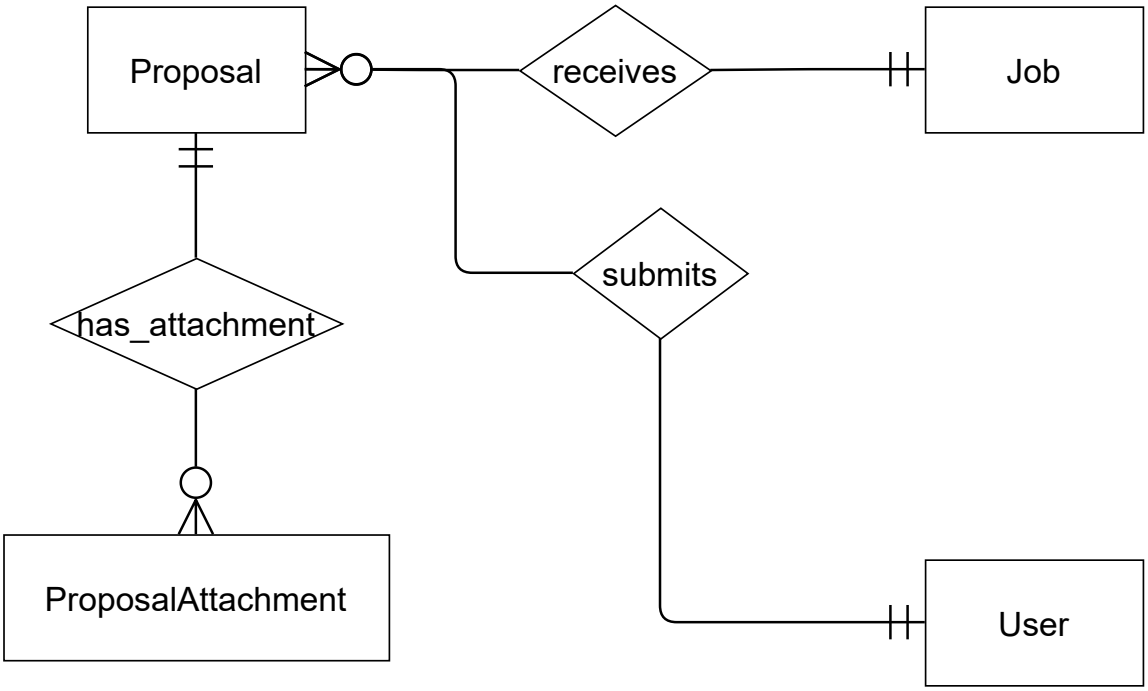


Figure 37: Proposal ERD

3.1.5.12 Contract

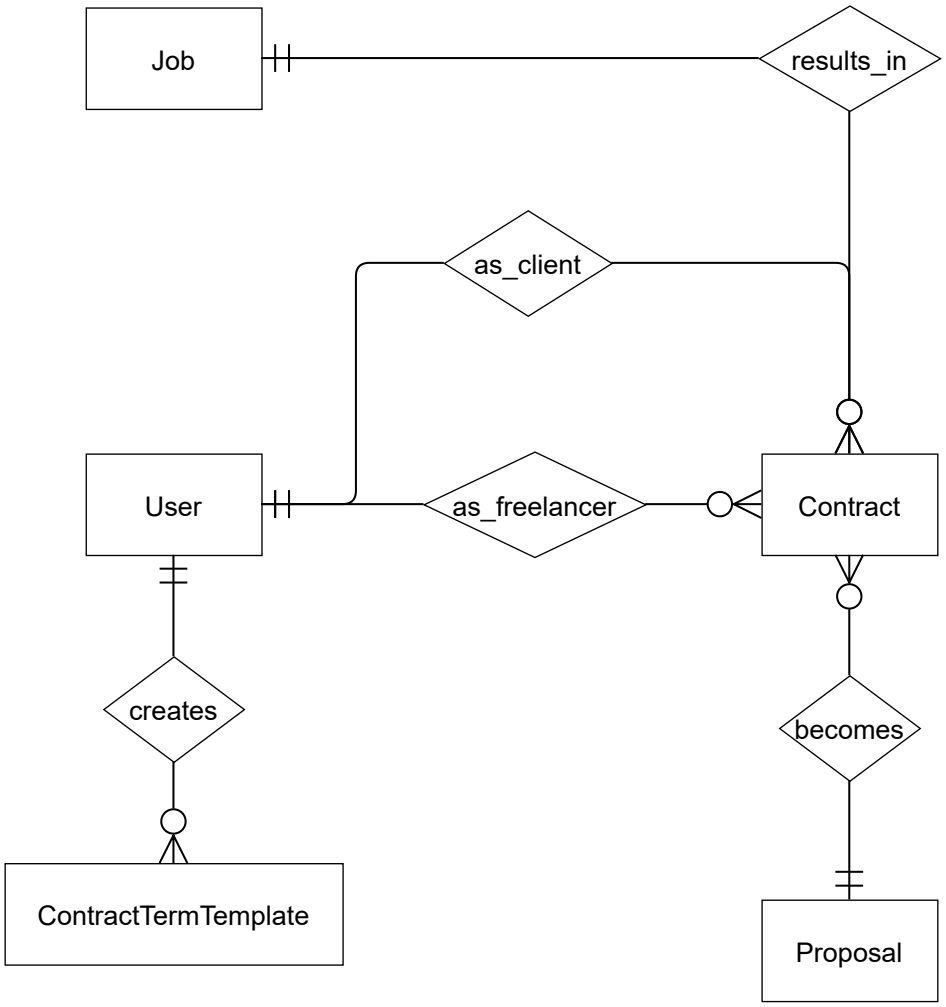


Figure 38: Contract ERD

3.1.5.13 Milestone & Project Progress

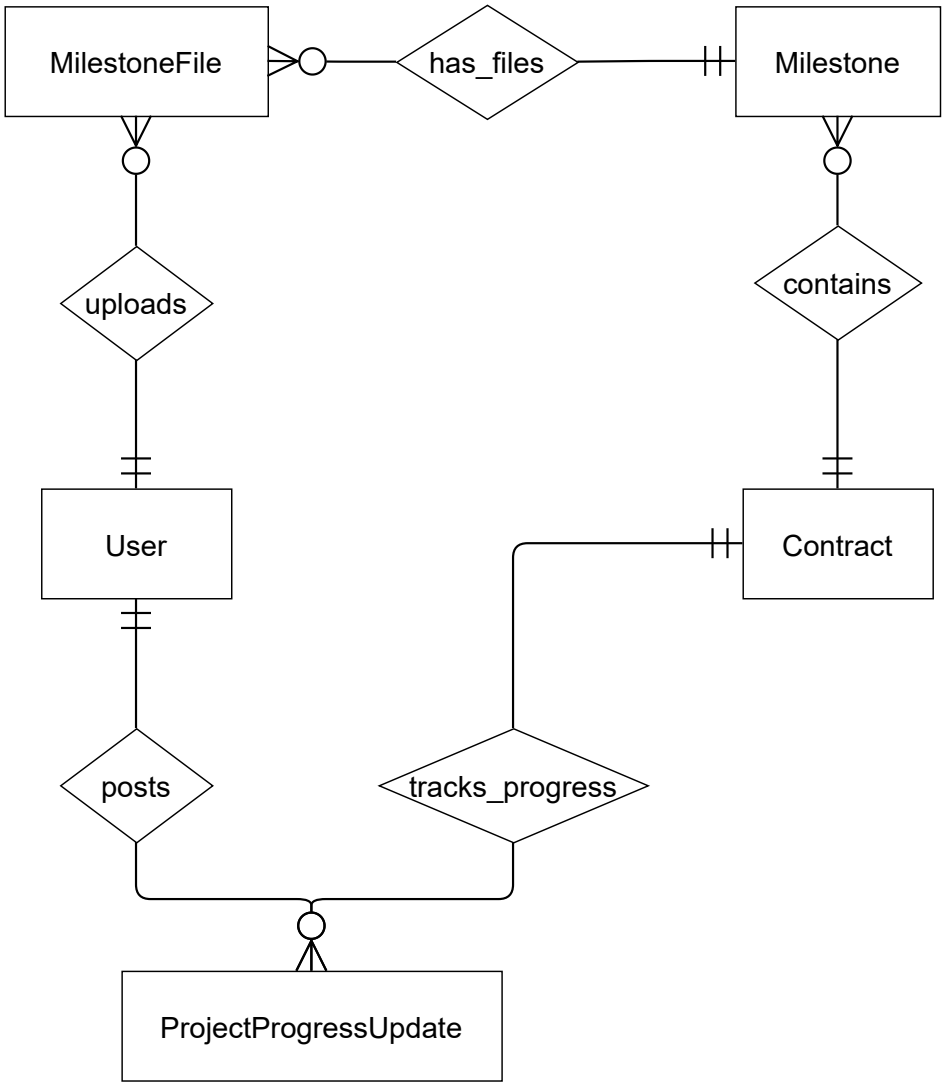


Figure 39: Milestone & Project Progress ERD

3.1.5.14 Milestone Submission

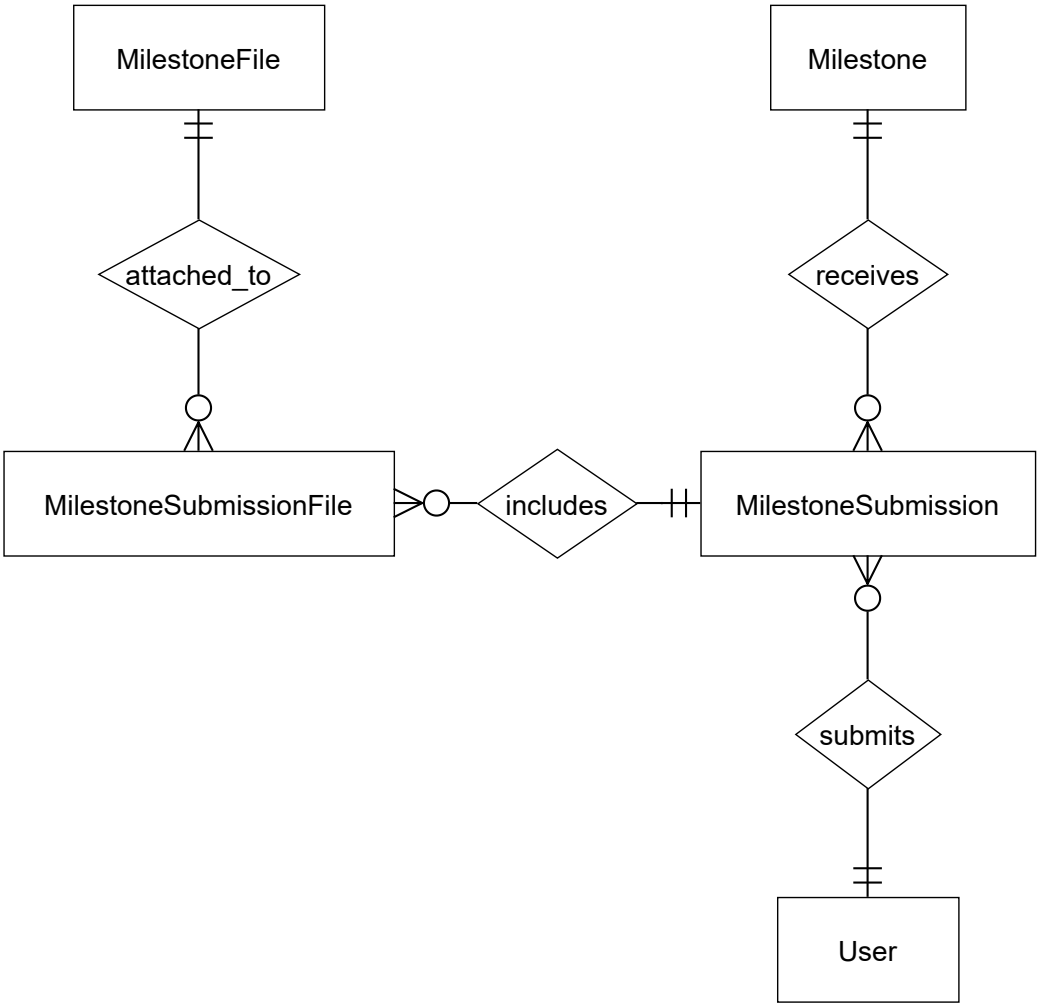


Figure 40: Milestone Submission ERD

3.1.5.15 Escrow, Payments & Certificates

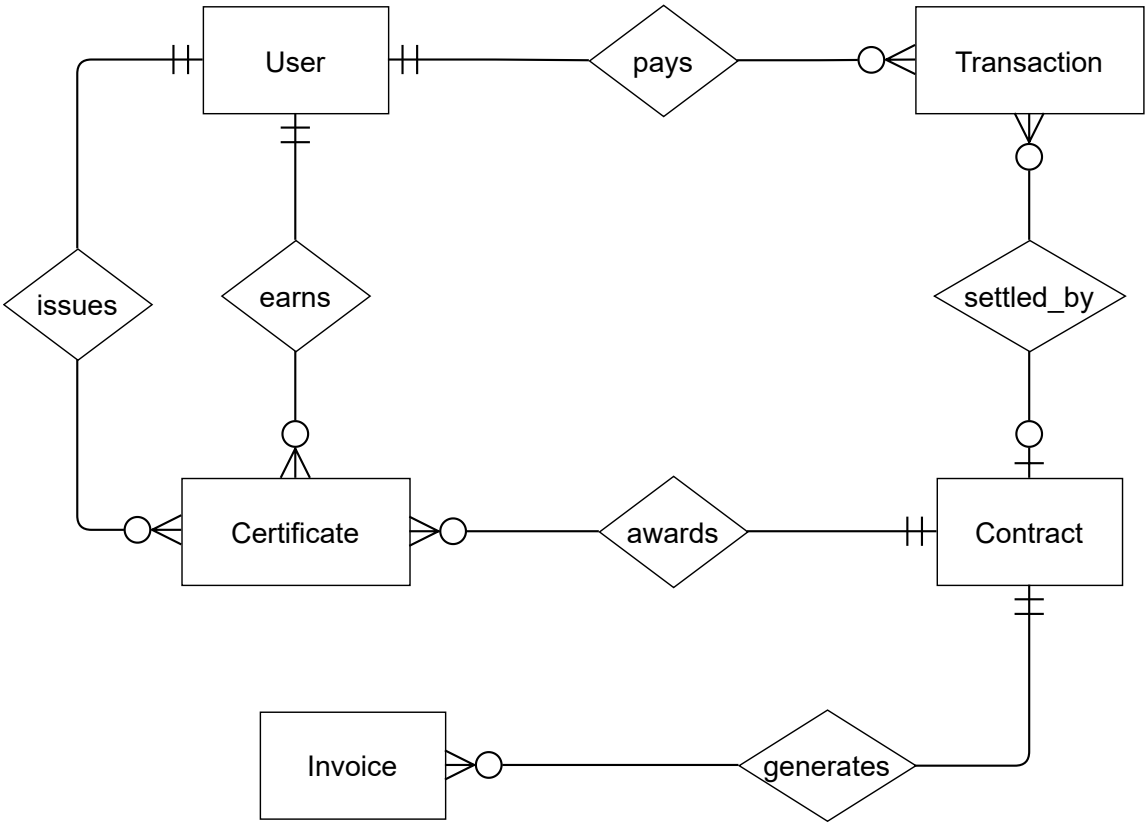


Figure 41: Escrow, Payments & Certificates ERD

3.1.5.16 Dispute Core

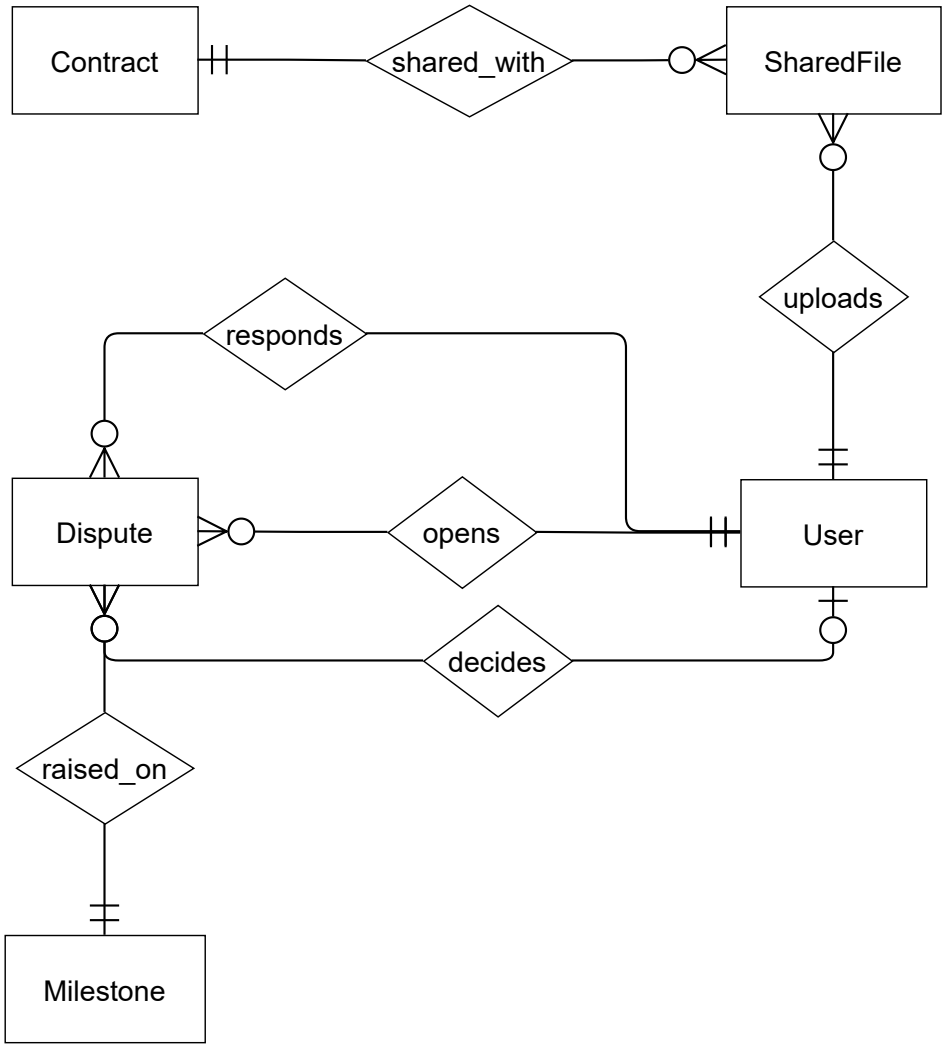


Figure 42: Dispute Core ERD

3.1.5.17 Dispute Resolution

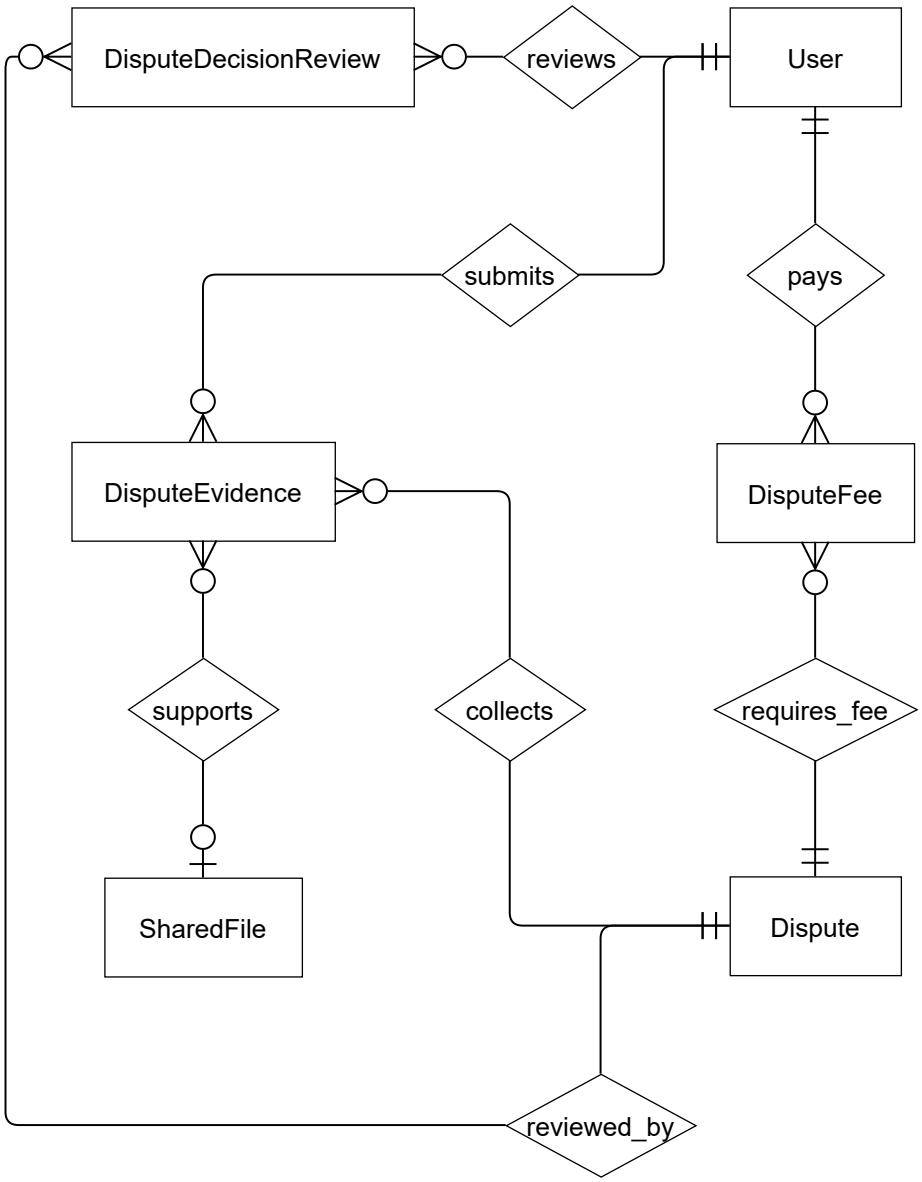


Figure 43: Dispute Resolution ERD

3.1.5.18 Reviews & Reputation

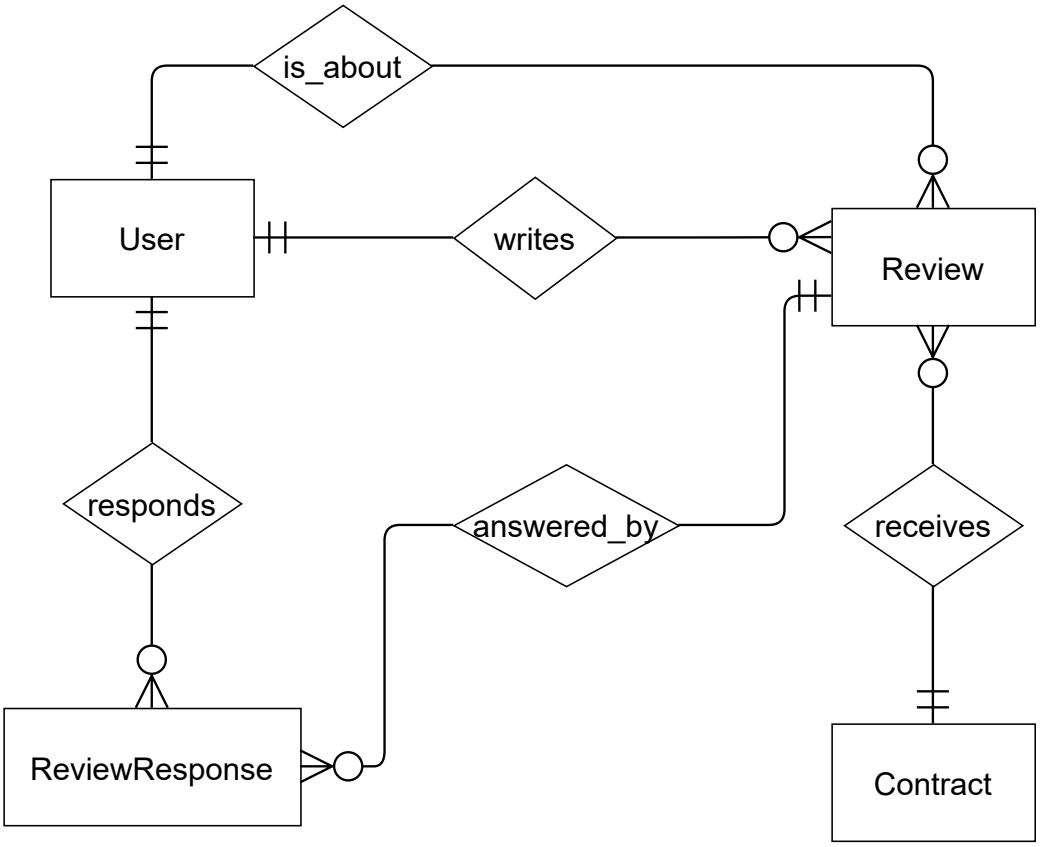


Figure 44: Reviews & Reputation ERD

3.1.5.19 Forum Posts & Comments

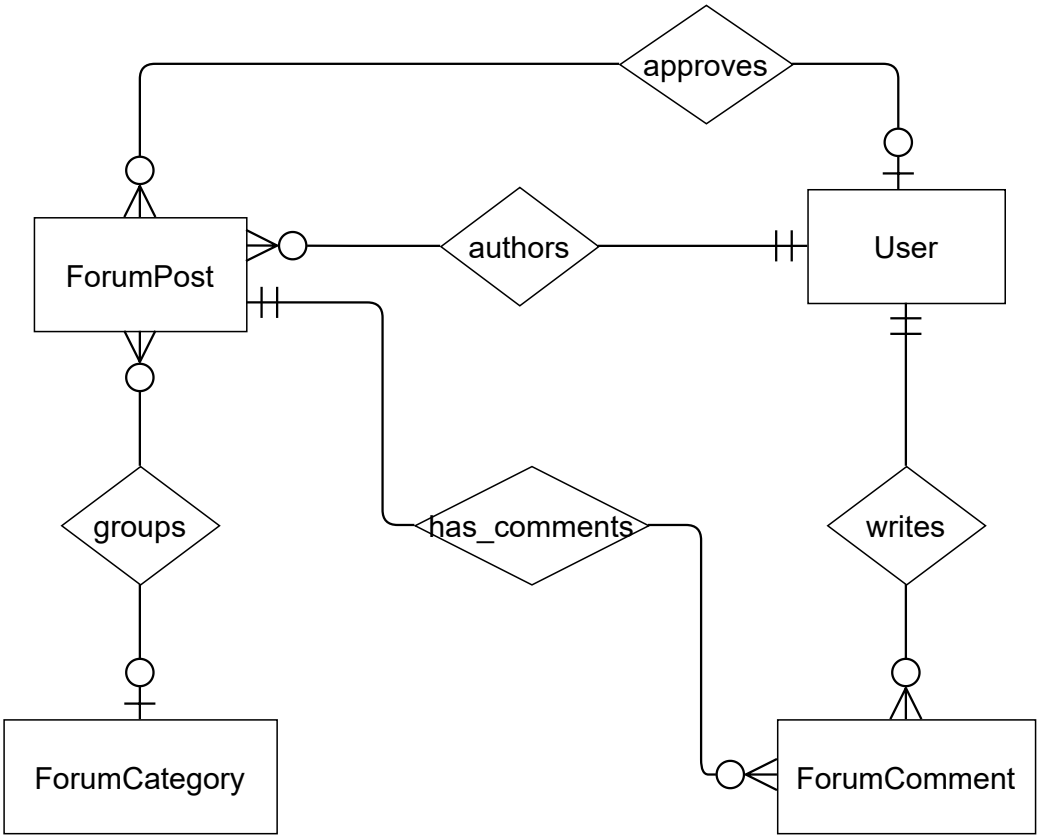


Figure 45: Forum Posts & Comments ERD

3.1.5.20 Forum Likes

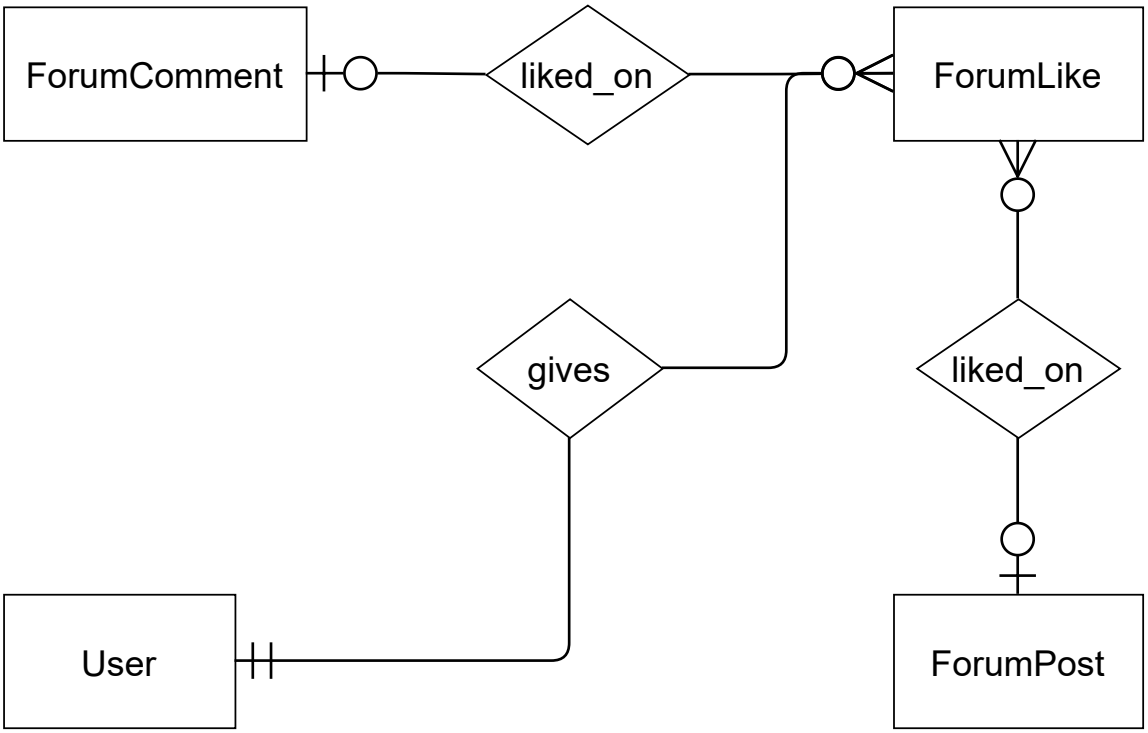


Figure 46: Forum Likes ERD

3.1.5.21 Forum Reports & Moderation

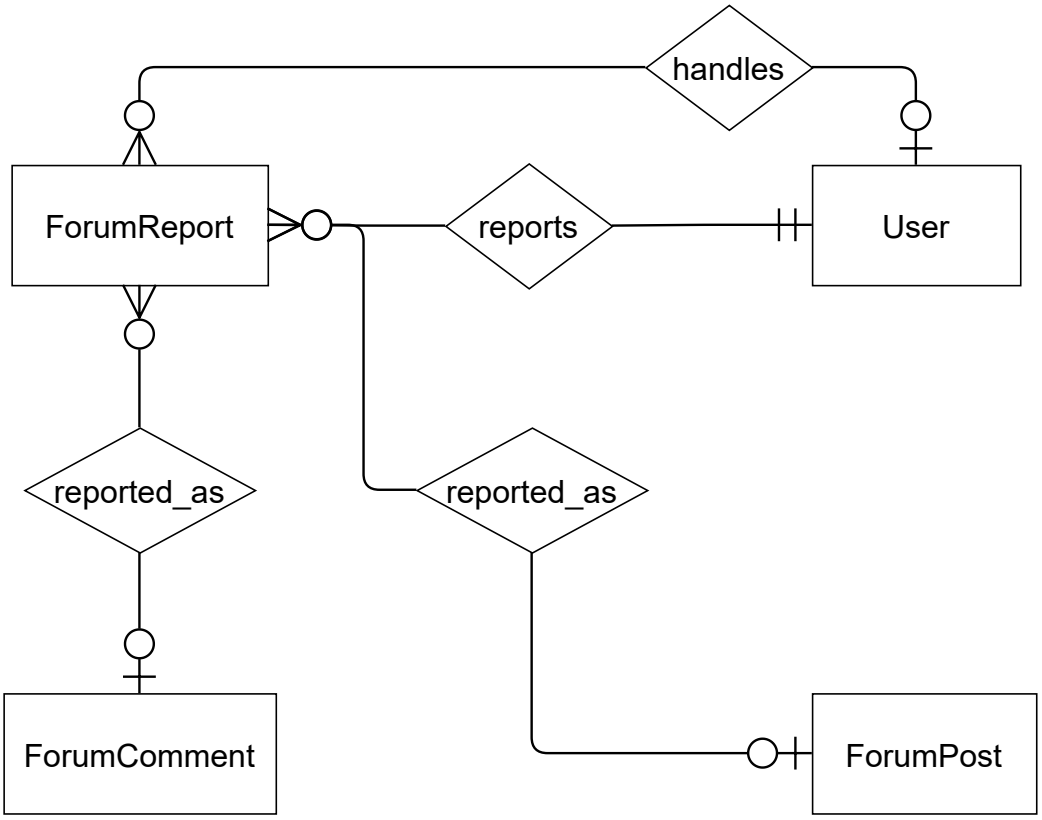


Figure 47: Forum Reports & Moderation ERD

3.1.5.22 Messaging & Notifications

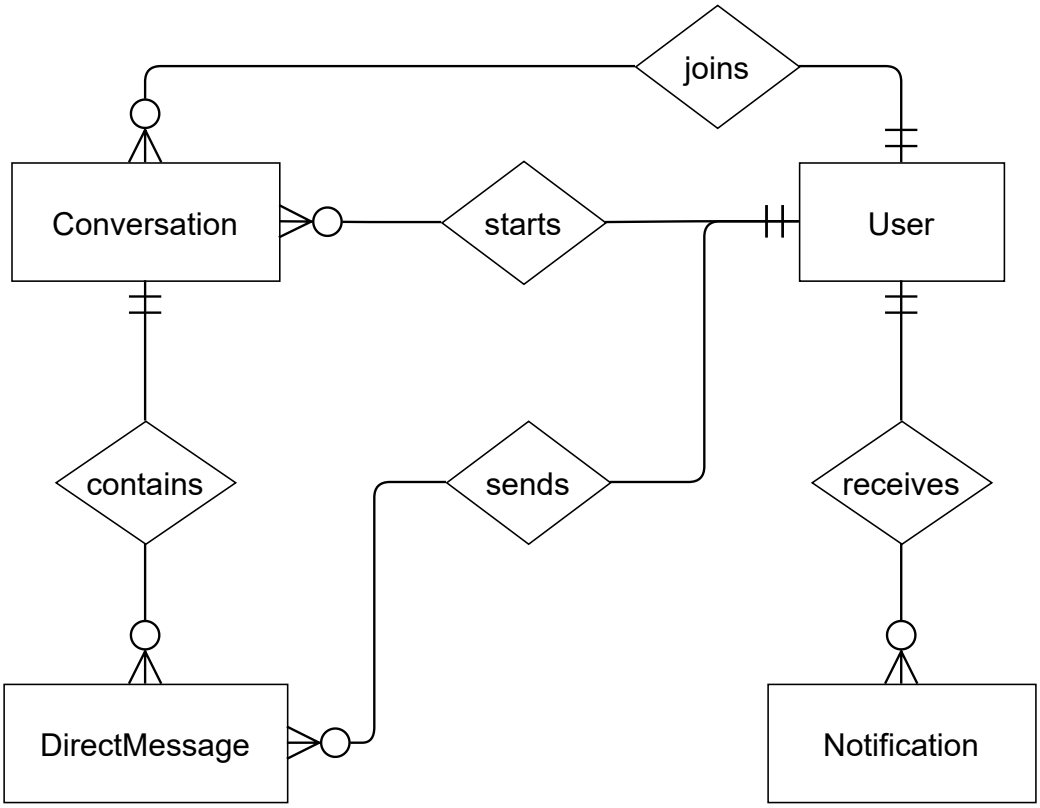


Figure 48: Messaging & Notifications ERD

3.1.5.23 Administration & System

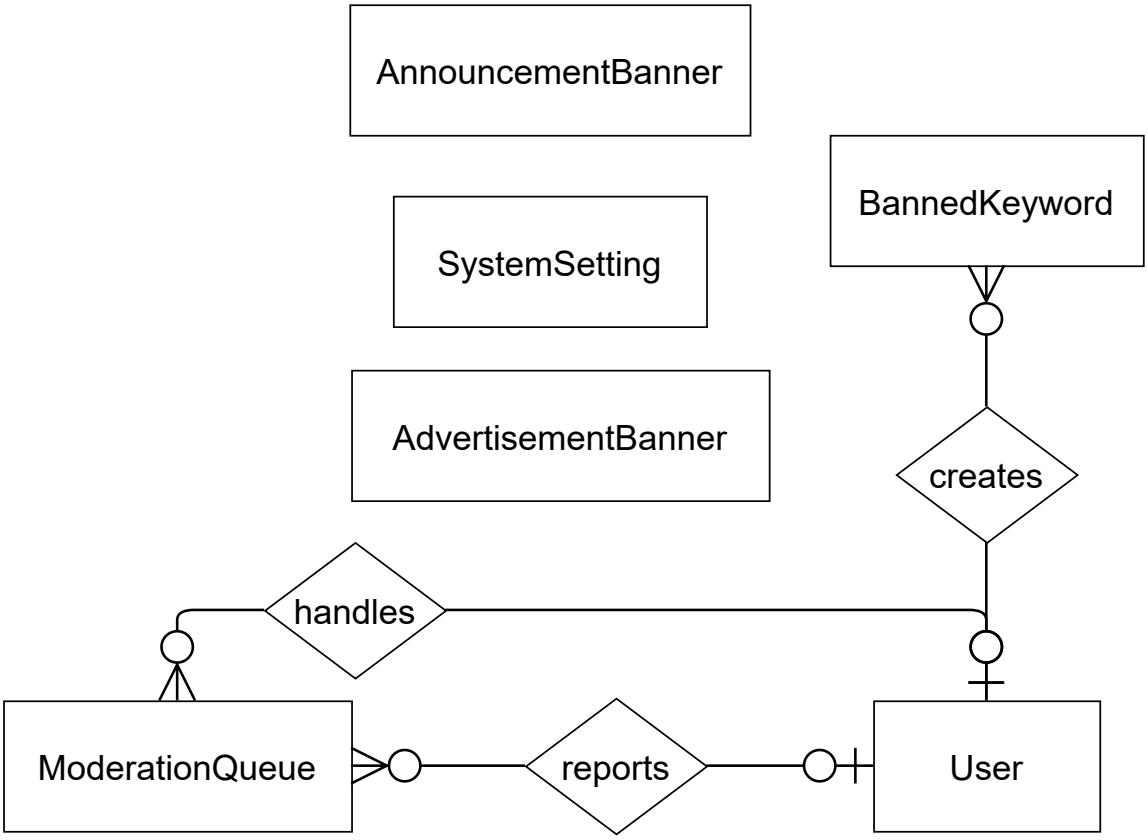


Figure 49: Administration & System ERD

3.2 Manage Forum

3.2.1 Function: View Forum Category List

Function Trigger

Any visitor opens the Community Forum area from the main navigation, or follows a direct link to the forum landing page. The function is public, therefore it runs on every visit of the page, whether or not the visitor is signed in.
Navigation path: Home → Forum (`/forum`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator. The screen is readable by every role, including visitors without an account.
- **Supporting Systems:** Frevia Web application (Next.js), Frevia BFF proxy (`/api/backend`), Frevia REST API (NestJS) and PostgreSQL accessed through Prisma ORM.

Purpose

The function is the entry point of the community area. It presents the active forum categories together with the number of published posts that each one contains, and it highlights both the categories and the members with the highest activity, so a visitor can reach an ongoing discussion in one click and can immediately see which topics are alive.

Interface

- Community Forum page at `/forum`, opened from the main navigation.
- Grid of category cards; each card shows the category name, its description and the number of approved posts.
- "Most Active" panel listing the three categories with the largest number of posts.
- "Top Contributors" panel listing the five members with the largest number of posts, with their post and comment counters.
- Empty-state panel displayed when no category has been created yet.

Data Processing Overview

- The page requests `GET /api/forums/categories`.
- The API keeps only the categories whose `deletedAt` is null and orders the result by `name` in ascending order.
- For every category the API counts the posts that are not soft deleted and whose `moderationStatus` is `APPROVED`; the count is returned as `postCount`, so posts awaiting moderation or rejected by it are never counted.
- The "Most Active" panel requests `GET /api/forums/categories/top?limit=3`; the API sorts the same projection by post count in descending order.
- The "Top Contributors" panel requests `GET /api/forums/users/top?limit=5`; the API selects the users that are neither deleted nor banned, counts their posts and comments with `deletedAt` null and orders them by post count in descending order.
- The category list is cached by the client for thirty minutes and is re-used while the visitor moves between forum screens, so returning to the list does not repeat the call.
- A category without a description is rendered with the fallback text "No description provided."; an empty result set is rendered as "No categories yet".

Screen Layout (Mock-up Description)

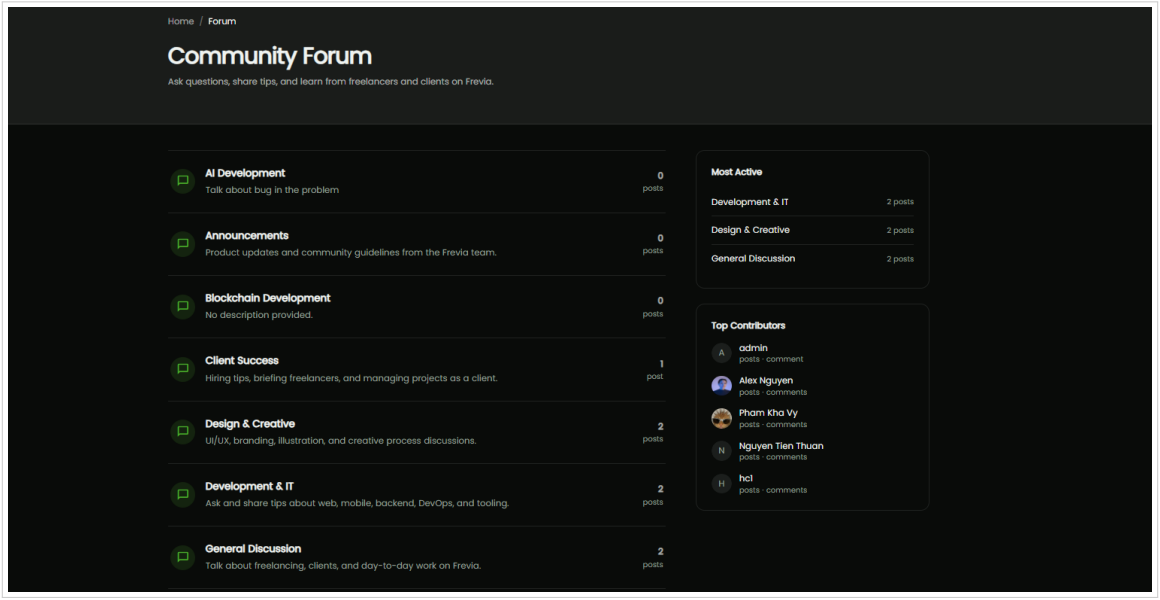


Figure 50: Forum category list screen

- The page header states the title “Community Forum” and a short description of the area, which introduces the forum to first-time visitors.
- Each category card is a link to the category detail screen and displays the category name, the description and the number of published posts.
- The “Most Active” panel ranks categories by post volume and gives a shortcut to the busiest discussions.
- The “Top Contributors” panel shows the avatar, the display name, the post count and the comment count of the most active members.
- When the list is empty, a placeholder explains that categories will appear once they are created, which happens after an administrator creates them.

Function Details

Input Data

Data Name	Description
Page route	`/forum`. No parameter is required to display the list.
`limit`	Number of records returned by the two ranking panels; integer between 1 and 10, default 3. It is only used by the "Most Active" and "Top Contributors" calls.
Visitor session	Optional access token forwarded by the BFF proxy. A guest receives the same list; the ranking panels do not expose any personal action.

Validation Rules

- The list endpoint takes no parameter, therefore it cannot be malformed; it always returns the complete list of visible categories.
- The `limit` of both ranking endpoints is coerced to an integer, must be at least 1 and at most 10, and defaults to 3 when it is absent.
- A category is visible only when its `deletedAt` column is null; a category moved to the trash is excluded from the list and from both rankings.
- A post contributes to a counter only when it is not soft deleted and its `moderationStatus` is `APPROVED`.
- A user appears in "Top Contributors" only when the account is not deleted and is not banned.

Business Rules

- **BR-FORUM-01**: Only categories with `deletedAt = NULL` shall be displayed.
- **BR-FORUM-02**: Categories shall be ordered by `name` in ascending order.
- **BR-FORUM-03**: `postCount` shall count only posts with `deletedAt = NULL` and `moderationStatus = APPROVED`.

Normal Flow

1. The visitor opens the Forum area from the navigation menu.
2. The web application renders the page shell and issues the category request.
3. The BFF proxy forwards the request to the REST API, adding the access token when the visitor is signed in.
4. The API queries the categories that are not soft deleted and counts their approved posts.
5. The API returns the ordered list of categories with `postCount`.
6. The client renders the category grid, applying the description fallback where needed.

7. The client issues the two ranking requests for the side panels.
8. The API returns the three most active categories and the five most active members.
9. The client renders both panels; the page is now fully displayed.
10. The visitor selects a category card and is routed to the category detail screen.

Abnormal Flow / Exception Handling

Case	Description
A1	The database cannot be queried. The API returns a server error carrying `Error.FailedToLoadForumCategories` and the page shows a generic error instead of the grid.
A2	No category exists or every category has been soft deleted. The page renders the empty state “No categories yet”; the ranking panels are hidden.
A3	A category has no description. The card shows “No description provided.” instead of an empty area.
A4	The ranking call fails while the list succeeds. The grid remains usable and only the affected panel is omitted, so navigation is never blocked.
A5	The visitor loses the network connection. The client displays the generic error message and offers a retry by reloading the page.
A6	The access token has expired. The BFF proxy refreshes the token once and repeats the call; if the refresh fails the visitor continues as a guest.
A7	The account was banned while the page was open. The BFF proxy detects the ban response, clears the authentication cookies and the client redirects to the sign-in page.

Output Data

Data Name	Description
`categories[]`	Visible categories ordered by name, each with `id`, `name`, `slug`, `description`, `createdAt`, `updatedAt` and `postCount`.
`categories` (top)	Up to three categories ordered by post count in descending order, with the same fields as the list.
`users` (top)	Up to five members with `id`, `displayName`, `avatarUrl`, `postCount` and `commentCount`.

Data Name	Description
Empty collection	An empty array when no category is visible; the client renders the empty state described above.

Post-Conditions

- The visitor sees every active category with the number of published posts it holds.
- No data is modified: the function is read-only.
- Deleted categories, deleted posts, rejected posts and pending posts are never exposed through this screen.

3.2.2 Function: View Forum Category Detail and Post List

Function Trigger

The visitor selects a category card on the forum landing page, follows a shared category link, or searches for a post from the category screen.
Navigation path: Home → Forum → *Category* (`/forum/{category-slug}-{category-id}`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lists the published posts of one category so that members can read, search and filter the discussions of a topic. It also gives signed-in members the entry point to create a post, to restrict the list to their own posts, and to reach the most discussed posts of the week.

Interface

- Breadcrumb Home → Forum → *Category* and the category header with its name, description and post count.
- Search box with the placeholder "Search posts…" and a clear-search button.
- "My Posts" toggle, displayed only to signed-in members.
- "New Post" button, which opens the post creation dialog; a guest is routed to the sign-in page instead.

- List of post cards with the title, an excerpt of the content, the author, the creation date, the like count and the comment count.
- Page-size selector, page numbers with ellipsis, and the “Showing x of y” summary.
- “Trending in {category}” panel listing the three most discussed posts of the last seven days.
- Empty states for no post, no search result and no personal post.

Data Processing Overview

- The page resolves the category from the slug in the URL and requests `GET /api/forums/posts?page=..`
- When the visitor submits the search box, the client navigates to the same route with a `search` parameter and resets the page to 1.
- When “My Posts” is active, the client adds the identifier of the signed-in member as the `userId` parameter.
- The API filters the posts whose `deletedAt` is null and whose `moderationStatus` is `APPROVED`, applies the category, author and title filters, and sorts the result by `createdAt` in descending order.
- Lists and counts are read in one database transaction, then the API returns the page together with the pagination block.
- The trending panel requests the top posts of the category for the last seven days, ordered by like count and then by comment count.
- Changing the page, the page size, the search term or the “My Posts” filter rewrites the query string, so the state of the list can be shared and restored by reloading.

Screen Layout (Mock-up Description)

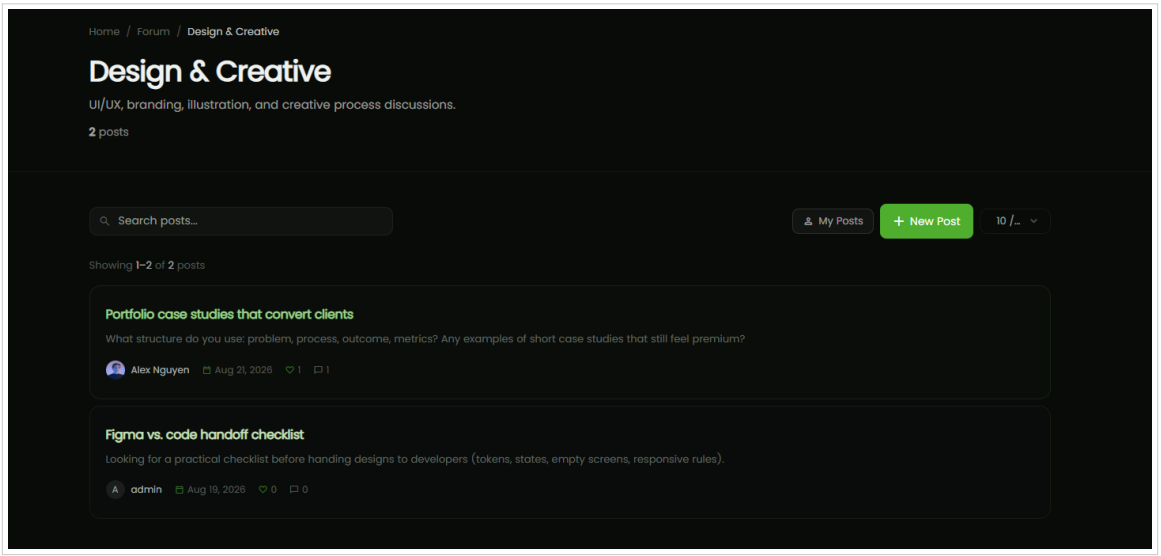


Figure 51: Category detail screen with the list of its posts

- The breadcrumb keeps the visitor oriented between the forum landing page, the current

category and the post that will be opened.

- The category header repeats the name and the description, which are maintained by the administrator who created the category.
- The toolbar groups the search box, the “My Posts” toggle and the “New Post” button; the search box submits on Enter and can be cleared with the cross icon.
- Every card shows the post title, the beginning of its content without the rich-text markup, the author avatar and name, the date, and the like and comment counters.
- Own posts are tagged with “(yours)” so a member can recognise them in the list.
- The footer of the list shows the pagination and the range of records currently displayed.

Function Details

Input Data

Data Name	Description
`categoryId`	Identifier of the category taken from the URL; it restricts the list to one category.
`page`	Page number, integer greater than or equal to 1, default 1.
`limit`	Page size, integer greater than or equal to 1, default 10; the client offers a selector and re-sends the request when it changes.
`search`	Optional search term applied to the post title, case insensitive.
`userId`	Optional author filter sent when “My Posts” is active; it always carries the identifier of the signed-in member.

Validation Rules

- `page` and `limit` are coerced to integers and must be at least 1; the default values are applied by the API when the parameters are absent.
- `categoryId` and `userId` must be positive integers when they are sent.
- `search` is optional and is compared with the title only, without case sensitivity; leading and trailing spaces are trimmed by the client.
- A requested page beyond the last page returns an empty list while the pagination block still reports the real total, so the client can render an empty state.
- Only approved and non-deleted posts are counted and displayed, therefore the totals never include posts that are pending, rejected or deleted.

Business Rules

- **BR-FORUM-06**: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed.
- **BR-FORUM-07**: Posts shall be sorted by `createdAt` in descending order.
- **BR-FORUM-08**: The system shall support pagination and configurable page sizes.
- **BR-FORUM-09**: `commentCount` shall count only comments with `deletedAt = NULL`.

Normal Flow

1. The visitor selects a category card on the forum landing page.
2. The client routes to the category screen and reads the page, the page size and the optional filters from the query string.
3. The client requests the posts of the category from the API.
4. The API validates the filter, resolves the pagination offset and runs the list and count queries in one transaction.
5. The API applies the visibility rules and orders the posts by creation date from the newest to the oldest.
6. The API returns the page of posts with their like and comment counters plus the pagination block.
7. The client renders the cards, strips the rich-text markup for the excerpt and applies the empty states when no card is returned.
8. The client requests the trending posts of the category and renders the side panel.
9. The visitor optionally searches, changes the page size, moves to another page or activates “My Posts”; each action re-issues the request with the new parameters.
10. The visitor selects a post card and is routed to the post detail screen.

Abnormal Flow / Exception Handling

Case	Description
A1	The category does not exist or was soft deleted. The API returns `Error.ForumCategoryNotFound` and the client displays the not-found state with a link back to the forum.
A2	The database query fails. The API returns `Error.FailedToLoadForumPosts` and the list area shows a generic error instead of the cards.
A3	The requested page is larger than the last page. The API returns an empty list and the client renders “No posts found” with the pagination intact.

Case	Description
A4	The search term matches no approved post. The client reports “No results for “{query}”. Try a different search.” and keeps the clear-search action available.
A5	“My Posts” is activated by a guest. The toggle is not rendered; if the parameter is forced in the URL the API answers with its validation error and the list is not displayed.
A6	The page size or page number is not a positive integer. The validation pipe rejects the request before the query is executed and the client falls back to the first page.
A7	The trending request fails. The list remains fully usable and only the side panel is omitted.
A8	The visitor loses the connection. The client shows the generic error message and the visitor can retry by reloading the current URL.

Output Data

Data Name	Description
`posts[]`	Visible posts of the requested page with `id`, `categoryId`, `userId`, `title`, `slug`, `content`, `moderationStatus`, `createdAt`, `updatedAt`, `likeCount`, `commentCount` and the author identity.
`pagination`	`page`, `limit`, `total` and `totalPages` computed from the filtered count.
`posts[]` (top)	Trending posts of the category with the interaction score used for the ranking.
Empty collection	An empty array when the page, the filter or the search matches no approved post.

Post-Conditions

- The visitor sees one page of approved posts of the selected category, newest first.
- The search, the page size and the author filter are reflected in the URL and can be restored by reloading the page.
- No data is modified: the function is read-only.

3.2.3 Function: View Post Detail

Function Trigger

The visitor selects a post card, follows a shared post link, or opens a related post from the side panel of another discussion.
Navigation path: Home → Forum → *Category* → *Post* (`/forum/{category}/{post-slug}-{post-id}`).

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function displays a complete discussion: the post with its formatted content and its author, the engagement counters, the list of members who liked it and the comment thread. It is the screen from which every write action on the discussion is started, and it shows the author-specific actions only to the author.

Interface

- Breadcrumb Home → Forum → *Category* → *Post title* and the line "Discussion in *Category*".
- Post title, author avatar, display name, creation date and the "Edited {date}" marker when the post was modified after its creation.
- Rich-text body rendered with headings, lists and links.
- Action bar with the like button and its counter, the comment counter, the "Report" action and, for the author, the "Edit" and "Delete" actions.
- Likes dialog and comment section described by the next functions.
- "Related Posts" side panel with other discussions of the same category.

Data Processing Overview

- The client resolves the post from the slug and requests `GET /api/forums/posts/by-slug/{slug}`.
- The API resolves the identifier of the post and returns the detail projection with the category summary and the author identity.
- Only a post that is not soft deleted and whose status is `APPROVED` can be resolved; a pending or rejected post answers with a not-found error.
- The client requests the likes of the post to compute the counter and to derive whether the signed-in member already liked it.

- The client requests the first fifty comments of the post to render the thread and the comment counter.
- The client requests the related posts of the category for the side panel.
- The page compares `updatedAt` with `createdAt` to decide whether the “Edited” marker is displayed, and compares the author identifier with the session identifier to decide whether the edit and delete actions are rendered.

Screen Layout (Mock-up Description)

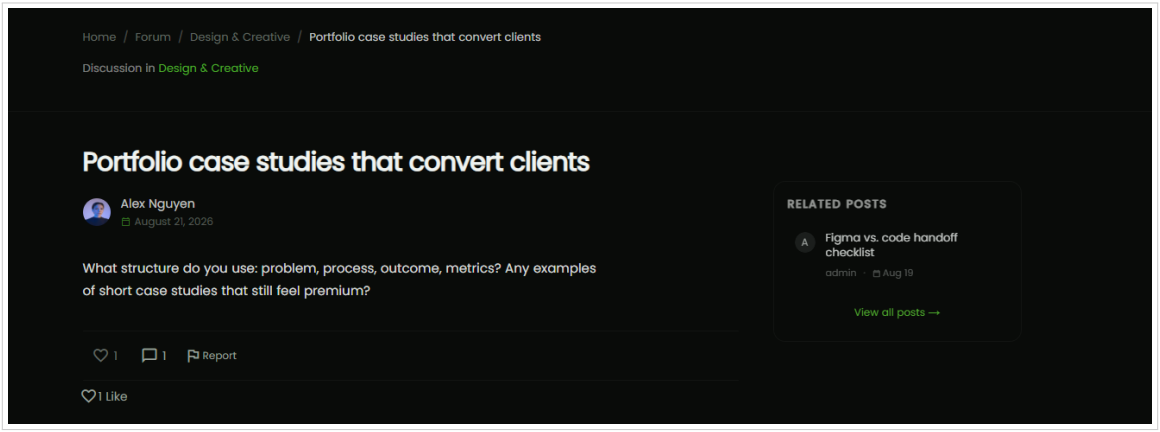


Figure 52: Post detail screen with the comment section

- The title block gives the discussion context and the path back to its category.
- The author block shows the avatar, the display name and the publication date; when the post was edited, the edition date is added next to the creation date.
- The body is rendered as formatted content, so headings, bullet lists and links written in the editor keep their appearance.
- The action bar holds the heart button with the like counter, the comment counter and the “Report” action; the heart is filled once the member liked the post.
- The edit and delete actions appear only for the author of the post.
- The comment section below the action bar lists the thread and gives the signed-in member the comment editor.
- The right side panel proposes related posts of the same category.

Function Details

Input Data

Data Name	Description
`slug`	Post slug taken from the URL; it uniquely identifies the discussion.
`postId`	Identifier resolved from the slug and used by the likes, comments and report calls of the same screen.
`page`, `limit`	Pagination of the comment thread; the detail screen requests the first page with a size of 50.
Session identity	Identifier of the signed-in member, used to compute the liked state and to decide whether the author actions are displayed.

Validation Rules

- The slug must resolve to a post that is not soft deleted and whose `moderationStatus` is `APPROVED`; otherwise the API answers with `Error.ForumPostNotFound`.
- The detail projection must contain a title and a content, which are required fields of the post model.
- The comment pagination follows the same rules as the post list: integers greater than or equal to 1, default page 1 and default size 10.
- The liked state is derived only from the likes that belong to the post and match the session identifier; a guest always sees the unliked state.
- The author actions are rendered only when the session identifier equals the post author identifier, never from a value supplied by the client.

Business Rules

- **BR-FORUM-10**: Only posts with `deletedAt = NULL` and `moderationStatus = APPROVED` shall be displayed.
- **BR-FORUM-11**: Post details shall include title, content, author, category and created date.
- **BR-FORUM-09**: `commentCount` shall count only comments with `deletedAt = NULL`.
- **BR-FORUM-28**: The like detail shall include the user identifier and the profile of each member who liked the post.

Normal Flow

1. The visitor selects a post card in the category screen.
2. The client extracts the slug from the URL and requests the post detail.
3. The API resolves the slug to a post that is visible and returns the detail projection.

4. The client renders the breadcrumb, the title, the author block and the formatted body.
5. The client requests the likes of the post and the first page of comments.
6. The API returns both collections with the profile of their authors.
7. The client computes the like counter, the liked state and the comment counter.
8. The client renders the action bar and exposes the author actions only to the author.
9. The client requests the related posts of the category and renders the side panel.
10. The visitor reads the discussion and can like the post, open the likes list, write a comment, report the content or, as the author, edit or delete the post.

Abnormal Flow / Exception Handling

Case	Description
A1	The slug does not resolve to any post. The client displays the not-found screen with the message “This post may have been deleted or the link is incorrect.” and the action “Back to category”.
A2	The post is still pending moderation or was rejected. The API hides it, therefore the same not-found screen is displayed and the post is never exposed before approval.
A3	The post was soft deleted by its author or by an administrator. The detail call answers with `Error.ForumPostNotFound` and the not-found screen is displayed.
A4	The likes request fails. The post body remains readable, the like counter falls back to zero and the action is disabled.
A5	The comments request fails. The thread area shows an error message while the post body and the action bar remain usable.
A6	The session expires while the page is open. The BFF proxy refreshes the token once; when the refresh fails the visitor continues as a guest without author actions.
A7	The visitor follows a link of a deleted category. The category is not found, so the client shows the not-found screen instead of a broken layout.
A8	The connection drops while the page loads. The client shows the generic error message and the visitor can retry by reloading.

Output Data

Data Name	Description
Post detail	`id`, `categoryId`, `userId`, `title`, `slug`, `content`, `moderationStatus`, `moderationScore`, `createdAt` and `updatedAt`.
`category`	Identifier, name and slug of the category the post belongs to, or null when the post has no category.
`user`	Identifier and profile of the author, with the display name and the avatar.
`likes[]`	Members who liked the post, with their identifier, reaction type, creation date and profile.
`comments[]`	First page of the thread with the content, the author, the dates, the like counter and the liked state of the current member.
`pagination`	Total number of visible comments and the pagination of the requested page.

Post-Conditions

- The visitor reads the complete discussion with its engagement counters.
- The action bar exposes only the actions allowed by the role and the ownership of the post.
- No data is modified by the read itself; the counters reflect only visible rows.

3.2.4 Function: Create Post

Function Trigger

A signed-in member selects the "New Post" button in the toolbar of a category screen, writes the post and confirms with the "Post" action. The button is rendered for Clients, Freelancers and Administrators; a guest who follows the button is routed to the sign-in page instead. **Navigation path:** Home → Forum → *Category* → New Post.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator uses the same dialog when browsing the forum.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, SensitiveAI moderation proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member start a new discussion inside a category. Because the content is user generated, every submission is screened by the moderation proxy before it reaches the forum, and the resulting post is stored with the moderation verdict so that only approved discussions become visible to the community.

Interface

- Modal dialog titled “Create Post in *Category*” with the description “Share your thoughts with the community.”
- “Title” input with the placeholder “What’s on your mind?”.
- “Content” rich-text editor with the placeholder “Write your post content here…” and the usual formatting tools.
- “Cancel” action, which discards the draft, and “Post” action, which submits it and shows a spinner while the request is running.
- Toast notification after the submission, either the confirmation of publication or the message that the post awaits moderator review.

Data Processing Overview

- The dialog trims the title and the content and blocks the submission while either of them is empty.
- The client posts the form to `/api/backend/forums/posts`.
- The BFF proxy recognises this exact route as the only forum route that must be screened and forwards it to the SensitiveAI moderation proxy instead of the REST API.
- The moderation proxy scans the text, attaches the verdict (status, score and categories) to the request body and forwards the request to the REST API.
- The API validates the body, verifies that the category exists and is not soft deleted, and creates the post.
- The slug is generated from the title, normalised to lower case without accents, and completed with a timestamp and a random suffix to keep it unique.
- The moderation status is taken from the verdict; when the proxy returns no verdict the post is stored as `PENDING`, which means it is invisible until an administrator approves it.
- The API returns the created post and the client invalidates the post lists of the category so the next visit shows the new discussion.
- When the returned status is `PENDING`, the client shows the moderation message and does not navigate, because the post detail would not be reachable yet; otherwise it navigates to the new post.

Screen Layout (Mock-up Description)

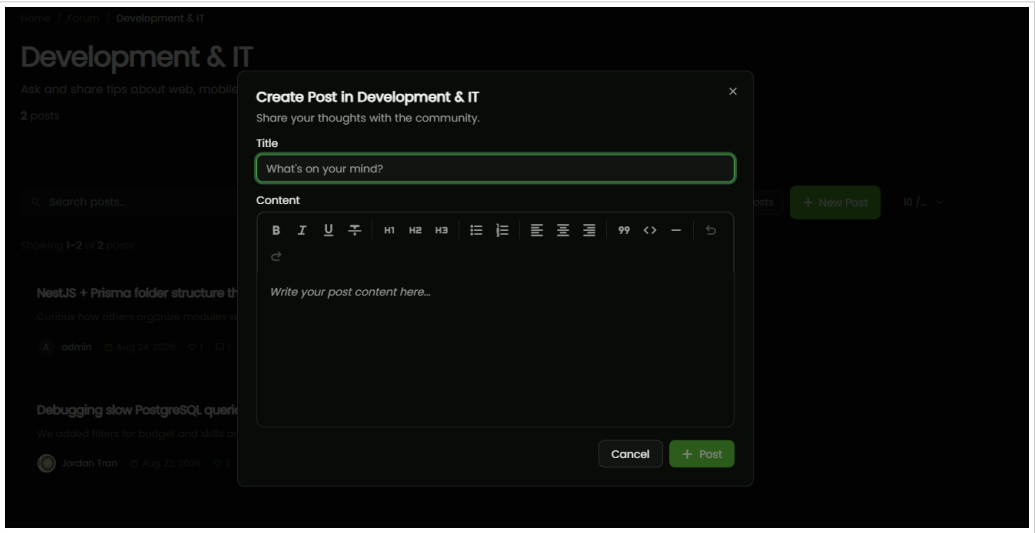


Figure 53: Create post dialog

- The dialog title names the category in which the post will be created, so the member cannot post in the wrong place by mistake.
- The title field is a single-line input; the content field is a rich-text editor that stores formatted HTML.
- The “Post” action stays disabled until both fields contain text, which prevents the most common validation error before the request is sent.
- The spinner replaces the icon of the action while the moderation proxy and the API are working, preventing a duplicate submission.
- The dialog closes only after a successful response; on failure it stays open with the draft intact.

Function Details

Input Data

Data Name	Description
`categoryId`	Identifier of the category in which the post is created; a positive integer sent by the dialog from the current category screen.
`title`	Post title, required, at most 255 characters in the database.
`content`	Post body, required; it is rich-text HTML produced by the editor.
`moderation`	Verdict attached by the moderation proxy: `status`, `score` between 0 and 1 and `categories`. It is an internal field, never typed by the member.
Access token	Bearer token forwarded by the BFF proxy; the API resolves the author from it, so the client cannot choose the author.

Validation Rules

- The title and the content are required and must contain at least one character after trimming.
- `categoryId` is optional for the API but must be a positive integer when it is present; the dialog always sends the category of the current screen.
- The category must exist and must not be soft deleted, otherwise the request fails with `Error.ForumCategoryNotFound`.
- The author is taken from the access token, never from the request body, so a member cannot create a post on behalf of somebody else.
- The moderation verdict must respect its own contract: a status among `PENDING`, `APPROVED` and `REJECTED`, and an optional score between 0 and 1.
- The generated slug must remain unique; the timestamp and random suffix guarantee that two posts with the same title never collide.

Business Rules

- **BR-FORUM-12**: Only authenticated users can create posts.
- **BR-FORUM-13**: Every post must belong to exactly one user and may belong to a category.
- **BR-FORUM-14**: Title and content are required fields.
- **BR-FORUM-15**: A new post shall default to `moderationStatus = PENDING` unless moderation data is provided.

Normal Flow

1. The member opens a category screen and selects “New Post”.
2. The client opens the dialog with the title of the target category.
3. The member writes a title and a content; the “Post” action becomes available.
4. The client trims both fields and sends the request through the BFF proxy.
5. The proxy routes the request to the moderation proxy, which scans the text.
6. The moderation proxy attaches its verdict and forwards the request to the REST API.
7. The API validates the body and checks that the category exists.
8. The API generates the slug, resolves the moderation status and inserts the post.
9. The API returns the created post to the client.
10. The client invalidates the cached lists of the category.
11. When the status is `APPROVED`, the client closes the dialog and navigates to the new post.
12. When the status is `PENDING`, the client closes the dialog, shows the message “Your post has been saved and is awaiting moderator review.” and returns the member to the list.

Abnormal Flow / Exception Handling

Case	Description
A1	The title or the content is empty after trimming. The action is disabled, and a forced request is rejected by the validation pipe with `Error.ForumPostTitleRequired` or `Error.ForumPostContentRequired`.
A2	The category was deleted while the dialog was open. The API answers `Error.ForumCategoryNotFound` and the dialog stays open with the draft so the member can choose another category.
A3	The member is not signed in or the session expired. The BFF proxy refreshes the token once; if the refresh fails the API rejects the request with an authentication error and the client redirects to the sign-in page.
A4	The moderation proxy is unavailable. The request fails before reaching the API, no post is created, and the client shows the generic error message.
A5	The moderation proxy flags the content. The post is stored with status `PENDING` or `REJECTED`, is not publicly visible, and the member is informed that the post awaits review.
A6	The moderation proxy answers that the request must be reviewed asynchronously. The client treats the response as a failure notification, shows the returned message and keeps the list unchanged.

Case	Description
A7	The insert fails at database level. The API returns `Error.FailedToCreateForumPost` and the client keeps the dialog open so nothing written by the member is lost.
A8	The member double clicks the action. The action is disabled while the mutation is pending, therefore only one request is sent.
A9	The member chooses “Cancel”. The dialog closes, the draft is cleared and no request is sent.

Output Data

Data Name	Description
`id`	Identifier of the created post, used by the client to build the detail route.
`slug`	Generated slug, unique for every post.
`categoryId`, `userId`	Category and author of the post.
`title`, `content`	Stored title and stored rich-text content.
`moderationStatus`	`APPROVED`, `PENDING` or `REJECTED`; it decides whether the post is immediately reachable.
`moderationScore`, `moderationCategories`	Verdict details kept for the moderation queue; the score may be null when the proxy did not compute one.
`createdAt`, `updatedAt`	Timestamps of the insert.

Post-Conditions

- One post is inserted and belongs to the signed-in member and to the selected category.
- The post carries a moderation status and a unique slug.
- An approved post becomes visible in the category list immediately after the cache is invalidated; a pending or rejected post stays hidden until an administrator resolves it.
- The post appears in the moderation queue of the administrator.

3.2.5 Function: Edit Post

Function Trigger

The author of a post opens the post detail screen and selects the "Edit" action placed next to the title. The action is rendered only for the author, therefore other members never see it.
Navigation path: Home → Forum → *Category* → *Post* → Edit.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer or Client who authored the post.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function allows the author to correct the title or the body of a discussion without losing the discussion itself, its comments and its likes. The post keeps its identity, its creation date and its engagement, and the screen marks it as edited so readers know that the content changed after publication.

Interface

- Modal dialog titled "Edit Post" with the description "Update your post title and content."
- Title input and rich-text editor pre-filled with the current values of the post.
- "Cancel" action, which leaves the post unchanged, and "Save Changes" action, which submits the new values.
- The post detail screen refreshes the title and the body in place after a successful save, and the author line starts showing "Edited {date}".

Data Processing Overview

- The client pre-fills the dialog from the post already loaded in the detail screen, so no additional read is required to open it.
- The client sends `PATCH /api/forums/posts/{id}` with the new title and content.
- The BFF proxy forwards the request directly to the REST API; only post creation goes through the moderation proxy, therefore an edition is not re-screened.
- The API loads the post with `deletedAt` null and rejects the request when the post does not exist.
- The API compares the author of the post with the identifier carried by the access token and rejects the request when they differ.

- The API updates the title, the content and the slug, then returns the updated post.
- The client updates the cached detail and the post lists of the category so the new title appears everywhere without a reload.

Screen Layout (Mock-up Description)

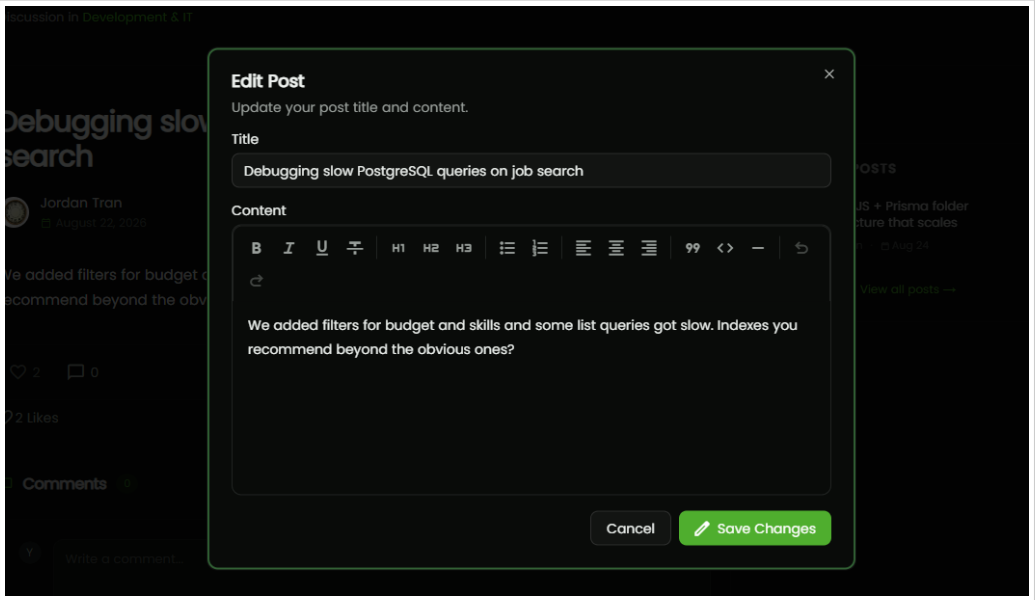


Figure 54: Edit post dialog with the current values

- The dialog reuses the layout of the creation dialog, which keeps the editing experience consistent with publishing.
- Both fields are pre-filled, so the member edits the existing text instead of rewriting it.
- The “Save Changes” action stays disabled while a field is empty, mirroring the requirement that both fields remain mandatory.
- A spinner is shown during the request; the dialog closes only on success.
- After the save, the detail screen shows the new title, the new body and the edition marker next to the publication date.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post taken from the current route.
`title`	New title, required, at most 255 characters in the database.
`content`	New body, required; rich-text HTML produced by the editor.
Access token	Identifies the requester; the API compares its identifier with the author of the post.

Validation Rules

- The post must exist and must not be soft deleted; otherwise the request fails with `Error.ForumPostNotFound`.
- The requester must be the author of the post; otherwise the request fails with `Error.ForumPostNotOwned`. The check is performed on the server, so hiding the action in the interface is only a convenience.
- The title and the content are required; an empty value is rejected by the validation pipe before the update is executed.
- The slug is regenerated from the new title on every update, which keeps the URL readable and unique while the post identifier stays unchanged.
- The category of the post is preserved unless a new category identifier is supplied; the dialog never changes it.

Business Rules

- **BR-FORUM-16**: Only the author of a post can edit that post.
- **BR-FORUM-17**: Title and content are required fields.
- **BR-FORUM-18**: The original post identifier remains unchanged after editing; the slug is regenerated when the title changes.

Normal Flow

1. The author opens the detail of a post they wrote.
2. The client renders the “Edit” and “Delete” actions next to the title.
3. The author selects “Edit” and the dialog opens with the current title and content.
4. The author changes one or both fields.
5. The client sends the update request with the access token.
6. The API loads the post and confirms that it exists and is not deleted.
7. The API confirms that the requester is the author.

8. The API updates the title, the content and the slug.
9. The API returns the updated post.
10. The client closes the dialog, refreshes the detail and invalidates the category lists.

Abnormal Flow / Exception Handling

Case	Description
A1	The post no longer exists or was deleted in the meantime. The API returns `Error.ForumPostNotFound` and the client shows the error and closes the dialog.
A2	The requester is not the author. The API returns `Error.ForumPostNotOwned`, the update is refused and the client displays the returned message.
A3	The title or the content is cleared before saving. The action is disabled, and a forced request is rejected by the validation pipe.
A4	The update fails at database level. The API returns `Error.FailedToUpdateForumPost` and the dialog stays open with the edited values.
A5	The author cancels the dialog. No request is sent and the post keeps its previous values.
A6	The session expires while the dialog is open. The BFF proxy refreshes the token once; when the refresh fails the request is rejected and the member is redirected to sign in.
A7	Two editions are submitted in a row. The action is disabled while the mutation is pending, so the last saved values are the ones returned by the server.

Output Data

Data Name	Description
`id`	Identifier of the post, which never changes.
`title`, `content`	New values stored in the post.
`slug`	Slug regenerated from the new title.
`updatedAt`	New edition timestamp, used by the detail screen to display the "Edited" marker.
`createdAt`	Original publication timestamp, preserved.

Post-Conditions

- The post contains the new title and the new content and keeps its author, category, comments and likes.
- The detail screen and the category list display the new values.
- The post is marked as edited because `updatedAt` differs from `createdAt`.
- No other post and no comment is modified.

3.2.6 Function: Delete Post

Function Trigger

The author of a post, or an Administrator, selects the "Delete" action on the post detail screen and confirms the operation in the confirmation dialog.
Navigation path: Home → Forum → *Category* → *Post* → Delete.

Function Description

Actors / Roles

- **Primary Actor:** the author of the post, or an Administrator acting on any post. The web interface renders the action for the author; the API also accepts an Administrator, who removes posts through the moderation and trash tools.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function removes a discussion from the public forum while preserving its record. Deletion is soft: the row receives a deletion timestamp and is filtered out of every public query, which keeps the data available for moderation, for the trash screen of the administrator and for any later audit.

Interface

- “Delete” action with a bin icon, rendered for the author and for the Administrator.
- Confirmation dialog titled “Delete Post” with the warning “Are you sure you want to delete this post? This action cannot be undone.”.
- “Cancel” action, which closes the dialog, and destructive “Delete” action, which performs the deletion.
- Redirect to the category screen after a successful deletion.

Data Processing Overview

- The client sends `DELETE /api/forums/posts/{id}` with the access token.
- The API loads the post with `deletedAt` null to confirm that it exists and is still visible.
- The API compares the author with the requester and also reads the role of the requester from the token.
- When the requester is neither the author nor an Administrator, the deletion is refused.
- When the check succeeds, the API writes the current date and time in the `deletedAt` column of the post.
- The client invalidates the post lists and navigates back to the category, where the post is no longer listed.

Screen Layout (Mock-up Description)

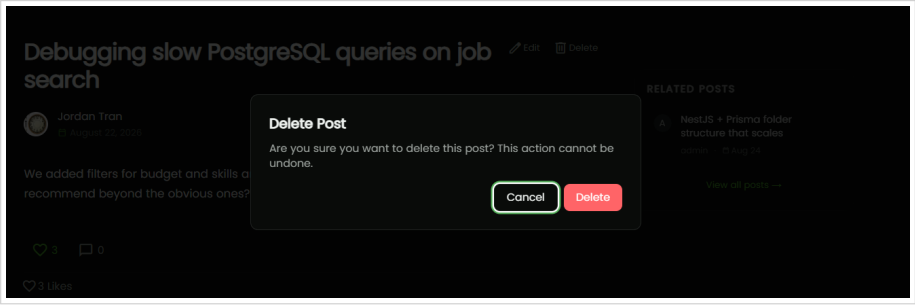


Figure 55: Delete post confirmation

- The confirmation dialog repeats the name of the object being deleted, which prevents deleting the wrong discussion.
- The warning states that the action cannot be undone from the member point of view, because the post disappears from the forum immediately.
- The destructive style of the confirmation action distinguishes it from the cancel action.
- After confirmation the screen returns to the category list, so the member sees the post removed from the discussion list.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post to delete, taken from the confirmation dialog.
Access token	Identifies the requester and carries the role used to authorize an Administrator.
`deletedAt`	Deletion timestamp written by the API; it is internal, not supplied by the client.

Validation Rules

- The post must exist and must not be already deleted; otherwise the API returns `Error.ForumPostNotFound`.
- The requester must be the author of the post or must hold the Administrator role; otherwise the API returns `Error.ForumPostNotOwned`.
- The deletion timestamp is produced by the server, so the client cannot backdate or falsify it.
- Only one row is affected per request; the comments and the likes of the post are preserved in the database and simply stop being reachable through the post.

Business Rules

- **BR-FORUM-19**: Only the author of a post or an Admin can delete that post.
- **BR-FORUM-20**: Post deletion shall be implemented as a soft delete using the `deletedAt` field.
- **BR-FORUM-21**: Soft-deleted posts shall not appear in public forum listings.

Normal Flow

1. The author or the Administrator opens the post detail screen.
2. The client renders the “Delete” action, which is available for this role.
3. The member selects the action and the confirmation dialog is displayed.
4. The member confirms the deletion.
5. The client sends the delete request with the access token.
6. The API loads the post and confirms that it exists and is not deleted.
7. The API confirms that the requester is the author or an Administrator.

8. The API writes the deletion timestamp in the post row.
9. The API returns the deleted post.
10. The client invalidates the lists, closes the dialog and routes back to the category.
11. The category list no longer contains the post and its counters are recomputed without it.

Abnormal Flow / Exception Handling

Case	Description
A1	The post does not exist or was already deleted. The API returns `Error.ForumPostNotFound` and the client shows the error message.
A2	A member tries to delete a post written by somebody else. The API returns `Error.ForumPostNotOwned` and nothing is deleted.
A3	An Administrator deletes the post of another member. The operation is allowed and the post leaves the public forum immediately.
A4	The database update fails. The API returns `Error.FailedToDeleteForumPost` and the post remains visible, so the member can retry.
A5	The member cancels the confirmation. No request is sent and the post is unchanged.
A6	The session expires before the confirmation. The request is rejected after the single token refresh attempt and the member is redirected to sign in.
A7	Another member opens the same post while it is being deleted. The next request returns `Error.ForumPostNotFound` and their screen shows the not-found state.

Output Data

Data Name	Description
`id`	Identifier of the deleted post.
`userId`, `categoryId`	Author and category of the deleted post, returned for confirmation.
`title`, `slug`	Title and slug of the deleted post.
`deletedAt`	Server timestamp recorded in the row; the post is soft deleted from this moment.

Post-Conditions

- The post is excluded from the category list, from the post detail and from the landing page counters.
- The row and its related comments and likes are retained for moderation and for the trash screen of the Administrator.
- The member is returned to the category screen, which reflects the new totals.

3.2.7 Function: Like a Post

Function Trigger

A signed-in member selects the heart button in the action bar of a post. A guest who selects the button is routed to the sign-in page instead of producing a request.
Navigation path: Home → Forum → *Category* → *Post* → Like.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also react while browsing.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member express appreciation for a discussion and lets the community measure which posts are relevant. It is implemented as a toggle: the same action adds the reaction when the member has not liked the post yet and removes it when the member already liked it, so a member can never like the same post twice.

Interface

- Heart button with the like counter, placed in the action bar under the post body.
- Outline heart for a post that the member has not liked, filled heart for a post that the member liked.
- The button is disabled while the request is running, which prevents repeated toggles from a single intention.

Data Processing Overview

- The client sends `POST /api/forums/posts/{id}/like` to the BFF proxy, which forwards it to the REST API.
- The API loads the post with `deletedAt` null and answers with `Error.ForumPostNotFound` when it is missing.

- The API looks for a like row that matches the member and the post.
- When the row exists, the API removes the reaction and answers with `liked: false`.
- When the row does not exist, the API inserts a like row with the reaction type `like` and answers with `liked: true`.
- The client invalidates the likes query of the post, which refreshes the counter and the filled state of the heart.
- The unique constraint of the like table on the pair member and post guarantees that a duplicate reaction cannot be stored, even if two requests are issued at the same time.

Screen Layout (Mock-up Description)

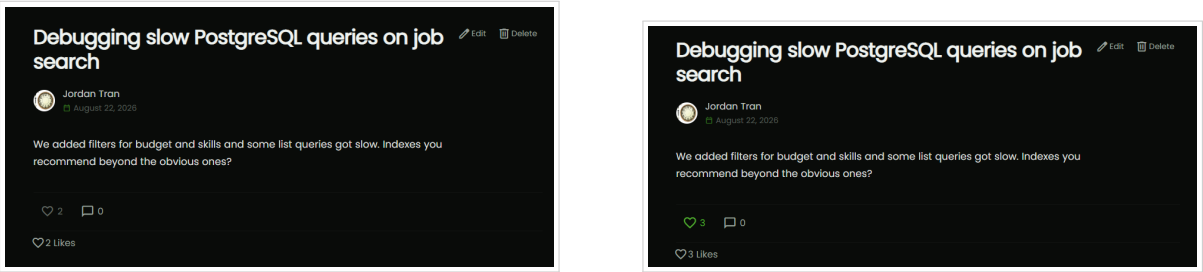


Figure 56: Post before and after being liked

- Before the reaction the heart is outlined and the counter shows the current number of likes.
- After the reaction the heart is filled, coloured with the accent of the application and slightly enlarged, and the counter is incremented by one.
- The counter is the single source of truth for the likes list: the same number is shown on the detail screen, in the likes dialog and on the post cards of the category.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post taken from the current route.
Access token	Identifies the member who reacts; the API derives the like owner from it.
Reaction type	Constant value `like` stored in the reactivity row; the interface does not offer another reaction for the moment.

Validation Rules

- The requester must be authenticated; a request without a valid token is rejected with an authentication error.
- The post must exist and must not be soft deleted.
- A member can hold only one reaction per post, enforced by the unique constraint on the pair member and post.
- The reaction is always attached to the member of the token; the client cannot send a different member identifier.

Business Rules

- **BR-FORUM-22**: Only authenticated users can like posts.
- **BR-FORUM-23**: A user can have only one active like reaction per post.
- **BR-FORUM-24**: The reaction shall be stored in the `ForumLike` table.
- **BR-FORUM-25 to BR-FORUM-27**: removing a reaction is allowed only when the corresponding like record exists, and it deletes that record.

Normal Flow

1. The signed-in member opens a post they have not liked yet.
2. The client displays an outlined heart next to the current counter.
3. The member selects the heart and the client sends the toggle request.
4. The API confirms that the post is visible.
5. The API finds no reaction of this member on this post.
6. The API inserts the like row.
7. The API answers with `liked: true`.
8. The client refetches the likes, fills the heart and increments the counter.
9. When the member selects the heart again, the same sequence removes the row and the interface returns to the outlined heart with the previous counter.

Abnormal Flow / Exception Handling

Case	Description
A1	The post does not exist or was deleted. The API returns `Error.ForumPostNotFound` and the counter is left unchanged.
A2	The member is not signed in. The client routes to the sign-in page before any request is sent, and a direct request is rejected with an authentication error.

Case	Description
A3	The member already liked the post. The request removes the reaction instead of adding a second one, which is the intended toggle behaviour.
A4	The insert fails at database level. The API returns `Error.FailedToLikeForumPost` and the interface keeps the previous state.
A5	The member double clicks the heart. The button is disabled while the request runs, so only one reaction is created.
A6	The likes query cannot be refreshed after a successful toggle. The heart keeps the state returned by the action and the counter is refreshed on the next visit.

Output Data

Data Name	Description
`liked`	`true` when the reaction was added, `false` when it was removed.
Like row	Member, post, reaction type and creation date of the stored reaction.
Updated counter	The like counter of the post, recomputed by the client from the refreshed likes list.

Post-Conditions

- The post holds exactly one reaction of the member, or none when the action removed it.
- The counter displayed on the post matches the number of likes stored for that post.
- The member appears in, or disappears from, the likes list of the post.

3.2.8 Function: View Post Likes

Function Trigger

A visitor selects the likes counter under the post body, which opens the list of the members who liked the post. The function is public, therefore it works for guests as well.
Navigation path: Home → Forum → *Category* → *Post* → Likes.

Function Description

Actors / Roles

- **Primary Actor:** Guest, Freelancer, Client and Administrator.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function answers the question “who liked this post”. It also feeds the state of the like button, because the detail screen derives both the counter and the liked state of the current member from the same list.

Interface

- Dialog titled “Likes” with the description “People who liked this post.”.
- Row per member with the avatar, the display name and, when the profile is incomplete, a fallback name built from the member identifier.
- Empty message “No likes yet.” when the post has not been liked.

Data Processing Overview

- The client calls `GET /api/forums/posts/{id}/like` and caches the result under a key that contains the post identifier.
- The API reads every like row of the post and joins the profile of the member.
- The payload contains the identifier, the member, the reaction type and the creation date of the reaction.
- The detail screen uses the same payload to count the likes and to detect the reaction of the signed-in member by comparing identifiers.
- The dialog renders the rows in the order returned by the query, which follows the insertion order of the reactions.

Screen Layout (Mock-up Description)

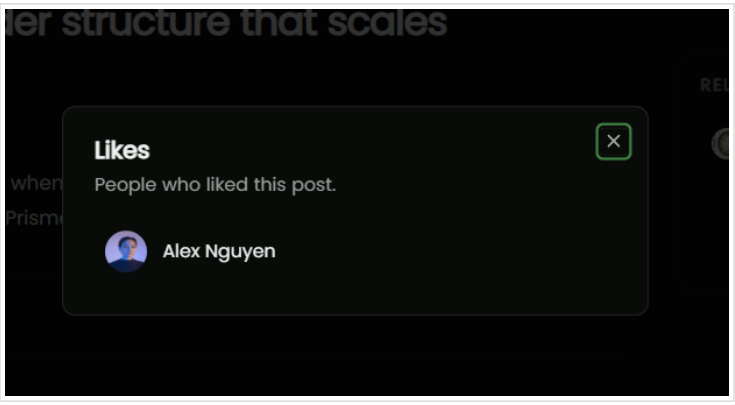


Figure 57: Likes dialog listing the members who liked the post

- The likes counter placed under the post body opens the dialog and states how many members liked the discussion.
- Each row shows the avatar and the display name of a member, which links the reaction to a visible identity of the community.
- When nobody liked the post, the dialog states “No likes yet.” instead of showing an empty panel.

Function Details

Input Data

Data Name	Description
`id`	Identifier of the post whose reactions are listed.
Session identity	Optional; used by the detail screen to highlight the role of the current member in the list, and not needed by the endpoint itself.

Validation Rules

- The post identifier must be a positive integer accepted by the route parameter pipe.
- The endpoint is public, so no token is required and no authorization rule applies.
- A post without reaction returns an empty array, which is a valid answer and not an error.
- Profile fields may be null; the interface then substitutes the fallback name and the initial of the display name for the avatar.

Business Rules

- **BR-FORUM-28**: The like detail shall include the user identifier and the user profile information for each liker.
- **BR-FORUM-23**: A user can have only one active like reaction per post, therefore a member never appears twice in the list.

Normal Flow

1. The visitor opens a post detail screen.
2. The client requests the likes of the post to compute the counter.
3. The API returns every reaction of the post with the profile of the member.
4. The client shows the counter under the post body.
5. The visitor selects the counter and the dialog opens with the list of members.
6. The visitor closes the dialog and returns to the discussion.

Abnormal Flow / Exception Handling

Case	Description
A1	The post has no reaction. The dialog shows “No likes yet.” and the counter stays at zero.
A2	A member has no profile record. The row shows the fallback name “User #{id}” and the initial letter in place of the avatar.
A3	The request fails. The API returns `Error.FailedToViewForumLikeDetail`, the dialog shows an error message and the counter keeps its last known value.
A4	The post was deleted while the dialog was open. The list refresh returns no row and the dialog shows the empty state.
A5	The visitor is offline. The dialog shows the generic error message and can be closed, which leaves the rest of the screen usable.

Output Data

Data Name	Description
`likes[]`	One entry per reaction with `id`, `userId`, `postId`, `commentId`, `reactionType` and `createdAt`.
`user`	Identifier, display name and avatar of the member who reacted.
Derived count	Length of the list, displayed as the likes counter of the post.

Post-Conditions

- The visitor sees every member who liked the post, without duplicates.
- The like counter of the post matches the number of rows returned.
- No data is modified: the function is read-only.

3.2.9 Function: Create Comment

Function Trigger

A signed-in member writes in the comment box placed under the post body and submits it with the "Comment" action or with the `Ctrl+Enter` shortcut. A guest sees an invitation to sign in instead of the editor.
Navigation path: Home → Forum → *Category* → *Post* → Comments.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also comment while browsing the forum.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member answer a discussion, ask a question or share an experience inside the thread of a post. Comments are attached to one post and one author, can be edited and deleted by their owner, and carry their own like counter.

Interface

- Comment card between the post body and the thread, with the avatar of the member and a text area using the placeholder "Write a comment…".

- Hint “Press `Ctrl+Enter` to submit” below the text area.
- “Comment” action with a send icon, disabled while the field is empty or a request is running.
- Guest state: the editor is replaced by the sentence “Log in to join the discussion.” with a link to the sign-in page.

Data Processing Overview

- The client trims the text and sends `POST /api/forums/posts/{postId}/comments` with the content.
- The BFF proxy forwards the request to the REST API, which validates the body.
- The API verifies that the post exists and is not soft deleted.
- The API inserts the comment with the author taken from the access token.
- The response contains the new comment with its author profile, a like counter of zero and an unliked state, because a comment cannot be liked before it exists.
- The client clears the editor and invalidates the comment queries of the post, which refreshes the thread and the comment counter.

Screen Layout (Mock-up Description)

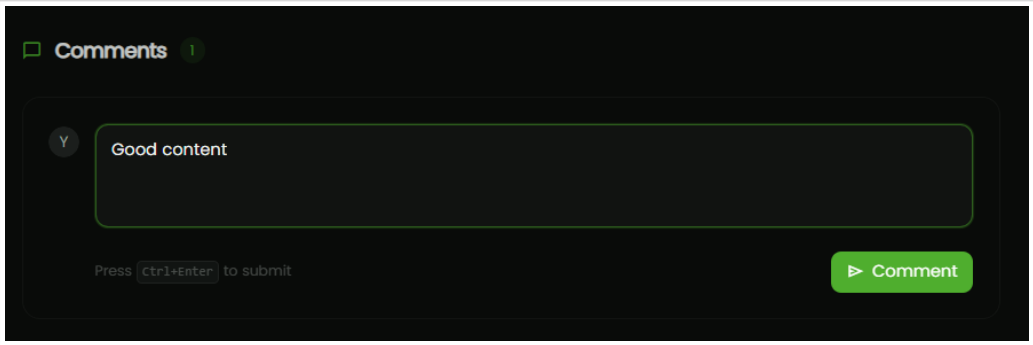


Figure 58: Comment editor under the post body

- The editor sits directly under the action bar, so the member writes without leaving the discussion.
- The counter next to the “Comments” heading shows how many comments the post currently holds.
- The “Comment” action stays disabled while the field is empty, which prevents empty submissions.
- The hint about `Ctrl+Enter` makes the keyboard shortcut discoverable.
- Every submitted comment appears at the top of the thread, with the avatar, the display name, the date and the like and edit actions.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post taken from the route; the comment is attached to it.
`content`	Comment text, required, at least one character after trimming.
Access token	Identifies the author of the comment; it is read by the API and never sent in the body.

Validation Rules

- The requester must be authenticated; the API rejects a request without a valid token.
- The content must not be empty; the validation pipe rejects an empty or whitespace-only body.
- The post must exist and must not be soft deleted, otherwise the request fails with `Error.ForumPostNotFound`.
- The author of the comment is always the member of the token, so a member cannot post a comment on behalf of somebody else.

Business Rules

- **BR-FORUM-32**: Only authenticated users can create comments.
- **BR-FORUM-33**: Comment content cannot be empty.
- **BR-FORUM-34**: Each comment shall be associated with a valid post and user.

Normal Flow

1. The signed-in member opens a post and scrolls to the comment section.
2. The client renders the editor with the placeholder and the hint.
3. The member writes the comment and submits it with the action or the shortcut.
4. The client trims the text and sends the request.
5. The API validates the content and loads the post.
6. The API inserts the comment with the author of the session.
7. The API returns the created comment with an empty like state.
8. The client clears the editor and refreshes the thread.
9. The new comment is displayed at the top of the list and the counter is incremented.

Abnormal Flow / Exception Handling

Case	Description
A1	The content is empty. The action is disabled, and a forced request is rejected by the validation pipe without touching the database.
A2	The post does not exist or was deleted. The API returns `Error.ForumPostNotFound` and the client displays the error message while keeping the draft.
A3	The member is not signed in. The editor is replaced by the sign-in invitation, and a direct request is rejected with an authentication error.
A4	The insert fails at database level. The API returns `Error.FailedToCreateForumComment` and the editor keeps the text so it is not lost.
A5	The member submits twice in a row. The action is disabled while the request runs, which prevents duplicated comments.
A6	The session expires during the submission. The BFF proxy attempts one refresh; when it fails the request is rejected and the member is redirected to sign in.

Output Data

Data Name	Description
`id`	Identifier of the created comment.
`postId`	Post the comment belongs to.
`content`	Stored comment text.
`user`	Identifier, display name and avatar of the author.
`likeCount`, `likedByMe`	Zero and false for a new comment.
`createdAt`, `updatedAt`	Timestamps of the insert.

Post-Conditions

- One comment is stored and belongs to the signed-in member and to the post.
- The thread and the comment counter of the post reflect the new comment.
- The comment is immediately visible to every visitor of the post.

3.2.10 Function: Update Comment

Function Trigger

The author of a comment selects the pencil action next to their comment, edits the text and confirms the change. The action is rendered only for the author of the comment.
Navigation path: Home → Forum → *Category* → *Post* → Comments → Edit.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer or Client who authored the comment.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function lets a member correct a comment after publication, for example to fix a typo or to add a clarification, without deleting the reply and losing its position in the thread. The comment keeps its identifier, its author and its likes, and the new text replaces the previous one.

Interface

- Pencil action on the header row of the comment, next to the author and the date.
- Inline editor pre-filled with the current text of the comment.
- "Save" action to confirm and "Cancel" action to discard the change.
- The thread is refreshed after the save, and the updated comment is displayed with its new text.

Data Processing Overview

- The client sends `PATCH /api/forums/posts/{postId}/comments/{commentId}` with the new content.
- The API loads the comment with `deletedAt` null and rejects the request when it does not exist.
- The API verifies that the comment belongs to the post of the route, so a comment cannot be edited through the route of another post.
- The API compares the author of the comment with the member of the token and refuses the request when they differ.
- The API updates the content and returns the comment with its author profile.
- The client invalidates the comment queries of the post and re-renders the thread.

Screen Layout (Mock-up Description)

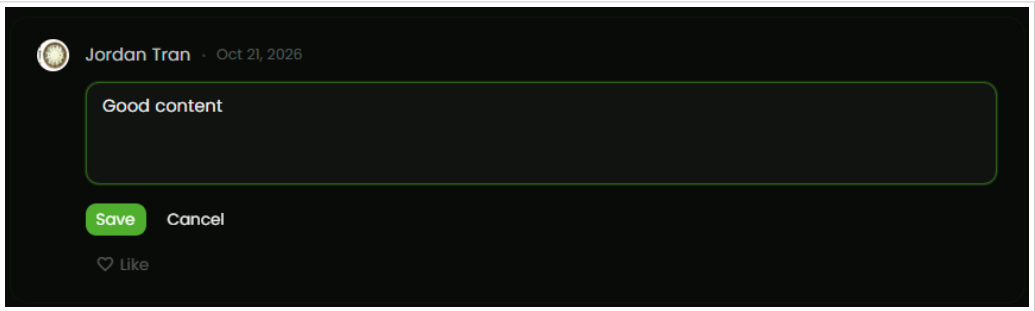


Figure 59: Inline editor of an existing comment

- The editor replaces the comment body in place, so the member keeps the context of the thread while editing.
- The existing text is pre-filled, which makes small corrections fast.
- The save action is disabled while the text is empty, because the content stays a required field.
- After the save the comment shows the edited text and the thread keeps the same order.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to update.
`content`	New comment text, required, at least one character.
Access token	Identifies the requester; the API compares it with the comment author.

Validation Rules

- The comment must exist and must not be soft deleted; otherwise the request fails with `Error.ForumCommentNotFound`.
- The comment must belong to the post of the route; a mismatch is reported as a missing comment, which hides the existence of the comment through another route.
- The requester must be the author of the comment; otherwise the request fails with `Error.ForumCommentNotOwner`.
- The content is required and must not be empty after trimming.

- The update touches the content only: the author, the post, the creation date and the likes of the comment are preserved.

Business Rules

- **BR-FORUM-35**: Only the owner of a comment can edit it.
- **BR-FORUM-36**: Comment content cannot be empty.
- **BR-FORUM-37**: A comment must belong to the specified post to be edited.

Normal Flow

1. The author of a comment selects the pencil action on their own comment.
2. The client opens the inline editor with the current text.
3. The author modifies the text and confirms.
4. The client sends the update request with the access token.
5. The API loads the comment and confirms that it exists.
6. The API confirms that the comment belongs to the post of the route.
7. The API confirms that the requester is the author.
8. The API stores the new content.
9. The API returns the updated comment.
10. The client refreshes the thread and displays the new text.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was deleted. The API returns `Error.ForumCommentNotFound` and the editor closes with an error message.
A2	The comment belongs to another post. The request is refused with the same not-found error, which prevents editing a comment through a foreign route.
A3	The requester is not the author. The API returns `Error.ForumCommentNotOwner` and the text is not changed.
A4	The new text is empty. The save action is disabled, and a forced request is rejected by the validation pipe.
A5	The update fails at database level. The API returns `Error.FailedToUpdateForumComment` and the editor keeps the text.

Case	Description
A6	The author cancels the edition. No request is sent and the previous text is restored.

Output Data

Data Name	Description
`id`	Identifier of the updated comment.
`postId`	Post the comment belongs to, unchanged.
`content`	New comment text.
`user`	Identifier and profile of the author, unchanged.
`updatedAt`	New edition timestamp.
`createdAt`	Original creation timestamp, preserved.

Post-Conditions

- The comment holds the new text and keeps its position, its author and its likes.
- The thread displays the updated comment after the refresh.
- No other comment and no post is modified.

3.2.11 Function: Delete Comment

Function Trigger

The author of a comment, or an Administrator, selects the bin action next to a comment and confirms the deletion in the confirmation dialog.
Navigation path: Home → Forum → *Category* → *Post* → Comments → Delete.

Function Description

Actors / Roles

- **Primary Actor:** the author of the comment, or an Administrator acting on any comment. The web interface exposes the action to the author of the comment; the API also accepts an Administrator, who deletes comments through the moderation tools.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function removes a reply from the thread. As for posts, the deletion is soft: the comment receives a deletion timestamp and is filtered out of every public query, so the record remains available for moderation and for the trash screen of the Administrator while the discussion stays coherent for the readers.

Interface

- Bin action on the header row of the comment, next to the edit action.
- Confirmation dialog titled “Delete Comment” with the warning “Are you sure you want to delete this comment? This action cannot be undone.”.
- “Cancel” action and destructive “Delete” action.
- The thread is refreshed after the deletion and the comment counter decreases.

Data Processing Overview

- The client sends `DELETE /api/forums/posts/{postId}/comments/{commentId}`.
- The API loads the comment with `deletedAt` null and rejects the request when it is missing.
- The API verifies that the comment belongs to the post of the route.
- The API accepts the requester when it is the author or when the token carries the Administrator role.
- The API writes the deletion timestamp in the comment row.
- The client invalidates the comment queries and refreshes the thread and the counter.

Screen Layout (Mock-up Description)

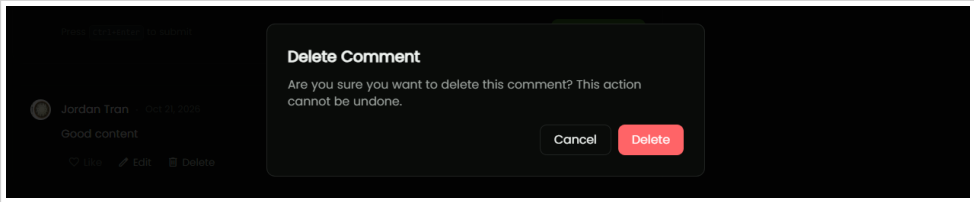


Figure 60: Delete comment confirmation

- The confirmation dialog names the object, so a member who owns several comments cannot delete the wrong one.
- The destructive style of the confirm action separates it from the cancel action and warns about the consequence.
- After the confirmation the comment disappears from the thread and the comment counter of the post is updated.
- The remaining comments keep their order, because the thread is sorted by creation date.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to delete.
Access token	Identifies the requester and carries the role used to authorize an Administrator.
`deletedAt`	Deletion timestamp written by the server, not supplied by the client.

Validation Rules

- The comment must exist and must not be already deleted; otherwise the request fails with `Error.ForumCommentNotFound`.
- The comment must belong to the post of the route; a mismatch is reported as a missing comment.
- The requester must be the author of the comment or must hold the Administrator role; otherwise the request fails with `Error.ForumCommentNotOwner`.
- The deletion timestamp is produced by the server.
- The post itself is not deleted, and the likes of the comment are kept in the database.

Business Rules

- **BR-FORUM-38**: Only the owner of a comment or an Admin can delete it.
- **BR-FORUM-39**: Comment deletion shall be implemented as a soft delete using the `deletedAt` field.
- **BR-FORUM-40**: Soft-deleted comments shall not be displayed in normal comment listings.

Normal Flow

1. The author or the Administrator opens the post detail screen.
2. The client renders the bin action on the comments the member may delete.
3. The member selects the action and the confirmation dialog appears.
4. The member confirms the deletion.
5. The client sends the delete request with the access token.
6. The API loads the comment and confirms that it exists.

7. The API confirms that the comment belongs to the post of the route.
8. The API confirms that the requester is the author or an Administrator.
9. The API writes the deletion timestamp.
10. The API returns the deleted comment.
11. The client refreshes the thread, where the comment is no longer present.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was already deleted. The API returns `Error.ForumCommentNotFound` and the client shows the error message.
A2	A member tries to delete a comment written by somebody else. The API returns `Error.ForumCommentNotOwner` and nothing is deleted.
A3	An Administrator deletes a comment of another member. The operation is allowed and the comment disappears from the thread.
A4	The comment belongs to another post. The request is refused with the not-found error, which prevents deleting through a foreign route.
A5	The delete fails at database level. The API returns `Error.FailedToDeleteForumComment` and the comment remains visible.
A6	The member cancels the confirmation. No request is sent and the comment is unchanged.
A7	The session expires before the confirmation. The single refresh attempt decides between a successful retry and a redirect to the sign-in page.

Output Data

Data Name	Description
`id`	Identifier of the deleted comment.
`postId`	Post the comment belonged to.
`content`	Text of the deleted comment, returned for confirmation.
`deletedAt`	Server timestamp recorded in the row.
Updated counter	The comment counter of the post, recomputed without the deleted comment.

Post-Conditions

- The comment is excluded from the thread and from every comment count.
- The row is retained for moderation and for the trash screen of the Administrator.
- The post remains published with its remaining comments.

3.2.12 Function: Like a Comment

Function Trigger

A signed-in member selects the heart action on a comment of the thread. A guest is routed to the sign-in page, because the reaction requires an account.
Navigation path: Home → Forum → *Category* → *Post* → Comments → Like.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client. An Administrator can also react while browsing.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM.

Purpose

The function allows members to acknowledge a useful reply without writing another comment. It is a toggle, exactly like the post reaction: the first activation likes the comment and the second one removes the reaction, so one member can never like the same comment twice.

Interface

- Heart action in the footer of each comment, followed by the like counter of the comment.
- Outlined heart when the member has not liked the comment, filled heart when the member liked it.
- The action is disabled while the request runs.

Data Processing Overview

- The client sends `POST /api/forums/posts/{postId}/comments/{commentId}/like`.
- The API loads the comment with `deletedAt` null and rejects the request when it is missing.
- The API verifies that the comment belongs to the post of the route.
- The API looks for a reaction of the member on the comment.
- When the reaction exists, the API removes it and answers with `liked: false`.
- When the reaction does not exist, the API inserts it and answers with `liked: true`.

- The client invalidates the comment queries of the post, which refreshes the counter and the state of the heart of that comment only.
- The comment list computes the liked state per comment by matching the reactions of the signed-in member, so the states of the other comments are unaffected.

Screen Layout (Mock-up Description)



Figure 61: Comment before and after being liked

- Before the reaction the heart is outlined and the counter shows the current number of likes of the comment.
- After the reaction the heart is filled with the accent colour and the counter is incremented by one.
- Only the selected comment changes; the reactions of the other comments and the likes of the post remain untouched.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the post from the route; it scopes the comment.
`commentId`	Identifier of the comment to react to.
Access token	Identifies the member who reacts.
Reaction type	Constant value `like`.

Validation Rules

- The requester must be authenticated.
- The comment must exist, must not be soft deleted and must belong to the post of the route.
- A member can hold only one reaction per comment, enforced by the unique constraint of the like table on the pair member and comment.
- The reaction is attached to the member of the token, never to an identifier sent by the client.

Business Rules

- **BR-FORUM-41**: Only authenticated users can like comments.

- **BR-FORUM-42**: A user can have only one active like reaction per comment.
- **BR-FORUM-43**: The reaction shall be stored in the `ForumLike` table.
- **BR-FORUM-44 to BR-FORUM-46**: removing a reaction is allowed only when the record exists, and it deletes that record.

Normal Flow

1. The signed-in member finds a useful comment in the thread.
2. The client renders an outlined heart with the like counter of that comment.
3. The member selects the heart and the client sends the toggle request.
4. The API confirms that the comment exists and belongs to the post.
5. The API finds no reaction of this member on this comment.
6. The API inserts the reaction row.
7. The API answers with `liked: true`.
8. The client refreshes the comment list, fills the heart and increments the counter.
9. Selecting the heart again removes the reaction and restores the previous counter.

Abnormal Flow / Exception Handling

Case	Description
A1	The comment does not exist or was deleted. The API returns `Error.ForumCommentNotFound` and the counter is unchanged.
A2	The comment belongs to another post. The request is refused with the same not-found error.
A3	The member is not signed in. The client routes to the sign-in page and no request is sent.
A4	The reaction already exists. The request removes it, which is the intended toggle behaviour.
A5	The insert fails at database level. The API returns `Error.FailedToLikeForumComment` and the interface keeps the previous state.
A6	The member clicks the heart twice quickly. The action is disabled while the request runs, so only one reaction is created.

Output Data

Data Name	Description
`liked`	`true` when the reaction was added, `false` when it was removed.
Like row	Member, post, comment, reaction type and creation date of the stored reaction.
Updated counter	The like counter of the comment, recomputed by the client.

Post-Conditions

- The comment holds exactly one reaction of the member, or none after the second activation.
- The counter of the comment matches the number of stored reactions on that comment.
- The reactions of the post and of the other comments are unchanged.

3.2.13 Function: Report Content

Function Trigger

A signed-in member who is not the author selects the "Report" action under a post or next to a comment, writes a reason and submits the report. The action is hidden from the author of the content, because a member cannot report their own post or comment.
Navigation path: Home → Forum → *Category* → *Post* → Report.

Function Description

Actors / Roles

- **Primary Actor:** Freelancer and Client.
- **Supporting Systems:** Frevia Web application, Frevia BFF proxy, Frevia REST API and PostgreSQL through Prisma ORM; the report is later handled by an Administrator in the moderation screen.

Purpose

The function gives the community a way to flag content that violates the rules, either a discussion or a single reply. A report never changes the visibility of the content by itself: it creates a pending moderation entry that the Administrator reviews through the moderation module.

Interface

- "Report" action with a flag icon under the post body, and the same action in the header of each comment.

- Dialog titled “Report Content” with the prompt “Please provide a reason for reporting this content.”.
- “Reason” text area with the placeholder “Why are you reporting this content?”.
- “Cancel” action and destructive “Report” action, which stays disabled while the reason is empty.
- “Reported” badge with a check icon replacing the action once the content has been reported by the member.

Data Processing Overview

- When the dialog is opened, the client checks whether the member already reported the content through `GET /api/forums/posts/{postId}/is-reported` or `GET /api/forums/posts/{postId}/comments/{commentId}/is-reported`.
- The client sends `POST /api/forums/posts/{postId}/reports` for a post, or `POST /api/forums/posts/{postId}/comments/{commentId}/reports` for a comment, with the reason.
- The API verifies that the reported target exists and is not soft deleted.
- For a comment, the API also verifies that the comment belongs to the post of the route.
- The API looks for an existing report of the same member on the same target and refuses a duplicate.
- The API inserts the report with the reporter of the token, the target and the reason; the status defaults to `PENDING` and no administrator is assigned yet.
- The client shows the confirmation toast, marks the report as done in its cache and closes the dialog after a short delay.
- From then on the action is replaced by the “Reported” badge, and reopening the dialog explains that the content is already under review.

Screen Layout (Mock-up Description)

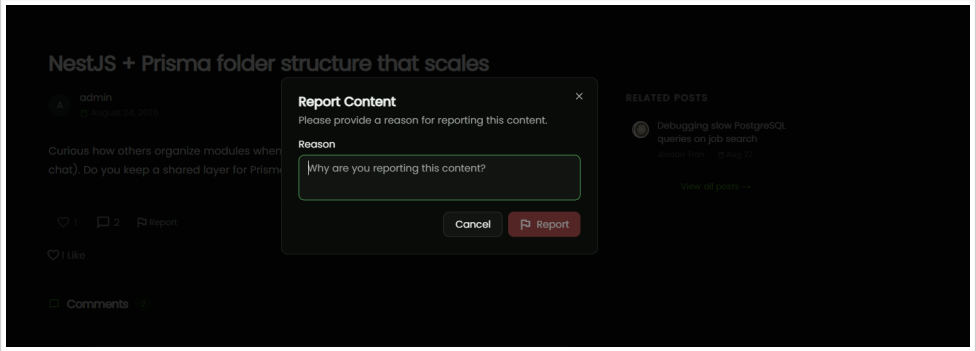


Figure 62: Report dialog with the reason field

- The action is available on the post and on every comment, so a member can flag the exact

item that is problematic.

- The dialog explains what is expected and gives a free-text reason, which the moderator reads before making a decision.
- The confirm action is destructive styled and disabled without a reason, which prevents empty reports.
- After a successful submission the toast “Report submitted successfully” confirms the action and the badge “Reported” prevents a second submission.
- When the content was already reported, the dialog displays a confirmation panel with “You have already reported this content. Our team will review it.”.

Function Details

Input Data

Data Name	Description
`postId`	Identifier of the reported post, or of the post that holds the reported comment.
`commentId`	Optional identifier of the reported comment; absent when the post itself is reported.
`reason`	Free-text justification of the report, required.
Access token	Identifies the reporter; it becomes `reporterId` in the stored report.

Validation Rules

- The requester must be authenticated.
- The reason is required and must contain at least one character.
- The reported post must exist and must not be soft deleted, otherwise the request fails with `Error.ForumPostNotFound`.
- The reported comment must exist, must not be soft deleted and must belong to the post of the route, otherwise the request fails with `Error.ForumCommentNotFound`.
- Exactly one member may report a given target once; a second attempt fails with `Error.ForumReportAlreadyExists`.
- The status of a new report is `PENDING` and the administrator field stays empty until a moderator handles it.

Business Rules

- **BR-FORUM-47**: Only authenticated users can submit reports.

- **BR-FORUM-48**: Reports must be associated with either a post or a comment.
- **BR-FORUM-49**: New reports shall be created with status PENDING.
- **BR-FORUM-50**: A user may submit only one report for the same content item.

Normal Flow

1. The signed-in member reads a post or a comment that violates the community rules.
2. The member selects the “Report” action of that item.
3. The client opens the dialog and checks whether the item was already reported.
4. The member writes a reason and confirms.
5. The client sends the report request with the access token.
6. The API validates the reason and loads the reported target.
7. The API confirms that the target exists and, for a comment, that it belongs to the post.
8. The API confirms that the member has not reported the same target yet.
9. The API inserts the report with status `PENDING`.
10. The API returns the created report.
11. The client shows the success toast, marks the cache as reported and closes the dialog.
12. The “Reported” badge replaces the action for this member.

Abnormal Flow / Exception Handling

Case	Description
A1	The reason is empty. The confirm action is disabled, and a forced request is rejected by the validation pipe.
A2	The post was deleted before the submission. The API returns `Error.ForumPostNotFound` and no report is stored.
A3	The comment was deleted before the submission, or does not belong to the post. The API returns `Error.ForumCommentNotFound`.
A4	The member already reported the same item. The API returns `Error.ForumReportAlreadyExists` and the client displays the message and switches to the reported state.
A5	An Administrator opens the moderation screen. Listing and updating reports require the Administrator role, otherwise the API returns `Error.ForumReportForbidden`.
A6	The insert fails at database level. The API returns `Error.FailedToCreateForumReport` and the dialog stays open with the reason.

Case	Description
A7	The member is not signed in. The action is not rendered for guests, and a direct request is rejected with an authentication error.
A8	The member reports their own content through a crafted request. The record can be created, but the action is never offered in the interface, and the moderation screen treats it like any other report.

Output Data

Data Name	Description
`id`	Identifier of the created report.
`reporterId`	Member who submitted the report.
`postId`, `commentId`	Reported targets; exactly one of them identifies the item, the other may be null for a comment report.
`reason`	Reason written by the reporter.
`status`	`PENDING` for a new report; later `REVIEWED`, `RESOLVED` or `DISMISSED` after the moderator decision.
`adminId`, `createdAt`	Administrator who handled the report, null at creation, and the creation timestamp.

Post-Conditions

- One pending report exists for the reported item and the reporter.
- The reported content is not modified and stays visible until a moderator decides.
- The report appears in the moderation queue of the Administrator, where it can be reviewed, resolved or dismissed.
- The reporter sees the “Reported” state and cannot submit the same report again.

3.3 Manage Conversation

3.3.1 Start Conversation

Function Trigger

A logged-in user starts a new chat, either by clicking the “Connect with client” button on the job detail page in Find Work, or by clicking the “New message” button in the conversation list.

Function Description

This function allows a logged-in user to start a direct conversation with another user. When the chat is started from a job, the other participant is the client who posted that job. When the chat is started from the conversation list, the user chooses the other participant. The conversation is created and the user can write the first message. If a conversation with that user already exists, the existing conversation is reopened instead of creating a duplicate.

Screen Layout (Mock-up Description)

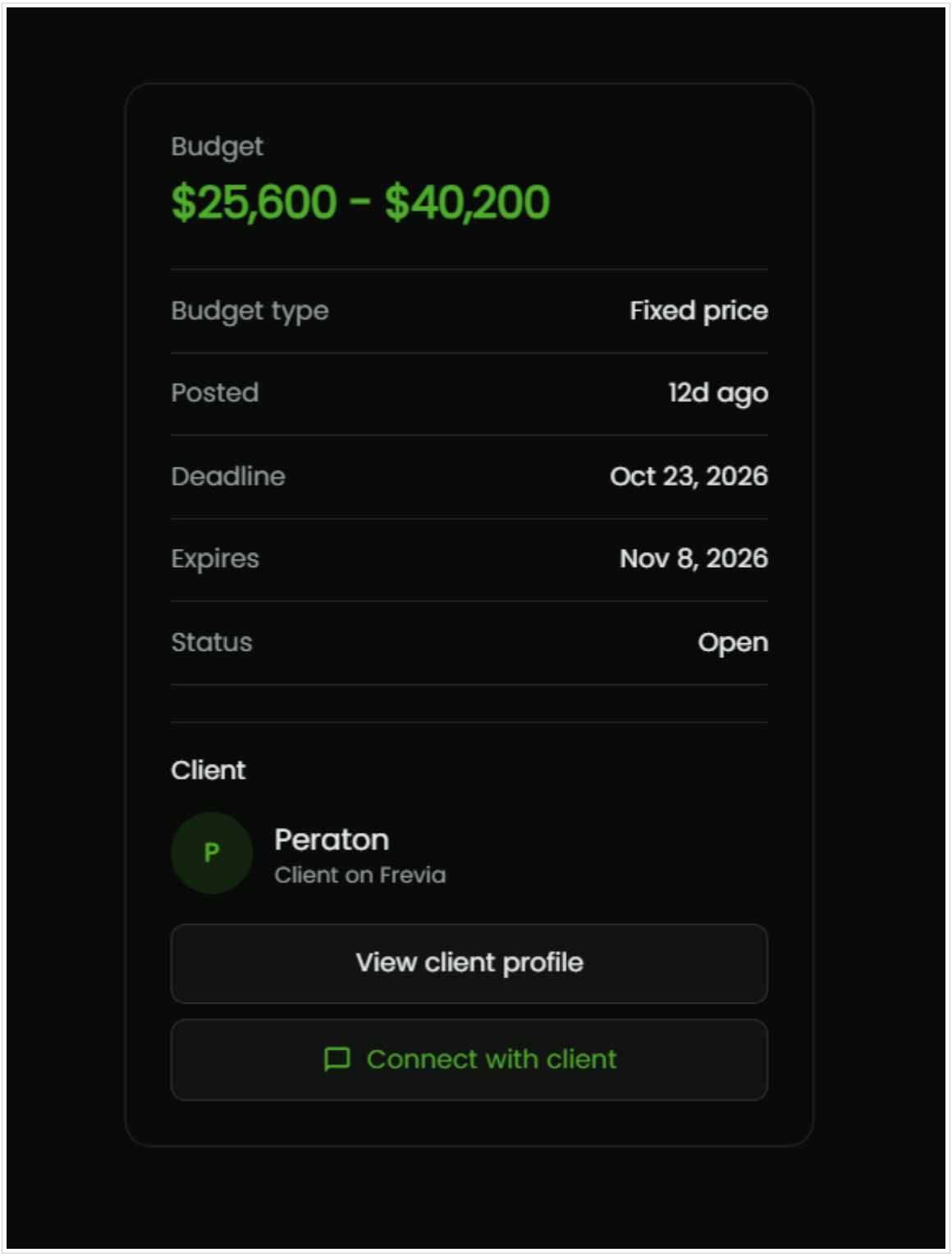


Figure 63: Start conversation screen

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user selected another user who is different from themselves.
- **Process**
  - **Start a New Conversation**: The user starts the chat in one of two ways.
    - From a job in Find Work, the user clicks the "Connect with client" button on the job detail page.
    - From the conversation list, the user clicks the "New message" button.
  - **Choose the Participant**: The user provides or confirms the other participant.
    - When the chat is started from a job, the client of that job is already selected, so the user does not have to choose anyone.
    - When the chat is started from the conversation list, the user enters the identifier of the other participant in the dialog and confirms.
  - **Create or Reopen the Conversation**: The system checks whether a conversation between the two users already exists.
    - When no conversation exists, a new conversation between the two users is created.
    - When a conversation already exists, that conversation is reopened, including a conversation that the user had previously deleted from their own list.
  - **Write the First Message**: The compose screen shows the name and avatar of the other participant, and the user can type and send the first message (see **Send Message**).
- **Alternative Flow**
  - **Conversation Already Exists**: When a conversation with the chosen user already exists, the system opens the existing conversation instead of creating a second one, so there is only one thread per pair of users.
  - **Not Logged In**: When a guest clicks the "Connect with client" button, the system asks the user to log in before the conversation can be started.
  - **Cancel**: When the user leaves the compose screen or closes the dialog without confirming, no conversation is created and nothing changes.
- **Postconditions**
  - A conversation between the two users exists and is open on the screen.
  - The conversation is visible in the conversation list of both participants.

- **Error Handling**
  - **Cannot Chat with Yourself**: When the user chooses their own account as the participant, the conversation is not created and a message tells the user that a chat with themselves is not allowed.
  - **Participant Not Found**: When the chosen user cannot be found, the conversation is not created and a message tells the user that the participant cannot be found.
  - **Cannot Start Chat**: When the conversation cannot be created, the screen shows a start-chat failure message so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only logged-in users can start a conversation, and the user is always recorded as one of the two participants.
  - When the chat is started from a job, the other participant is always the client of that job, so the button cannot be used to start a chat with an unrelated account.
  - Only one conversation can exist for a pair of users, so a user cannot create duplicated threads with the same person.

3.3.2 View Conversation List

Function Trigger

The logged-in user opens the Messages area by clicking the chat icon in the main navigation.

Function Description

This function allows a logged-in user (Client, Freelancer) to see all of their conversations in one list. Each row shows the other participant, a preview of the last message, the time of that message and the number of unread messages. Conversations that the user has pinned are always shown at the top of the list.

Screen Layout (Mock-up Description)

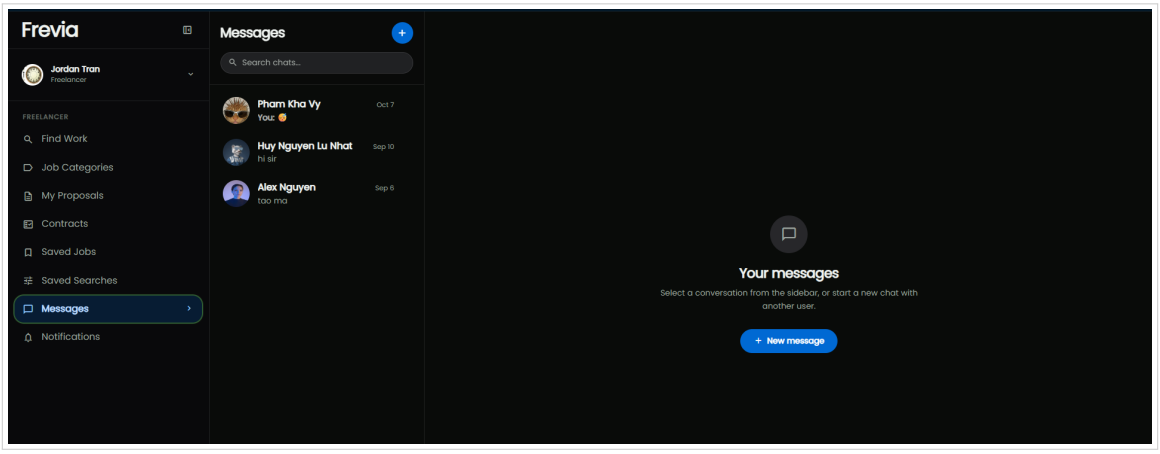


Figure 64: Conversation list screen

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user has at least one conversation, or the empty state is displayed instead.
- **Process**
  - **Open the Messages Area**: The user clicks the chat icon in the main navigation and the conversation list is displayed.
  - **Read Each Conversation Row**: Every row in the list presents the following information.
    - The avatar and the display name of the other participant.
    - A short preview of the last message, with a paperclip icon when the last message contains a file.
    - The relative time of the last message.
    - A blue unread dot on the avatar and an unread count badge when there are unread messages.
    - A pin icon when the conversation is pinned.
  - **Search for a Conversation**: The user types a keyword in the search box and the list shows only the conversations whose participant name or last message contains that keyword.
  - **Sorting**: Pinned conversations are shown first, and the remaining conversations are ordered by the time of the last message, newest first.

- **Open a Conversation**: The user clicks a conversation row to open the chat thread (see **View Conversation Detail**).

- **Alternative Flow**
  - **No Conversation**: When the user has no conversation at all, the screen shows an empty state with a "New message" button, and the user can start a new chat from there (see **Create Conversation**).
  - **No Search Result**: When the search keyword does not match any conversation, the list is replaced by a message telling the user that no chat matches the keyword.
- **Postconditions**
  - The user sees all of their conversations, with the pinned ones at the top and the unread counts for each conversation.
  - Opening the list does not change any message; unread messages are only cleared when the user opens a thread (see **Mark Conversation as Read**).
- **Error Handling**
  - **Cannot Load Conversations**: When the conversation list cannot be loaded, the screen shows a load-failure message so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only the conversations of the logged-in user are shown, so a user can never see the chats of another account.
  - Conversations that the user has deleted from their own list are not shown again until a new message arrives.

3.3.3 View Conversation Detail

Function Trigger

The user clicks a conversation row in the conversation list.

Function Description

This function allows the user to read the whole chat thread with the other participant. The thread shows all messages in chronological order, the avatar and display name of the other user, the attachments that were exchanged and the read state of the messages that the user has sent.

Screen Layout (Mock-up Description)

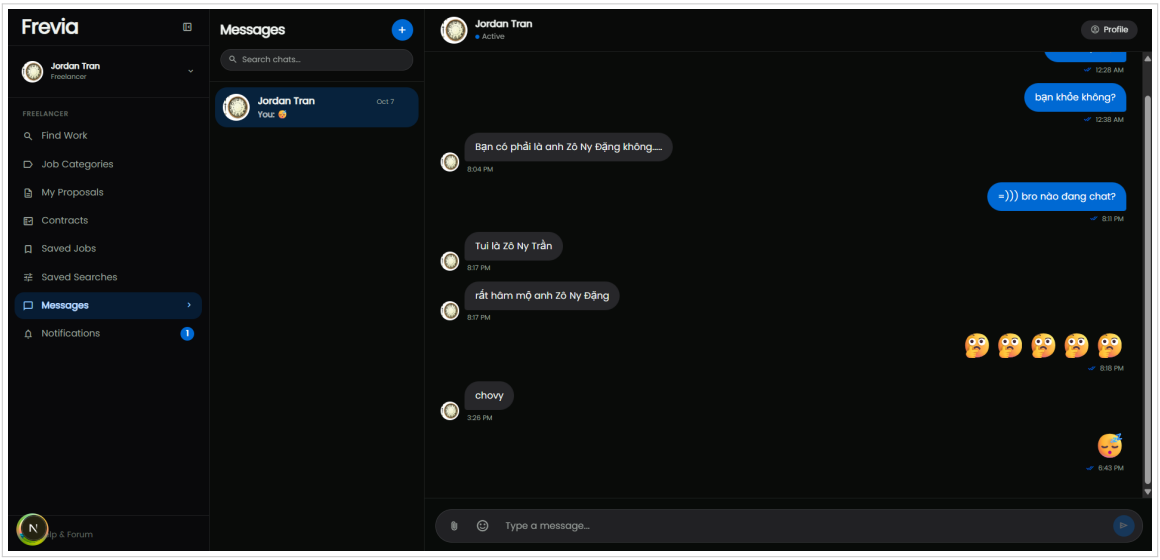


Figure 65: Conversation detail screen (chat thread)

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation that is being opened.
- **Process**
  - **Open the Conversation**: The user clicks a conversation row and the chat thread is displayed.
  - **Read the Thread Header**: The header of the thread shows the following information.
    - The avatar and the display name of the other participant.
    - A button that opens the profile of the other participant.
    - A back button that returns to the conversation list.
  - **Read the Messages**: The messages are shown in chronological order, oldest at the top and newest at the bottom.
    - Messages sent by the user are aligned on one side, and messages received from the other user on the other side.
    - Image attachments are shown as pictures that can be opened in a new tab.
    - Other files are shown as a download card with the file name and the file size.
    - A message that contains only emojis is displayed with a larger font.
  - **Check the Read State**: Messages that the user has sent show a read indicator.

        - A single check mark means that the message has been delivered but not read.
        - A double check mark means that the other user has read the message.
    - **Use the Thread Actions**: The user can send a message (see **Send Message**), attach a file (see **Send File**) or delete one of their own messages (see **Delete Message**).
- **Alternative Flow**
    - **Empty Thread**: When the conversation has no message yet, the thread shows an empty state with a hint to write the first message.
    - **User Is Typing**: When the other user is typing, an animated typing indicator is shown at the bottom of the thread.
- **Postconditions**
    - The user sees the whole chat thread with the other participant, including the attachments and the read state of their own messages.
    - The messages of the other user are marked as read while the thread is open (see **Mark Conversation as Read**).
- **Error Handling**
    - **Conversation Not Found**: When the conversation no longer exists, the screen shows a not-found state and offers the user a way back to the conversation list.
    - **Access Denied**: When the user is not a participant of the conversation, the thread is not opened and a permission message is shown.
    - **Cannot Load Messages**: When the messages cannot be loaded, the thread shows a load-failure message so that the user can try again.
    - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
    - Only the two participants of a conversation can read its messages; another user is refused even when the conversation link is known.
    - Messages that have been deleted are never shown again in the thread.

3.3.4 Send Message

Function Trigger

The user types a message in the input box at the bottom of the chat thread and clicks the send button.

Function Description

This function allows a logged-in user to send a text message to the other participant of the conversation. The user types the text and sends it, and the message appears in the thread of both users immediately.

Screen Layout (Mock-up Description)



Figure 66: Sending a message in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.
- **Process**
  - **Type the Message**: The user types the text in the input box at the bottom of the thread. The send button stays disabled while the input box is empty.
  - **Add an Emoji if Wanted**: The user can click the emoji button to pick an emoji, and the chosen emoji is added to the input box.
  - **Send the Message**: The user clicks the send button or presses the Enter key.
  - **View the Result**: When the message is sent, the screen shows the result.
    - The message appears at the bottom of the thread on the user's side.
    - The input box is cleared.
    - The message shows a single check mark, meaning that it has been sent but not read yet.
  - **Message Arrives for the Other User**: The other participant receives the message in real time, and the conversation list of that user shows the new message preview and the unread count.
- **Alternative Flow**
  - **Enter Key**: When the user presses the Enter key without holding Shift, the message is sent in the same way as clicking the send button.
  - **Empty Message**: When the input box is empty, the send button remains disabled and the user cannot send an empty message.

- **Emoji Only**: When the message contains only emojis, the message is displayed with a larger font in the thread.

- **Postconditions**
  - The new message is stored in the conversation and is shown to both participants.
  - The conversation list of both users is updated with the new last message.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the message is not sent and the thread shows a message telling the user that the conversation cannot be found.
  - **Access Denied**: When the user is not a participant of the conversation, the message is refused and a permission message is shown.
  - **Cannot Send Message**: When the message cannot be sent, the thread shows a send-failure message and the text is kept so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only a participant of the conversation can send a message into it, and the sender is always the account that is currently logged in.
  - The length of a message is limited, so very large text cannot be sent.

3.3.5 Send File

Function Trigger

The user clicks the attach-file button in the input box of the chat thread and chooses a file from their device.

Function Description

This function allows a logged-in user to send a file, such as a picture or a document, to the other participant of the conversation. The user chooses a file, the system uploads it and the file is sent as a message with a preview or a download card.

Screen Layout (Mock-up Description)

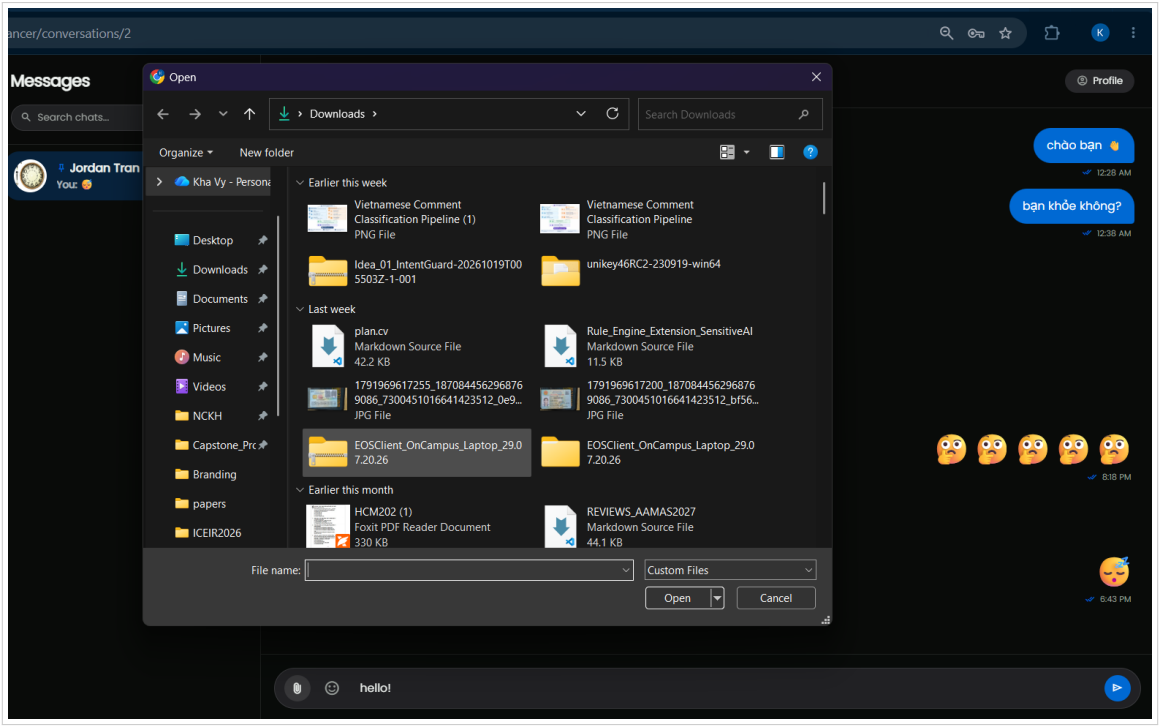


Figure 67: Sending a file in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The user is a participant of the conversation and the chat thread is open.
- **Process**
  - **Click the Attach Button**: The user clicks the paperclip button in the input box, and the file chooser of the device is opened.
  - **Choose the File**: The user selects a file. The file chooser accepts images, documents, archives and other supported file types.
  - **Upload the File**: The system uploads the file before sending it, and a loading state is shown while the upload is running.
  - **Send the File as a Message**: When the upload finishes, the file is sent to the conversation as a message.
    - When the file is an image, it is displayed as a picture that can be opened in a new tab.
    - When the file is not an image, it is displayed as a download card with the file name and the file size.

- **View the Result**: The file message appears at the bottom of the thread, and the conversation list of both users shows the file preview with a paperclip icon.

- **Alternative Flow**
  - **Cancel the File Chooser**: When the user closes the file chooser without choosing a file, no upload is started and nothing changes.
  - **Image Preview**: When the chosen file is an image, the picture is rendered directly inside the message bubble instead of a download card.
- **Postconditions**
  - The file is uploaded and attached to a new message in the conversation.
  - Both participants can see the file message in the thread and can open or download the file.
- **Error Handling**
  - **File Too Large**: When the chosen file is larger than the allowed size, the upload is stopped and a message tells the user that the file is too large.
  - **File Type Not Allowed**: When the chosen file type is not supported, the upload is stopped and a message tells the user that this file type is not allowed.
  - **Cannot Upload File**: When the upload fails, the file message is not sent and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - Only supported file types can be uploaded, and the size of the file is limited before the upload starts.
  - Only a participant of the conversation can send a file into it, and the sender is always the account that is currently logged in.

3.3.6 Mark Conversation as Read

Function Trigger

The user opens a conversation from the conversation list, or a new message arrives while the thread is already open.

Function Description

This function allows a logged-in user to clear the unread messages of a conversation. The unread badge disappears from the conversation list and the double check mark appears on the messages of the other user in the thread, so the other participant can see that the messages have been read.

Screen Layout (Mock-up Description)

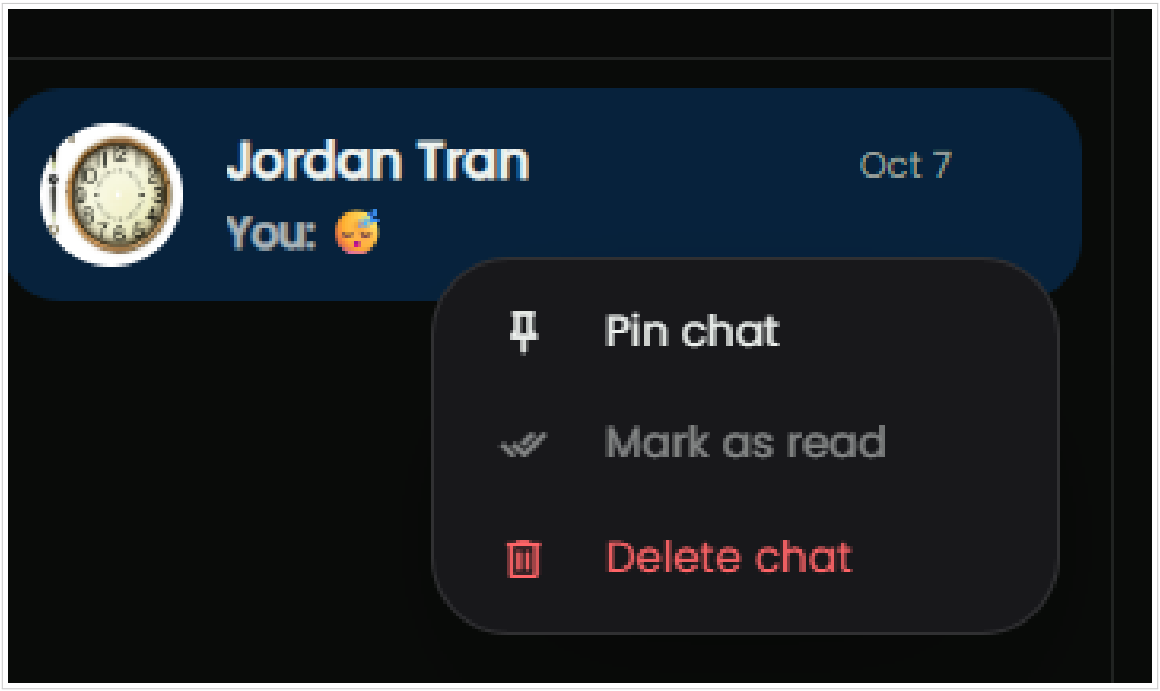


Figure 68: Conversation after being marked as read

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation contains at least one unread message from the other participant.
- **Process**
  - **Open the Conversation**: The user opens the conversation, and the system automatically marks the received messages as read.
  - **View the Result in the List**: The conversation row in the conversation list is updated.
    - The blue unread dot disappears from the avatar.
    - The unread count badge is removed from the row.

- **Mark Manually if Wanted**: The user can also choose the “Mark as read” action from the “...” menu of a conversation row.
- **Notify the Sender**: The other participant sees the read state change in real time.
  - Messages that were shown with a single check mark are updated to a double check mark.

- **Alternative Flow**
  - **Nothing to Read**: When the conversation has no unread message, the “Mark as read” action is disabled in the menu.
  - **New Message While Reading**: When a new message arrives while the thread is open, that message is marked as read immediately.
- **Postconditions**
  - All messages received from the other participant in this conversation are marked as read.
  - The unread count of the conversation is cleared for the current user, and the other participant sees the double check mark on their sent messages.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the read state is not updated and a message tells the user that the conversation cannot be found.
  - **Cannot Mark as Read**: When the read state cannot be updated, the unread badge stays on the row and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a “Lost Connection” state and asks the user to reconnect.
- **Security Measures**
  - Only the messages received by the current user are marked as read, so a user cannot change the read state of their own sent messages through this action.
  - Only a participant of the conversation can mark it as read.

3.3.7 Pin Conversation

Function Trigger

The user opens the “...” menu of a conversation row and chooses “Pin chat”.

Function Description

This function allows a logged-in user to keep an important conversation at the top of the conversation list. The user pins a conversation and the row is moved to the pinned group, which is always shown above the other conversations.

Screen Layout (Mock-up Description)

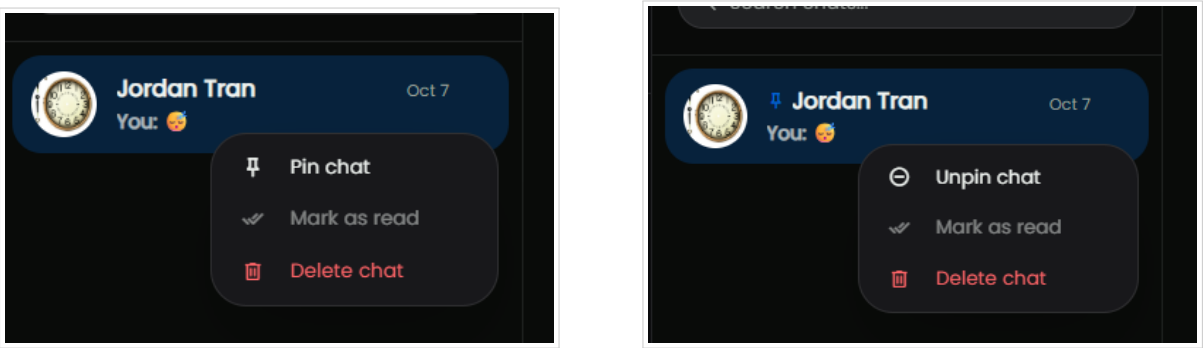


Figure 69: Conversation list before pinning (left) and after pinning (right)

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The conversation that the user wants to pin is shown in the conversation list.
- **Process**
  - **Open the Row Menu**: The user clicks the "..." button of the conversation row and the menu is displayed.
  - **Choose Pin Chat**: The user clicks the "Pin chat" action in the menu.
  - **View the Result**: The conversation list is updated.
    - The conversation is moved to the top of the list, above the not-pinned conversations.
    - A pin icon is shown on the conversation row.
  - **Unpin if Wanted**: The user can open the same menu again and choose "Unpin chat" to remove the conversation from the pinned group.
- **Alternative Flow**
  - **Unpin Conversation**: When the user chooses "Unpin chat", the pin icon is removed and the conversation returns to the normal order of the list.
  - **Pin State Is Personal**: The pin of one user does not change the order of the conversation list of the other participant.

- **Postconditions**
  - The pinned conversation stays at the top of the conversation list of the current user.
  - The content of the conversation and the messages are not changed by this action.
- **Error Handling**
  - **Conversation Not Found**: When the conversation no longer exists, the pin is not applied and a message tells the user that the conversation cannot be found.
  - **Cannot Pin Conversation**: When the pin state cannot be saved, the row returns to its previous position and a failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - The pin belongs to the current user only, so pinning a chat never affects the conversation list of the other participant.
  - Only a participant of the conversation can pin it.

3.3.8 Delete Message

Function Trigger

The user moves the pointer over one of their own messages in the chat thread and clicks the delete button.

Function Description

This function allows a logged-in user to remove a message that they have sent. The user deletes their own message and the message disappears from the chat thread of both participants. The user can only delete messages that they sent themselves.

Screen Layout (Mock-up Description)

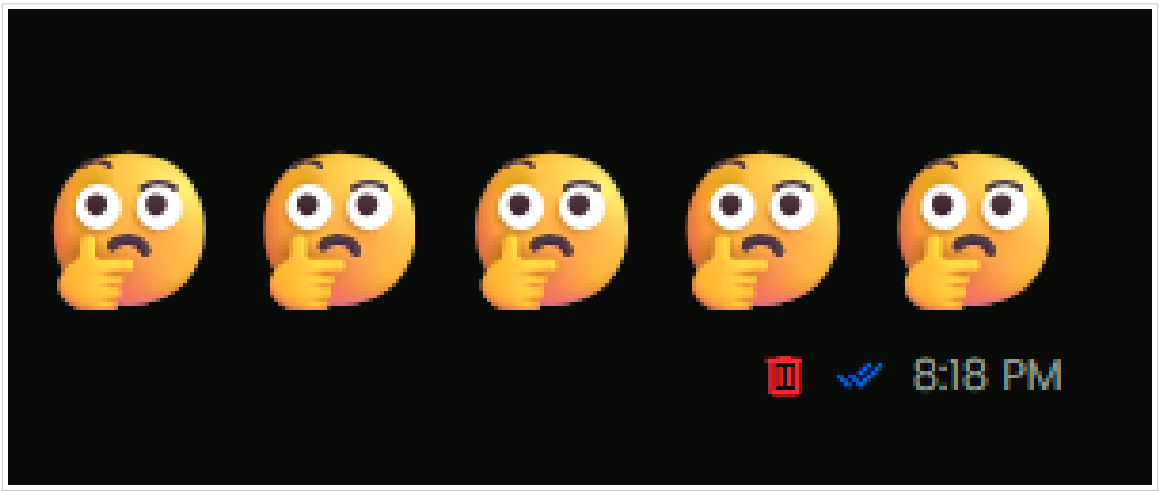


Figure 70: Deleting a message in the chat thread

Function Details

- **Precondition**
  - The user is logged in.
  - The user is connected to the internet.
  - The message belongs to the current user and is shown in the chat thread.
- **Process**
  - **Point at the Message**: The user moves the pointer over one of their own messages and the delete button appears.
  - **Click the Delete Button**: The user clicks the delete button of that message.
  - **View the Result**: The message is removed from the thread.
    - The message disappears from the thread of the current user.
    - The message also disappears from the thread of the other participant in real time.
  - **Conversation List Update**: When the deleted message was the last message of the conversation, the preview in the conversation list is refreshed to show the previous message.
- **Alternative Flow**
  - **Delete a File Message**: When the deleted message contains a file, the whole file message is removed from the thread in the same way.
  - **Message Already Deleted**: When the message has already been deleted, the delete button is not available for it.
- **Postconditions**

- The message no longer appears in the chat thread of either participant.
- The message is kept in the system for record-keeping but is never shown to users again.

- **Error Handling**
  - **Message Not Found**: When the message no longer exists or has already been deleted, the page shows a message telling the user that the message cannot be found.
  - **Not the Sender**: When the user tries to delete a message that they did not send, the action is refused and a permission message is shown.
  - **Cannot Delete Message**: When the message cannot be deleted, the message stays in the thread and a delete-failure message is shown so that the user can try again.
  - **Lost Connection**: When the connection to the server is lost, the screen shows a "Lost Connection" state and asks the user to reconnect.
- **Security Measures**
  - The delete button is shown only on the messages that the current user sent.
  - The system also checks the sender when the deletion is confirmed, so a user can never delete a message that belongs to the other participant.

3.4 Moderation

3.4.1 View Moderation Queue

Function Trigger

The logged-in administrator opens the Moderation screen by choosing **Forum** $\rightarrow$ **Moderation** in the admin sidebar.

Function Description

This function allows an administrator to see all forum posts that are waiting to be reviewed. Every row shows the identifier of the post, its title, the automatic moderation score, the categories that were detected, the author and the creation date. The list only contains posts that are still pending, so the administrator always works on the items that need a decision.

Screen Layout (Mock-up Description)

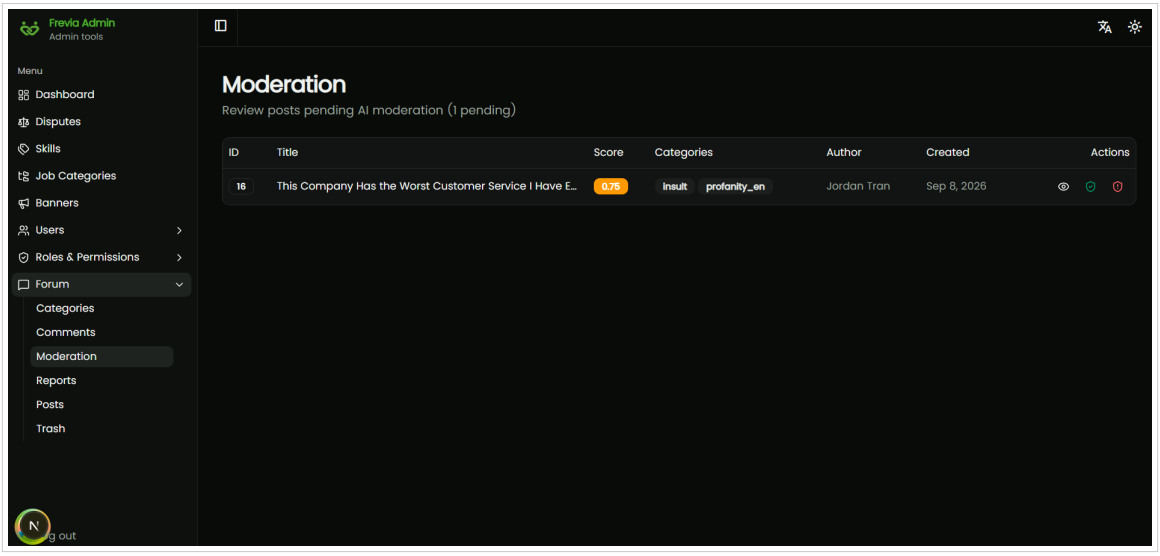


Figure 71: Moderation queue screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the admin area and is allowed to open the Moderation screen.
- **Process**
  - **Open the Moderation Screen**: The user selects **Forum** in the sidebar and then **Moderation**, and the screen titled "Moderation" is displayed.
  - **Read the Summary Line**: The subtitle shows the total number of posts that are waiting, in the form "Review posts pending AI moderation (n pending)".
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the post, shown as a small badge.
    - **Title**: The title of the post, shortened when it is too long.
    - **Score**: The automatic moderation score of the post, shown with two decimals, or "N/A" when no score was computed.
    - **Categories**: The categories detected by the automatic check, shown as badges, or "—" when none were detected.
    - **Author**: The display name of the user who wrote the post, or "User #id" when the user has no display name.
    - **Created**: The date on which the post was created.
  - **Read the Score Colour**: The score badge uses a colour that reflects the risk level.

        - A score of 0.80 or more is shown in red, a score from 0.30 to 0.79 is shown in amber and a score below 0.30 is shown in green.
    - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list, and the counter shows “Page x of y (n total)”.
    - **Choose an Item**: The user opens, approves or rejects a post with the action buttons at the end of the row (see **View Moderation Item Detail** and **Resolve Moderation Item**).
- **Alternative Flow**
    - **No Pending Post**: When no post is waiting, the table shows the message “No posts awaiting moderation.” instead of rows.
    - **Only One Page**: When all posts fit on a single page, the page numbers are hidden and only the table is shown.
- **Postconditions**
    - The administrator sees the complete list of posts that are waiting for a decision.
    - No data is changed by this function, so the queue stays exactly as it was.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the moderation queue.
    - **Lost Connection**: When the queue cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
    - The whole admin area is protected: a guest or a banned account is sent to the login page, and a logged-in user without the administrator role is sent to the access-denied page.
    - The moderation queue is only returned to administrator accounts, so ordinary users can never see posts that are waiting for review, no matter how they call the screen.

3.4.2 View Moderation Item Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the moderation queue.

Function Description

This function allows an administrator to read a pending post in full before making a decision. The dialog shows the title, the author, the creation date, the moderation score and the complete content of the post, together with the **Approve** and **Reject** buttons so that the decision can be taken from the same screen.

Screen Layout (Mock-up Description)

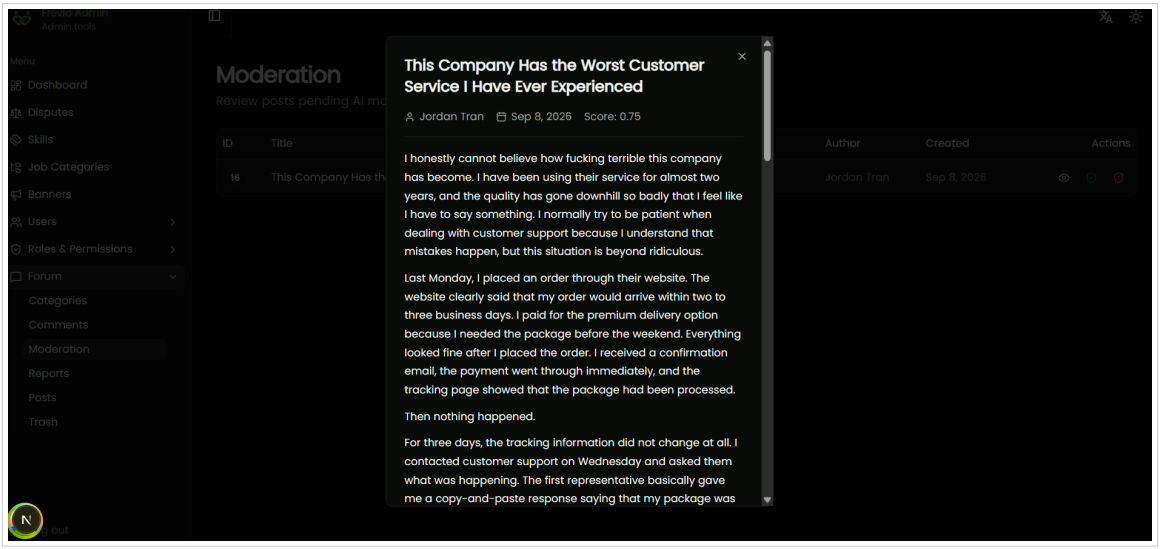


Figure 72: Moderation item detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Moderation screen and there is at least one pending post.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post as its heading.
  - **Read the Item Summary**: Under the title, the dialog shows the author of the post, the creation date and the moderation score (with “N/A” when no score exists).
  - **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it would appear to readers.
  - **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read without leaving the screen.
  - **Decide**: The user gives the decision with the **Approve** or **Reject** button at the bottom

of the dialog (see **Resolve Moderation Item**).

- **Close the Detail**: The user closes the dialog with the close button or by clicking outside it, and returns to the queue.

- **Alternative Flow**
  - **No Score**: When the automatic check produced no score, the badge shows "N/A" and the rest of the dialog is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the post stays in the queue.
- **Postconditions**
  - The administrator has read the complete post and knows its author, date, score and content.
  - Nothing has changed while only viewing the item.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may review pending posts.
  - **Item Already Handled**: When the post has already been approved or rejected by another administrator, the decision fails and a message tells the user that the post cannot be found or updated any more.
  - **Lost Connection**: When the connection to the server is lost, the decision cannot be saved and the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so a pending post can never be read by an ordinary user.
  - Every decision is recorded together with the administrator who made it and the moment it was made, so each review can be traced.

3.4.3 Resolve Moderation Item

Function Trigger

The logged-in administrator clicks the approve button or the reject button on a row of the moderation queue, or presses **Approve** or **Reject** inside the detail dialog.

Function Description

This function allows an administrator to close a pending review by approving or rejecting a post. An approved post becomes visible to everyone in the forum. A rejected post stays hidden from the public and is moved to the trash, where it can be restored later. Before a rejection is applied, the administrator is asked to confirm it.

Screen Layout (Mock-up Description)

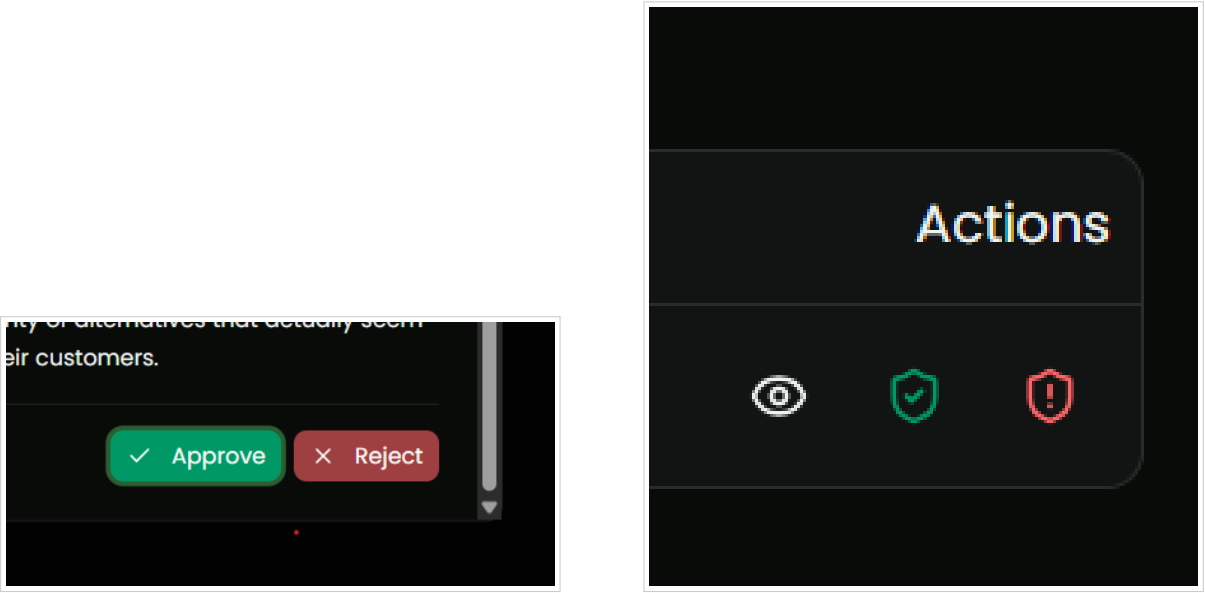


Figure 73: Approve action (left) and reject confirmation (right)

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post that is being resolved is still waiting in the moderation queue.
- **Process**
  - **Approve a Post**: The user clicks the approve button on a row, and the post is accepted straight away.
    - The post leaves the moderation queue and becomes visible to everyone in the forum.
    - The screen shows the message “Post approved and now visible publicly”.
  - **Start a Rejection**: The user clicks the reject button on a row, or presses **Reject** inside the detail dialog.
  - **Confirm the Rejection**: When the reject button of a row is used, a confirmation dialog titled “Reject Post” appears with the question “Reject “title”? The post will not be visible publicly and will be moved to trash.”.
    - The user presses **Reject** to confirm, or **Cancel** to keep the post in the queue.
  - **Apply the Rejection**: When the rejection is confirmed, the post is refused and is moved to the trash.
    - The post is never shown to ordinary users while it stays rejected.

        - The screen shows the message “Post rejected and moved to trash”.
    - **Update the Queue**: The resolved post disappears from the list and the remaining items move up, so the queue always shows only the posts that still need a decision.
- **Alternative Flow**
    - **Cancel the Rejection**: When the user closes the confirmation dialog, the post stays pending and nothing changes.
    - **Reject from the Detail Dialog**: When **Reject** is pressed inside the detail dialog, the rejection is applied immediately without an extra confirmation dialog.
    - **Empty Queue**: When the last pending post is resolved, the table shows the message “No posts awaiting moderation.”.
- **Postconditions**
    - The post is no longer pending and has been marked as approved or rejected.
    - An approved post is visible in the forum, while a rejected post is hidden and can be found in the trash.
    - The decision is recorded with the administrator who made it and the moment it was made.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator tries to approve or reject a post, the action is refused because only administrators may resolve moderation items.
    - **Item Not Found**: When the post has already been resolved by another administrator, the action fails and the screen shows the message “Couldn’t approve post. Try again.” or “Couldn’t reject post. Try again.”.
    - **Cannot Save the Decision**: When the decision cannot be saved, the post stays in the queue and a message asks the administrator to try again.
    - **Lost Connection**: When the connection to the server is lost, the decision is not applied and the screen asks the user to try again.
- **Security Measures**
    - Only administrators may approve or reject a post, and the check is made again on the server for every decision.
    - The post can only be resolved while it is still pending, so a decision cannot be applied twice to the same post.
    - A rejected post is hidden from the public and kept in the trash instead of being erased, so the decision can be reviewed and undone later.

3.5 ID Verification

3.5.1 View Verification Request List

Function Trigger

The logged-in administrator opens the ID Verification screen by choosing **Users** → **Identity Verification** in the admin sidebar.

Function Description

This function allows an administrator to see all the identity verification requests that users have submitted. Every row shows the identifier, the user, the type of document, the status and the submission date. The administrator can filter the requests by status and search by email or display name.

Screen Layout (Mock-up Description)

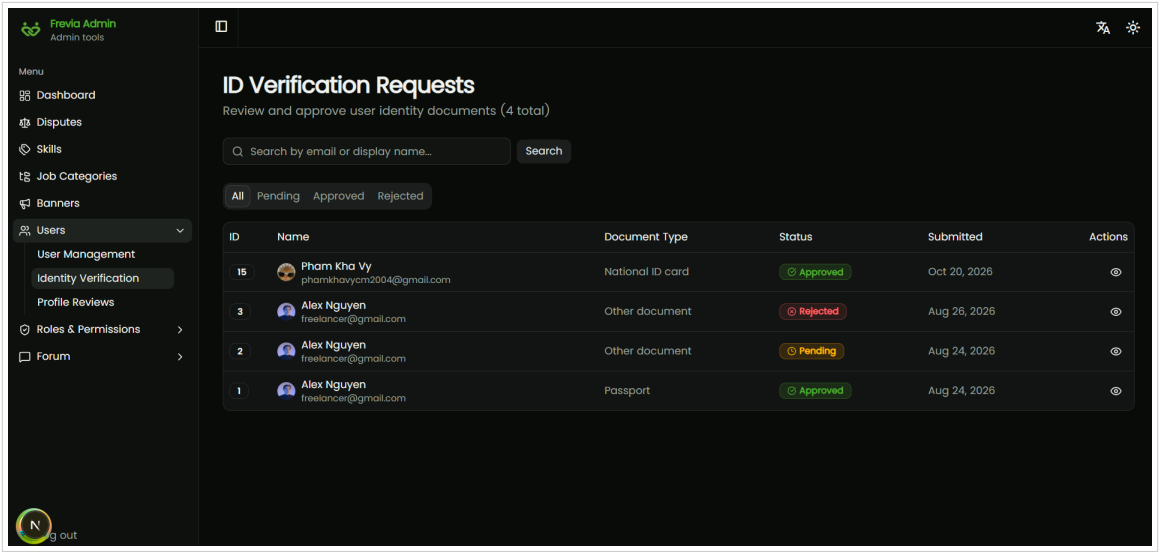


Figure 74: ID verification request list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one user has already submitted an identity document, otherwise the list is empty.
- **Process**
  - **Open the ID Verification Screen**: The user selects **Users** in the sidebar and then **Identity Verification**, and the screen titled "ID Verification Requests" is displayed.
  - **Read the Summary Line**: The subtitle shows the total number of requests in the form "Review and approve user identity documents (n total)".

- **Read Each Row**: Every row of the table presents the following information.
  - **ID**: The identifier of the request, shown as a small badge.
  - **Name**: The avatar, the display name and the email of the user who submitted the document.
  - **Document Type**: The kind of document, shown as National ID card, Passport, Driver license, Residence permit or Other document.
  - **Status**: The current status, shown as a badge with the labels Pending, Approved or Rejected.
  - **Submitted**: The date on which the document was submitted.
- **Read the Status Colour**: A pending request is shown in amber, an approved request in green and a rejected request in red.
- **Filter by Status**: The user chooses one of the tabs **All**, **Pending**, **Approved** or **Rejected** to show only the requests with that status.
- **Search**: The user types a text in the search box with the placeholder "Search by email or display name..." and submits it, and only the matching requests are shown.
- **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
- **Choose a Request**: The user opens the request with the view button (see **View ID Verification Request Detail**).

- **Alternative Flow**
  - **No Request**: When no request matches the current filter or search, the table shows the message "No verification requests found.".
  - **Clear the Search**: When the user clears the search box, all requests are listed again.
- **Postconditions**
  - The administrator sees the requests that match the chosen status and search text.
  - No data is changed by this function, so the requests stay exactly as they were.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read identity documents.
  - **Lost Connection**: When the requests cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
  - The list of requests is only returned to administrator accounts, so the identity data of a user is never exposed to other users.

- Only the basic information is listed, while the images of the document are shown only inside the detail of a request.

3.5.2 View ID Verification Request Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the ID verification request list.

Function Description

This function allows an administrator to examine an identity verification request in full. The dialog shows the user, the submission date, the type of document, the front and back images of the document, the review notes if a decision has already been made, and the buttons that let the administrator approve or reject a pending request.

Screen Layout (Mock-up Description)

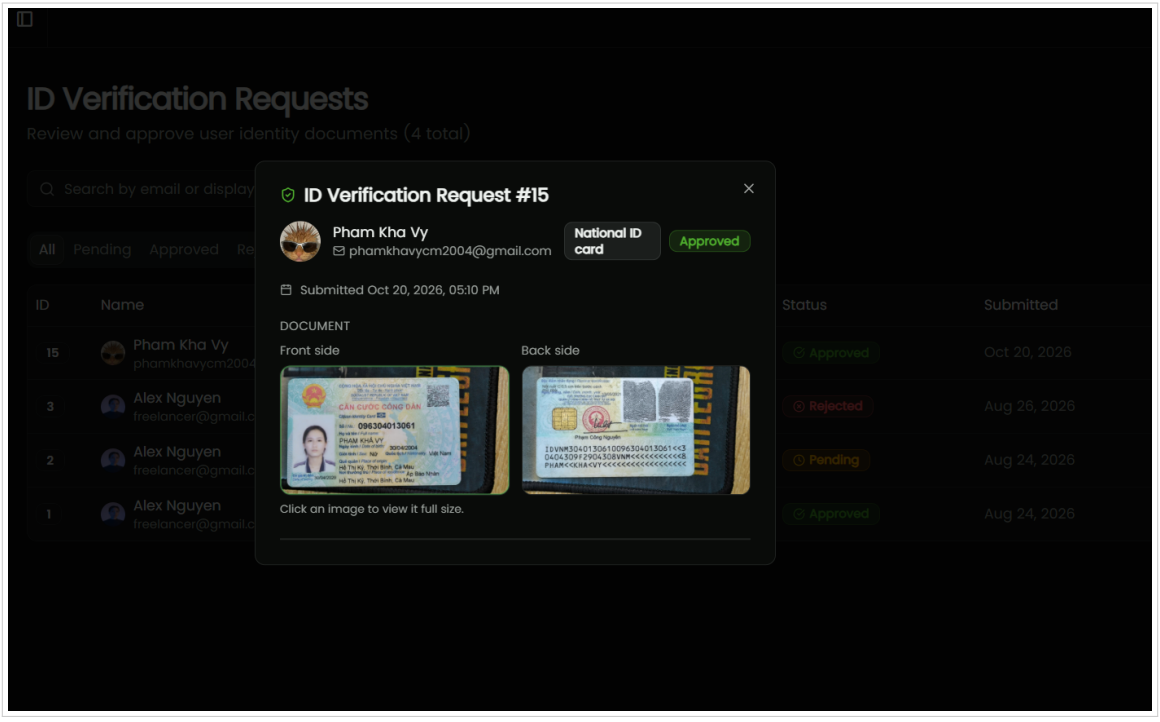


Figure 75: ID verification request detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the ID Verification screen and the request exists.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled “ID Verification Request #id” opens.
  - **Read the Header**: The header shows the avatar, the display name and the email of the user, together with the badge of the document type and the badge of the status.
  - **Read the Submission Date**: The line “Submitted . . . ” shows the moment at which the document was sent, including the time.
  - **Read the Document**: The section **Document** shows the front side of the document and, when it was uploaded, the back side as well.
    - When the back side was not submitted, a dashed placeholder shows the message “No back side uploaded with this submission.”.
  - **Zoom an Image**: The user clicks an image to open it in full size in a larger dialog, and closes the larger dialog to come back to the request.
  - **Read the Review Notes**: When a decision has already been made, the section **Review Notes** shows the note that was left by the administrator who handled the request.
  - **Decide on a Pending Request**: When the request is pending, the dialog also shows a review notes box and the **Approve** and **Reject** buttons (see **Approve or Reject ID Verification**).
  - **Close the Detail**: The user closes the dialog and returns to the list, where the row reflects the current status.
- **Alternative Flow**
  - **Already Decided**: When the request has already been approved or rejected, the dialog only shows the images, the status and the review notes, and no decision button appears.
  - **No Back Side**: When only the front side exists, the dialog shows the placeholder instead of the second image and the rest is unchanged.
  - **Close Without Deciding**: When the user closes the dialog without pressing a button, no decision is recorded and the request stays pending.
- **Postconditions**
  - The administrator has seen the user, the document, the date and any previous review

notes.

- Nothing has changed while only viewing the request.

- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read identity documents.
  - **Request Not Found**: When the request no longer exists, the detail cannot be opened and a message tells the user that the request cannot be found.
  - **Document Not Available**: When an image of the document can no longer be read, the image is not shown and a message tells the user that the document is not available.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The images of the document are delivered only to an administrator account, so the document of a user is never exposed to anyone else.
  - The document images are opened inside the admin area as read-only content, so they cannot be saved or edited from the screen.
  - The decision buttons are hidden as soon as the request has been handled, so a request can be reviewed only while it is pending.

3.5.3 Approve or Reject ID Verification

Function Trigger

The logged-in administrator presses the **Approve** or **Reject** button inside the detail of a pending ID verification request.

Function Description

This function allows an administrator to close an identity verification request by approving or rejecting the document that the user submitted. The administrator can write an optional review note that is stored with the decision. Approving a request marks the user as verified, while rejecting a request leaves the user unverified.

Screen Layout (Mock-up Description)

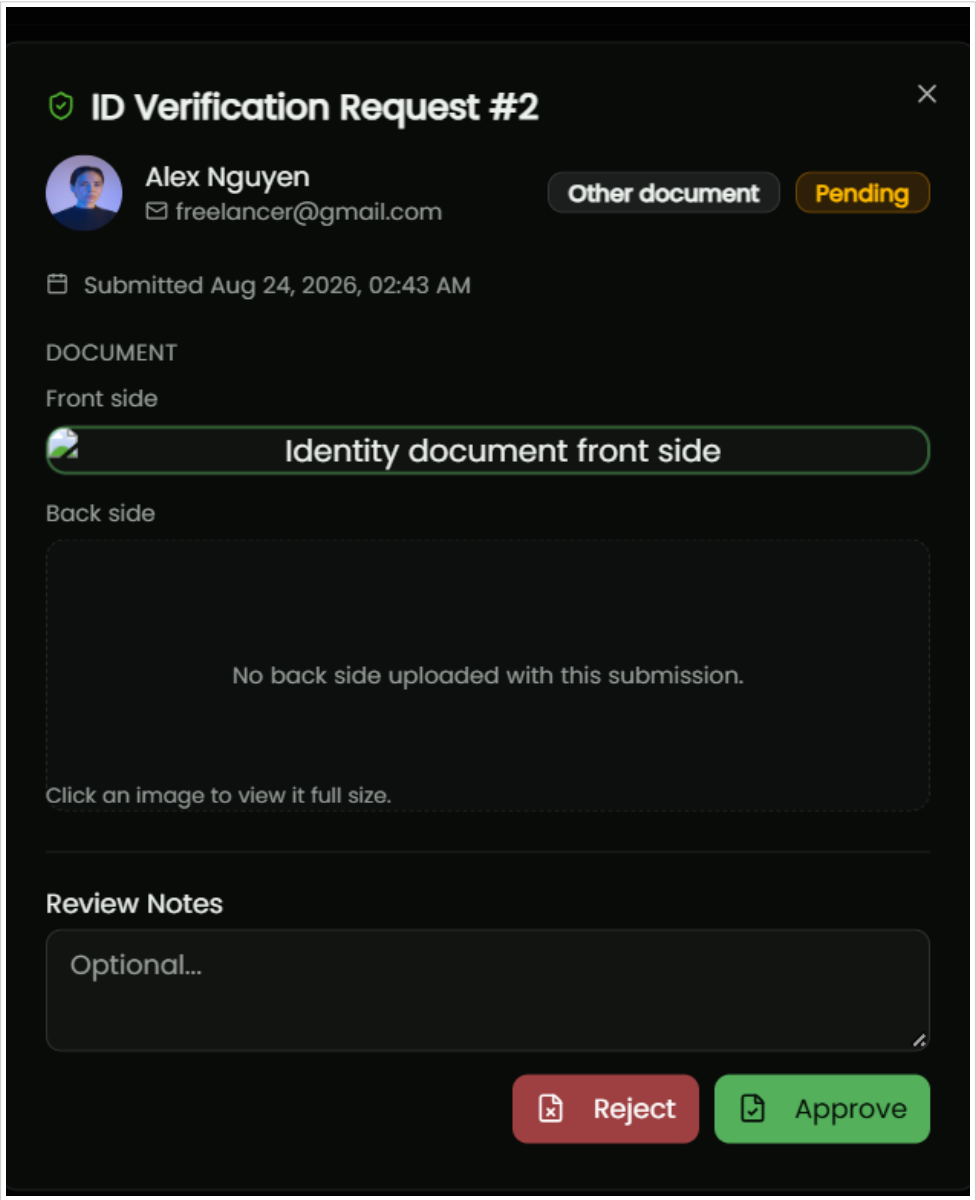


Figure 76: Approving or rejecting an ID verification request

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The request is pending and its document has been examined in the detail dialog.
- **Process**
  - **Write a Review Note**: The user may type a note in the box labeled “Review Notes”, which is optional.

        - The note is stored with the decision and is shown again the next time the request is opened.
    - **Approve the Request**: The user presses **Approve**, and the button shows "Approving…" while the decision is being saved.
        - The request is marked as approved, the user is marked as verified and the screen shows the message "Identity verification approved".
    - **Reject the Request**: The user presses **Reject**, and the button shows "Rejecting…" while the decision is being saved.
        - The request is marked as rejected, the user stays unverified and the screen shows the message "Identity verification rejected".
    - **See the Result**: When the decision is saved, the status badge of the request changes and the decision buttons disappear, so the request is now read-only.
- **Alternative Flow**
    - **No Review Note**: When the user leaves the box empty, the decision is saved without a note.
    - **Note Too Long**: When the note is longer than the allowed length, the note is refused and the administrator is asked to shorten it before the decision can be saved.
    - **Reopen the Detail Later**: When the request is opened again, it shows the chosen status, the review note and no decision buttons.
- **Postconditions**
    - The request has the status that the administrator chose, and it is no longer pending.
    - An approved user is marked as verified, while a rejected user stays unverified.
    - The decision, the note and the moment of the decision are stored with the request.
- **Error Handling**
    - **Not Allowed**: When a user who is not an administrator tries to approve or reject a request, the action is refused because only administrators may decide on identity documents.
    - **Request Not Found**: When the request no longer exists, the decision is refused and the screen shows the message "Couldn't update the request. Try again.".
    - **Already Reviewed**: When another administrator has already handled the request, the decision is refused and a message tells the user that the request has already been reviewed.
    - **Lost Connection**: When the connection to the server is lost, the decision is not saved and the screen asks the user to try again.
- **Security Measures**
    - Only administrators may approve or reject a request, and the check is made again on

the server for every decision.

- A request can only be decided while it is still pending, so a decision cannot be applied twice to the same request.
- Each decision is recorded together with the administrator who made it, the note and the moment of the decision, so every review can be traced.

3.6 Trash

3.6.1 View Deleted Post List

Function Trigger

The logged-in administrator opens the Trash screen by choosing **Forum** → **Trash** in the admin sidebar while the **Posts** tab is selected.

Function Description

This function allows an administrator to see all the posts that have been removed from the forum, including the posts that were rejected during moderation. Every row shows the identifier, the title, the status, the author and the moment of deletion, so the administrator can decide which posts to restore.

Screen Layout (Mock-up Description)

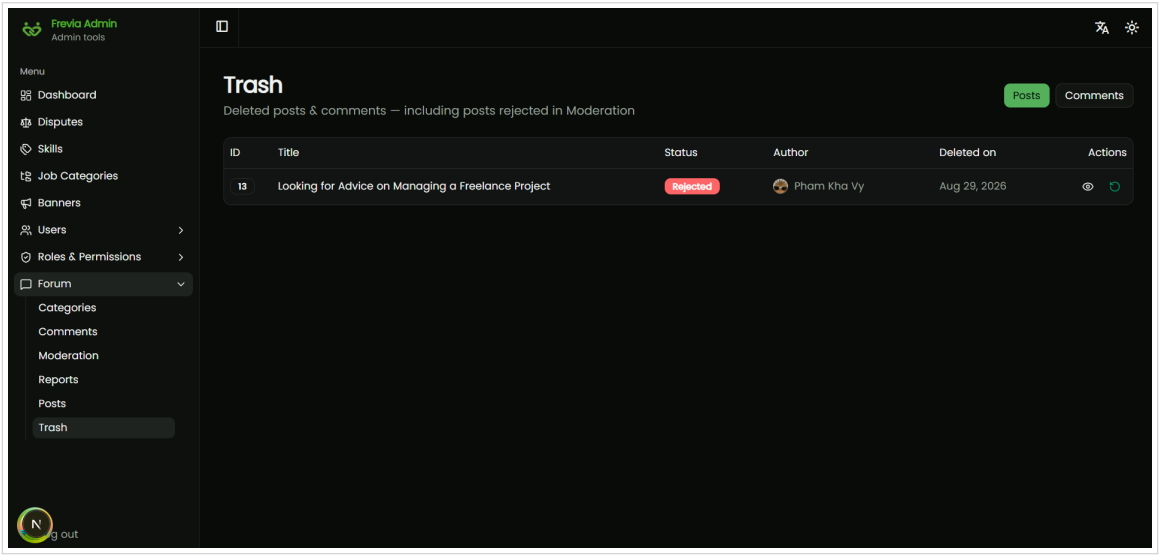


Figure 77: Deleted post list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one post has been deleted or rejected, otherwise the trash is empty.
- **Process**
  - **Open the Trash Screen**: The user selects **Forum** in the sidebar and then **Trash**, and the screen titled “Trash” is displayed with the tab **Posts** active.
  - **Read the Summary Line**: The subtitle explains that the trash contains deleted posts and comments, including the posts that were rejected during moderation.
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the post, shown as a small badge.
    - **Title**: The title of the post, shortened when it is too long.
    - **Status**: The moderation status of the post, shown as the badge Rejected, Pending or Approved.
    - **Author**: The avatar and the display name of the user who wrote the post, or “User #id” when the user has no display name.
    - **Deleted on**: The date on which the post was deleted, or “—” when the post is in the trash because it was rejected.
  - **Switch to Comments**: The user presses the **Comments** tab at the top right of the screen to see the deleted comments instead (see **View Deleted Comment List**).
  - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.
  - **Choose a Post**: The user opens a post with the view button, or restores it with the restore button in the row (see **View Deleted Post Detail** and **Restore Deleted Post**).
- **Alternative Flow**
  - **Empty Trash**: When nothing has been deleted or rejected, the table shows the message “Trash is empty.”.
  - **Rejected Post Without a Deletion Date**: When a post is in the trash because it was rejected, the column shows “—” while the status badge shows Rejected.
- **Postconditions**
  - The administrator sees all the posts that are currently out of the forum.
  - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is

sent to the access-denied page, because only administrators may read the trash.

- **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.

- **Security Measures**
  - The trash is only returned to administrator accounts, so deleted and rejected posts are never visible to ordinary users.
  - A rejected post is kept in the trash instead of being erased, so it can be restored later if the decision was wrong.

3.6.2 View Deleted Post Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted post list.

Function Description

This function allows an administrator to read a deleted or rejected post in full before restoring it. The dialog shows the title, the status, the author, the creation date, the moment of deletion and the complete content of the post.

Screen Layout (Mock-up Description)

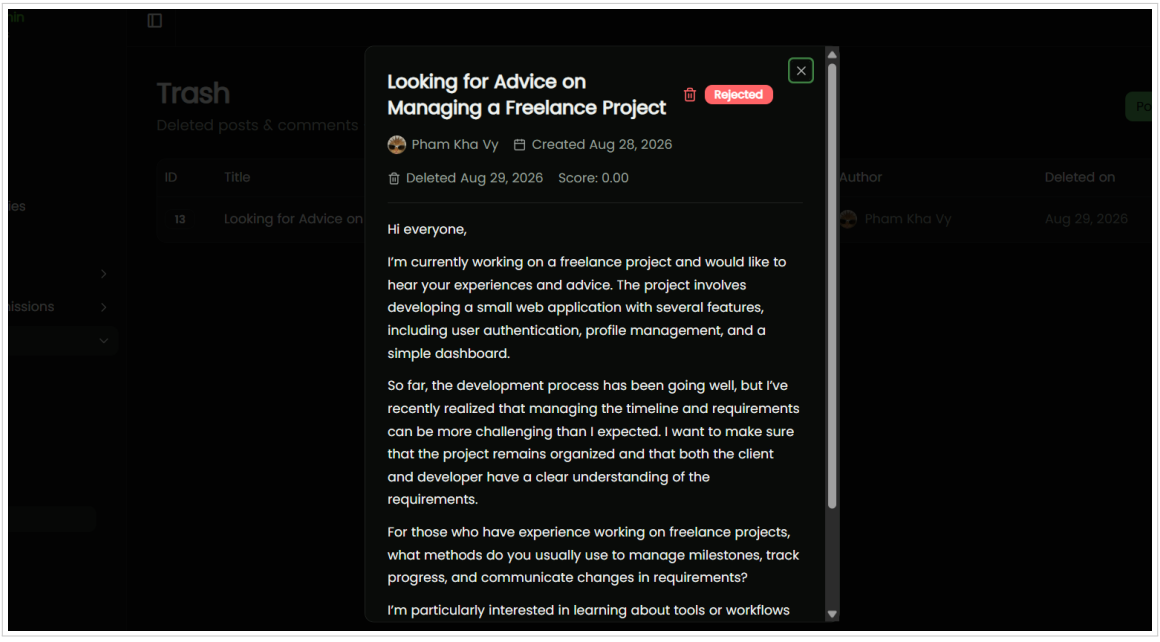


Figure 78: Deleted post detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Posts** tab active and the post exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog opens with the title of the post and its status badge as its heading.
  - **Read the Summary**: Under the title, the dialog shows the author of the post, the creation date, the moment of deletion and the moderation score.
    - When the post is in the trash because it was rejected and not because it was deleted, the line reads “In trash (rejected)” instead of the deletion date.
  - **Read the Content**: The dialog shows the whole content of the post with its formatting, exactly as it was written by the author.
  - **Scroll When Needed**: When the post is long, the dialog becomes scrollable so that the whole content can be read.
  - **Close the Detail**: The user closes the dialog and returns to the trash list.
- **Alternative Flow**
  - **No Score**: When the automatic check produced no score, the badge shows “N/A” and the rest of the dialog is unchanged.
  - **Rejected Post**: When the post was rejected during moderation, the dialog shows the Rejected badge and the line “In trash (rejected)”.
- **Postconditions**
  - The administrator has read the complete post and knows why it is out of the forum.
  - Nothing has changed while only viewing the post.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
  - **Post Not Found**: When the post is no longer in the trash, the detail cannot be opened and a message tells the user that the post cannot be found.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so the content of a deleted post is

never shown to ordinary users.

- The dialog is read-only, so the content of a deleted post cannot be edited from this screen.

3.6.3 Restore Deleted Post

Function Trigger

The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted post list.

Function Description

This function allows an administrator to bring a post back into the forum. A post that was only deleted becomes visible again straight away, while a post that was rejected goes back to the moderation queue so that it can be reviewed again. Before the post is restored, the administrator is asked to confirm the action.

Screen Layout (Mock-up Description)

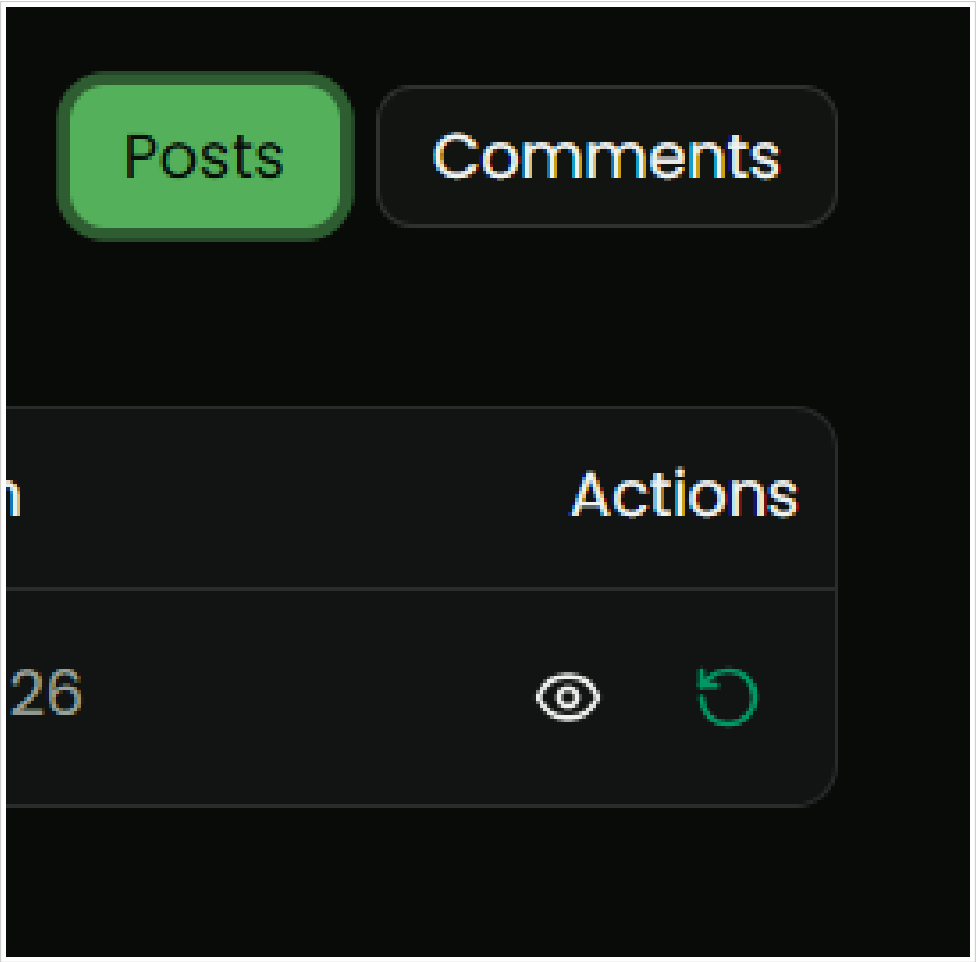


Figure 79: Restore post confirmation dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The post is currently in the trash, either because it was deleted or because it was rejected during moderation.
- **Process**
  - **Start the Restore**: The user clicks the restore button on a row, and a confirmation dialog titled “Restore Post” appears.
  - **Read the Confirmation**: The dialog shows the question “Restore “title” back to the forum?” and explains what will happen.
    - When the post was rejected, the dialog adds “It will return to the moderation queue for review.”.
    - When the post was only deleted, the dialog adds “It will be visible publicly again immediately.”.
  - **Confirm the Restore**: The user presses **Restore**, and the button shows “Restoring. . . ” while the action is being applied.
  - **Apply the Restore**: The post is taken out of the trash and the screen shows the message “Post restored successfully”.
    - A post that was only deleted becomes visible in the forum again straight away.
    - A post that was rejected is put back into the moderation queue with the status Pending, so the administrator can review it again in the Moderation screen.
  - **Update the Trash**: The restored post disappears from the trash list and the remaining rows move up.
- **Alternative Flow**
  - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the post stays in the trash and nothing changes.
  - **Empty Trash**: When the last post is restored, the table shows the message “Trash is empty.”.
  - **Restore Several Posts**: The user repeats the action on other rows, one post at a time.
- **Postconditions**
  - The post is no longer in the trash.
  - A restored post that was only deleted is visible in the forum again.
  - A restored post that was rejected is waiting again in the moderation queue with the status Pending and without a previous review.

- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to restore a post, the action is refused because only administrators may restore content.
  - **Post Not Found**: When the post is no longer in the trash, the restore fails and the screen shows the message “Couldn’t restore post. Try again.”.
  - **Cannot Restore**: When the post cannot be restored, it stays in the trash and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the restore is not applied and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may restore a post, and the check is made again on the server for every restore.
  - Only a post that is really in the trash can be restored, so the action cannot be used on a post that is already visible in the forum.
  - A restored post that had been rejected is never made public directly: it is put back into the moderation queue, so it must pass a new review before it can be seen by users.

3.6.4 View Deleted Comment List

Function Trigger

The logged-in administrator opens the Trash screen by choosing **Forum** $\rightarrow$ **Trash** in the admin sidebar and then presses the **Comments** tab.

Function Description

This function allows an administrator to see all the comments that have been removed from the forum. Every row shows the identifier, the text of the comment, the post it belongs to, the author and the moment of deletion, so the administrator can decide which comments to restore.

Screen Layout (Mock-up Description)

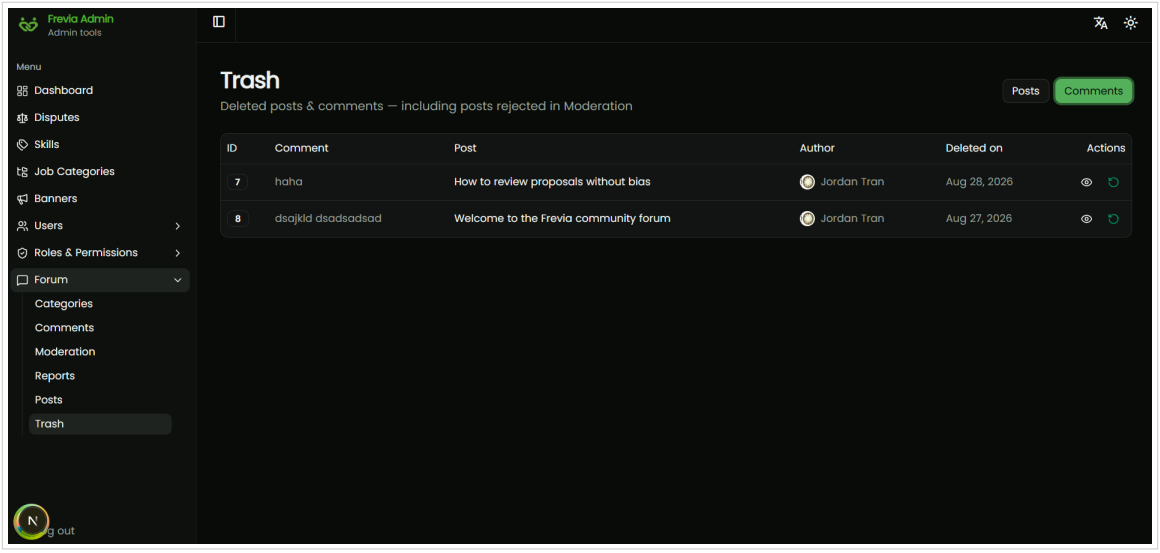


Figure 80: Deleted comment list screen

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - At least one comment has been deleted, otherwise the trash is empty.
- **Process**
  - **Open the Comments Tab**: The user opens the Trash screen and presses the **Comments** tab at the top right, and the screen titled "Trash" is displayed with the comments of the trash.
  - **Read Each Row**: Every row of the table presents the following information.
    - **ID**: The identifier of the comment, shown as a small badge.
    - **Comment**: The text of the comment, shortened when it is too long.
    - **Post**: The title of the post that the comment belongs to, or "Post #id" when the title is not available.
    - **Author**: The avatar and the display name of the user who wrote the comment, or "User #id" when the user has no display name.
    - **Deleted on**: The date on which the comment was deleted.
  - **Switch to Posts**: The user presses the **Posts** tab at the top right of the screen to see the deleted posts instead (see **View Deleted Post List**).
  - **Move Between Pages**: When there is more than one page, the user uses the page numbers or the **Previous** and **Next** buttons at the bottom of the list.

- **Choose a Comment**: The user opens a comment with the view button, or restores it with the restore button in the row (see **View Deleted Comment Detail** and **Restore Deleted Comment**).

- **Alternative Flow**
  - **Empty Trash**: When no comment has been deleted, the table shows the message "Trash is empty.".
  - **Missing Post Title**: When the post of the comment is not available, the row shows only the identifier of the post.
- **Postconditions**
  - The administrator sees all the comments that are currently out of the forum.
  - No data is changed by this function, so the trash stays exactly as it was.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator opens the screen, the user is sent to the access-denied page, because only administrators may read the trash.
  - **Lost Connection**: When the trash cannot be loaded, the screen shows an empty list so that the administrator can reload the page and try again.
- **Security Measures**
  - The list of deleted comments is only returned to administrator accounts, so a deleted comment is never shown to ordinary users.
  - The delete action of a comment does not erase its content, so the administrator can still read what was removed before deciding to restore it.

3.6.5 View Deleted Comment Detail

Function Trigger

The logged-in administrator clicks the view button (the eye icon) at the end of a row in the deleted comment list.

Function Description

This function allows an administrator to read a deleted comment in full before restoring it. The dialog shows the post that the comment belongs to, the author, the complete text of the comment, the identifier of the comment and the moment of deletion.

Screen Layout (Mock-up Description)

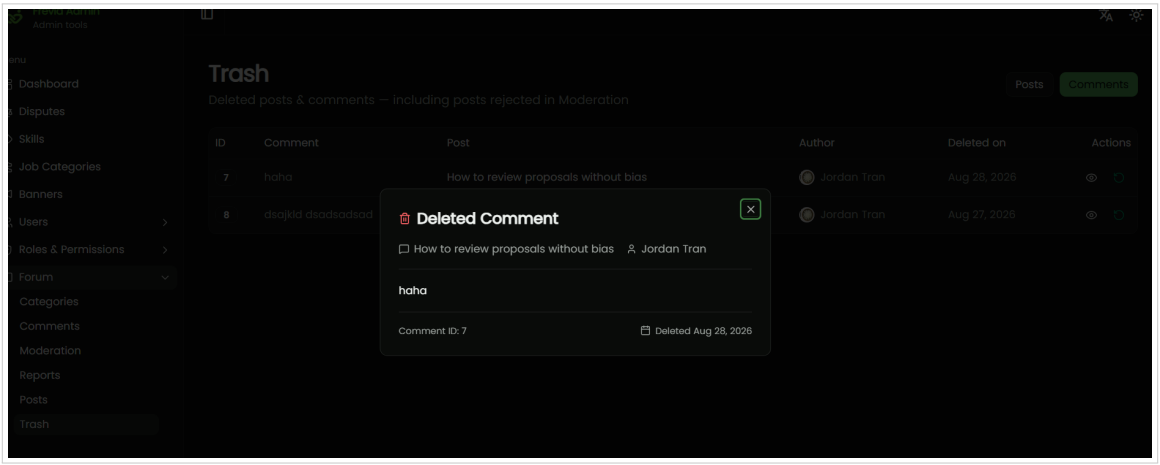


Figure 81: Deleted comment detail dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The user is on the Trash screen with the **Comments** tab active and the comment exists in the trash.
- **Process**
  - **Open the Detail**: The user clicks the view button on a row, and a dialog titled "Deleted Comment" opens.
  - **Read the Summary**: Under the title, the dialog shows the title of the post that the comment belongs to and the name of the author.
  - **Read the Content**: The dialog shows the complete text of the comment, exactly as it was written by the author.
  - **Read the Footer**: The footer of the dialog shows the identifier of the comment and the moment at which it was deleted.
  - **Close the Detail**: The user closes the dialog and returns to the trash list.
- **Alternative Flow**
  - **Missing Post Title**: When the title of the post is not available, the dialog shows only the identifier of the post.
  - **Missing Deletion Date**: When the deletion date is not available, the footer shows "—".
- **Postconditions**
  - The administrator has read the complete comment and knows to which post it belongs.

- Nothing has changed while only viewing the comment.

- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to open the detail, the action is refused because only administrators may read the trash.
  - **Comment Not Found**: When the comment is no longer in the trash, the detail cannot be opened and a message tells the user that the comment cannot be found.
  - **Lost Connection**: When the detail cannot be loaded, the screen asks the user to try again.
- **Security Measures**
  - The detail is only available inside the admin area, so the content of a deleted comment is never shown to ordinary users.
  - The dialog is read-only, so the content of a deleted comment cannot be edited from this screen.

3.6.6 Restore Deleted Comment

Function Trigger

The logged-in administrator clicks the restore button (the round arrow icon) at the end of a row in the deleted comment list.

Function Description

This function allows an administrator to bring a deleted comment back to its post. The comment is shown again to users when its post is still visible in the forum. Before the comment is restored, the administrator is asked to confirm the action.

Screen Layout (Mock-up Description)

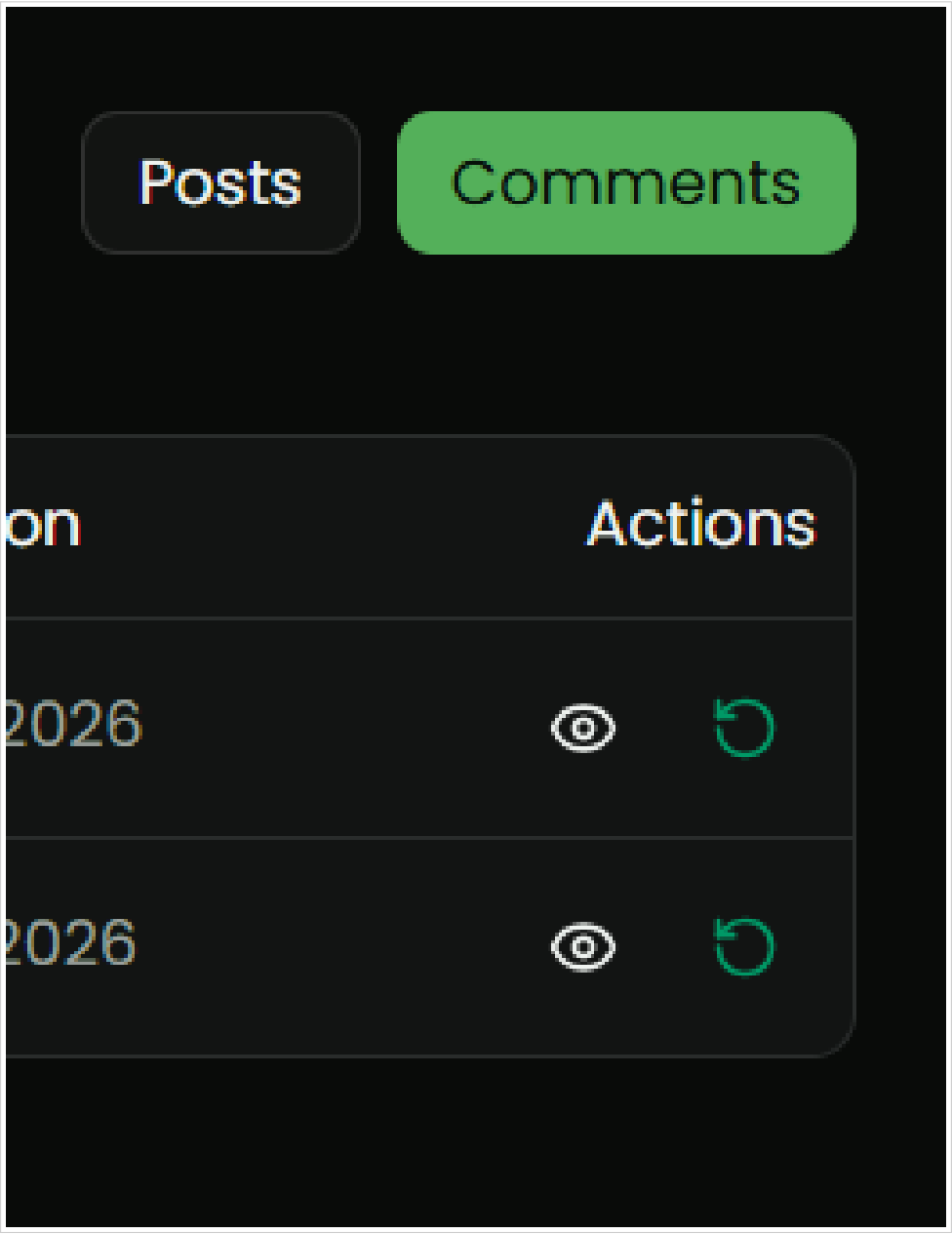


Figure 82: Restore comment confirmation dialog

Function Details

- **Precondition**
  - The user is logged in with an administrator account.
  - The user is connected to the internet.
  - The comment is currently in the trash because it was deleted.
- **Process**
  - **Start the Restore**: The user clicks the restore button on a row, and a confirmation

dialog titled “Restore Comment” appears.

- **Read the Confirmation**: The dialog shows the question “Restore this comment on post “title”? It will be visible again (as long as its post is not deleted).”.
- **Confirm the Restore**: The user presses **Restore**, and the button shows “Restoring…” while the action is being applied.
- **Apply the Restore**: The comment is taken out of the trash and the screen shows the message “Comment restored successfully”.
  - The comment appears again under its post when the post itself is still visible in the forum.
  - When the post has also been deleted, the comment stays hidden until the post is restored as well.
- **Update the Trash**: The restored comment disappears from the trash list and the remaining rows move up.

- **Alternative Flow**
  - **Cancel the Restore**: When the user presses **Cancel** or closes the dialog, the comment stays in the trash and nothing changes.
  - **Deleted Post**: When the post of the comment has also been deleted, the comment is restored but remains hidden until the post comes back to the forum.
  - **Empty Trash**: When the last comment is restored, the table shows the message “Trash is empty.”.
- **Postconditions**
  - The comment is no longer in the trash and is visible again under its post when the post is visible.
  - The rest of the comments and the post itself are unchanged.
- **Error Handling**
  - **Not Allowed**: When a user who is not an administrator tries to restore a comment, the action is refused because only administrators may restore content.
  - **Comment Not Found**: When the comment is no longer in the trash, the restore fails and the screen shows the message “Couldn’t restore comment. Try again.”.
  - **Cannot Restore**: When the comment cannot be restored, it stays in the trash and a message asks the administrator to try again.
  - **Lost Connection**: When the connection to the server is lost, the restore is not applied and the screen asks the user to try again.
- **Security Measures**
  - Only administrators may restore a comment, and the check is made again on the server for every restore.

- Only a comment that is really in the trash can be restored, so the action cannot be used on a comment that is already visible.
- A restored comment can never become visible outside a visible post, so the owner of a deleted post cannot see comments that belong to removed content.

4. Non-Functional Requirements

This section describes the non-functional requirements of the Frevia platform: the external interfaces through which the system communicates, and the quality attributes that the delivered system must satisfy.

4.1 External Interfaces

The system communicates with four kinds of external element. Each interface is listed below with the constraint that the implementation must respect.

- **User interfaces.** The platform is delivered as a responsive web client that runs in a current version of Chrome, Edge, Firefox, or Safari. The layout must remain usable from a viewport width of 320 px upwards, and every screen must be available in English and Vietnamese.
- **Software interfaces.** Payments are processed through Stripe (Connect Express, PaymentIntent, Checkout Session, and signature-verified webhooks); profile, portfolio, and portfolio media are stored on Cloudinary; one-time verification and password reset codes are delivered over SMTP through Nodemailer; curriculum vitae analysis and identity document verification are performed by external AI services called over HTTPS; and all persistent data is stored in PostgreSQL and accessed through Prisma.
- **Communication interfaces.** Browser-to-server traffic uses HTTPS with TLS 1.3. Calls from the API to Stripe, Cloudinary, the mail server, and the external AI services are made server-side over HTTPS with an explicit timeout, and a failure of any of them must be reported to the caller as a handled error rather than as an unhandled exception. Real-time messaging between the web client and the API uses a Socket.IO connection authenticated with the same JWT as the REST endpoints.
- **Hardware interfaces.** No dedicated hardware is required beyond the device that runs the browser and the camera or file system used to supply identity documents, avatars, and portfolio media.

4.2 Quality Attributes

4.2.1 Usability

- **Responsive Design:** The user interface must be fully responsive, supporting mobile devices, tablets, and desktop viewports (minimum width 320px).
- **Toast Notification System:** Interactive toast notifications (success, warning, error) must have the highest z-index to overlay all content, auto-dismiss after 3000ms, and must not freeze or block other user interactions.
- **Client-side Image Processing:** The avatar cropping tool must handle images locally before upload. The system must restrict upload sizes to under 5MB and validate formats (.jpg, .jpeg, .png, .webp) before initiating API requests.

4.2.2 Reliability

- **System Uptime:** The system core services must maintain a 99.9% monthly uptime, excluding scheduled maintenance.
- **Blockchain Fault Tolerance:** In case of RPC node connection failures, the platform must gracefully fall back to backup RPC endpoints within 3 seconds to avoid breaking the wallet balance display or transaction signing flows.
- **Escrow Reliability:** Funds locked in the Escrow Smart Contracts must be secure. Contracts must be verified on Etherscan/BscScan and use standard libraries (e.g., OpenZeppelin) to prevent vulnerabilities.

4.2.3 Performance

- **Response Time:** The system's API read operations must respond within 200ms under normal load (up to 500 concurrent requests). Write/update operations, excluding blockchain transactions, must complete within 500ms.
- **Page Load Time:** The web application must achieve a Largest Contentful Paint (LCP) of less than 2.5 seconds, adhering to Google's Core Web Vitals guidelines.
- **Concurrency:** The system backend must support at least 1,000 concurrent active users and handle a database connection pool efficiently.
- **Search and Filtering Efficiency:** The job search page must employ debouncing (300ms delay) and pagination (default 10 items per page) to optimize query performance and reduce server load.

4.2.4 Security & Privacy

- **Authentication and Authorization:** All network communications must be encrypted using HTTPS/TLS 1.3. User passwords must be hashed using bcrypt with a work factor of 12.
- **Session Management:** JWT Access Tokens must have a short lifespan (e.g., 15 minutes) and be stored in memory. Refresh Tokens must have a longer lifespan (e.g., 7 days), be stored in HTTP-only, Secure, and SameSite=Strict cookies, and be invalidated in the database upon user logout.
- **Role-Based Access Control (RBAC):** The system must strictly enforce role boundaries (Guest, Freelancer, Client, Admin). Administrative features, such as changing smart contract addresses or viewing audit logs, must be blocked by a server-side gateway for non-admin accounts.
- **Web3 Wallet and Transaction Safety:** Input wallet addresses must be verified on the client and server side to conform to the standard EVM format (starts with `0x` followed by 40 hexadecimal characters).

5. Requirement Appendix

This appendix collects the requirements that apply across features rather than to a single screen: the business rules that every use case must respect, the common requirements that hold for the whole system, and the standard application messages that the interface reuses.

5.1 Business Rules

The rules below are grouped by the feature they constrain. A use case that violates a rule is rejected, and the identifier of the rule is quoted in the corresponding alternative flow.

Table 45: Business Rules

ID	Rule Definition
BR-PROF-01	Both basic details (from profiles) and professional details (from freelancer_profiles) must be presented together.
BR-PROF-02	Hidden or private fields should not be displayed to unauthorized users (e.g., clients vs. the profile owner).
BR-PROF-03	Freelancers can only update their own profile.
BR-PROF-04	Mandatory fields (e.g., display_name, title) cannot be left empty.

ID	Rule Definition
BR-SKILL-01	All skills belonging to the freelancer must be displayed along with their proficiency level.
BR-SKILL-02	A freelancer cannot add duplicate skills.
BR-SKILL-03	Both skill name and proficiency level are required.
BR-SKILL-04	Skill deletion must be a hard delete operation; the record is permanently removed from the mapping table.
BR-SKILL-05	Only the owning freelancer or an administrator can delete a skill from the profile.
BR-PORT-01	Only portfolios with deleted_at = NULL shall be displayed.
BR-PORT-02	Portfolio title is a required field.
BR-PORT-03	Media uploads must conform to system-defined size and type restrictions.
BR-PORT-04	Only portfolios with deleted_at = NULL can be accessed.
BR-PORT-05	Only the portfolio owner can edit it.
BR-PORT-06	Portfolio title remains a required field.
BR-PORT-07	Portfolio deletion must be a soft delete (updating the deleted_at column).
BR-PORT-08	Soft-deleted portfolios must not be visible to clients or guests.
BR-FORUM-01	Only categories with is_active = TRUE shall be displayed.
BR-FORUM-02	Category information displayed includes name and description.
BR-FORUM-03	Only categories with is_active = TRUE shall be accessible.
BR-FORUM-04	Category information displayed includes name and description.
BR-FORUM-05	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-06	Posts with is_pinned = TRUE shall appear before regular posts.
BR-FORUM-07	Posts shall be sorted by created_at in descending order within each group.

ID	Rule Definition
BR-FORUM-08	The system shall support pagination for post listings.
BR-FORUM-09	Only posts with deleted_at = NULL shall be displayed.
BR-FORUM-10	Post details shall include title, content, author, category, and created_at information.
BR-FORUM-11	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-12	Comment information shall include content, author, and created_at.
BR-FORUM-13	Only comments with deleted_at = NULL shall be displayed.
BR-FORUM-14	Comment details shall include content, author, and created_at information.
BR-FORUM-15	Only authenticated users can create posts.
BR-FORUM-16	Every post must belong to exactly one category.
BR-FORUM-17	Title and content are required fields.
BR-FORUM-18	Only the author of a post can edit that post.
BR-FORUM-19	Every post must belong to a valid category.
BR-FORUM-20	Title and content are required fields.
BR-FORUM-21	Only the author of a post can delete that post.
BR-FORUM-22	Post deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-23	Soft-deleted posts shall not appear in public forum listings.
BR-FORUM-24	Only authenticated users can like posts.
BR-FORUM-25	A user can have only one active like reaction per post.
BR-FORUM-26	The reaction shall be stored in the forum_likes table.
BR-FORUM-27	Only authenticated users can remove post reactions.
BR-FORUM-28	A like reaction can only be removed if a corresponding like record exists.

ID	Rule Definition
BR-FORUM-29	Removing a reaction shall delete the corresponding record from the forum_likes table.
BR-FORUM-30	Only authenticated users can create comments.
BR-FORUM-31	Comment content cannot be empty.
BR-FORUM-32	Comment content must contain at least 2 characters.
BR-FORUM-33	Each comment shall be associated with a valid post_id and user_id.
BR-FORUM-34	Only the owner of a comment can edit it.
BR-FORUM-35	Comment content cannot be empty.
BR-FORUM-36	Comment content must contain at least 2 characters.
BR-FORUM-37	The system shall record the updated_at timestamp when a comment is modified.
BR-FORUM-38	Only the owner of a comment can delete it.
BR-FORUM-39	Comment deletion shall be implemented as a soft delete using the deleted_at field.
BR-FORUM-40	Soft-deleted comments shall not be displayed in normal comment listings.
BR-FORUM-41	Only authenticated users can like comments.
BR-FORUM-42	A user can have only one active like reaction per comment.
BR-FORUM-43	The reaction shall be stored in the forum_likes table using comment_id.
BR-FORUM-44	Only authenticated users can remove comment reactions.
BR-FORUM-45	A like reaction can only be removed if a corresponding like record exists.
BR-FORUM-46	Removing a reaction shall delete the corresponding record from the forum_likes table using comment_id.
BR-FORUM-47	Only authenticated users can submit reports.
BR-FORUM-48	Reports must be associated with either a post_id or comment_id.

ID	Rule Definition
BR-FORUM-49	New reports shall be created with status = PENDING.
BR-FORUM-50	A user may submit only one active report for the same content item.
BR-JOB-01	Only jobs with status = 'open' or 'in_progress' shall be displayed to freelancers and guests.
BR-JOB-02	Clients shall see all their jobs regardless of status.
BR-JOB-03	Default sorting shall be by created_at in descending order.
BR-JOB-04	The system shall support pagination with default page size of 10.
BR-JOB-05	Search shall match keywords in title and description fields (case-insensitive).
BR-JOB-06	When multiple filters are applied, all conditions shall be combined using AND logic.
BR-JOB-07	Job information displayed includes: title, description, budget, deadline, category, required skills, client name, created date, status.
BR-JOB-08	If the job is inactive (status = 'closed' or 'deleted'), only the job owner can view full details.
BR-JOB-09	Job title is required and must not exceed 200 characters.
BR-JOB-10	Job description is required and must not exceed 5000 characters.
BR-JOB-11	Budget must be a positive number (greater than 0).
BR-JOB-12	Category must be selected from predefined categories.
BR-JOB-13	New jobs shall have status = 'draft' by default when saved as draft, or 'open' when published.
BR-JOB-14	Only the job owner can edit the job.
BR-JOB-15	Editing is restricted if the job has an active contract.
BR-JOB-16	All validation rules from UC-06.2 apply to edits.
BR-JOB-17	Edit actions shall be logged for auditing purposes.
BR-JOB-18	Only the job owner can delete the job.

ID	Rule Definition
BR-JOB-19	Deletion should be soft delete (set deleted_at timestamp, status = 'deleted').
BR-JOB-20	Jobs with active proposals or contracts cannot be deleted.
BR-JOB-21	Delete actions shall be logged for auditing purposes.
BR-JOB-22	Job status workflow: draft → open → in_progress → completed → closed.
BR-JOB-23	Allowed transitions: draft → open (client); open → in_progress (client, when contract created); open → closed (client, if cancelled); in_progress → completed (client, when work done); in_progress → closed (client, if cancelled); completed → closed (system/auto); Any status → deleted (soft delete).
BR-JOB-24	Only the job owner can manually change status.
BR-JOB-25	Status changes shall be logged for auditing purposes.
BR-JOB-26	Notifications shall be sent to freelancers when status changes from draft to open.
BR-JOB-27	Only the freelancer can view their own bookmarks.
BR-JOB-28	Bookmarks shall be sorted by created_at in descending order by default.
BR-JOB-29	Displayed information includes: job title, budget, client name, bookmark date.
BR-JOB-30	The system shall support pagination with default page size of 10.
BR-JOB-31	Only freelancers can bookmark jobs.
BR-JOB-32	Bookmark is a toggle action. Clicking the button toggles between adding and removing.
BR-JOB-33	If the bookmark does not exist, the system creates it and changes the button to "Bookmarked".
BR-JOB-34	If the bookmark exists, the system deletes it and changes the button to "Bookmark".

ID	Rule Definition
BR-JOB-35	Bookmarks can only be created for active jobs (status = 'open' or 'in_progress').
BR-JOB-36	Only the freelancer can remove their own bookmarks.
BR-JOB-37	The system must check if the bookmark exists before deletion.
BR-JOB-38	If the bookmark does not exist, the system shall return a 404 Not Found error.
BR-JOB-39	Removing a bookmark is a hard delete from the database.

5.2 Common Requirements

The following requirements apply to every feature of the platform and are therefore not repeated in each use case.

- **Authorisation.** Every endpoint is protected by the server-side permission guard, which checks the method, the path, and the module against the permissions of the caller's role. A hidden button or a disabled menu item is never treated as an access control.
- **Pagination.** Every list endpoint is paginated, with a default page size of 10 items and an explicit page parameter, so that no screen loads an unbounded result set.
- **Deletion.** Records that other records refer to are soft-deleted by setting `deleted_at`; a soft-deleted record is excluded from every public listing and from every lookup used by a normal user. Hard deletion is limited to join records such as skills, bookmarks, and reactions.
- **Timestamps.** Creation and modification times are stored in UTC and displayed in the time zone of the browser.
- **Money.** All amounts are stored in the smallest currency unit and displayed with two decimals and an explicit currency; a financial record is never updated in place but is corrected by a new ledger entry.
- **File uploads.** Every uploaded file is validated for type and size on the client before the request is sent and on the server before the file is stored.
- **Language.** Every label, message, and validation error is read from the message catalogue, so that the same screen can be rendered in English and in Vietnamese without a code change.

5.3 Application Messages List

The messages below are reused across features. The context column states when the message is raised, and the content column gives the English source text that the message catalogue translates.

#	Message code	Message Type	Context	Content
1	MSG01	Toast, red	The API cannot be reached or the network is down	Cannot connect to the server. Please try again later.
2	MSG02	Toast, red	The server returns an unexpected error	Unable to process your request. Please try again later.
3	MSG03	Toast, red	The caller requests a route outside the permissions of its role	You do not have permission to access this page.
4	MSG04	Inline, under the field	A required field is left empty	All fields are required.
5	MSG05	Inline, under the field	The email does not match a valid format	Please enter a valid email address.
6	MSG06	Inline, under the field	The password does not meet the password policy	Password must be at least 8 characters and include at least one uppercase letter, one number, and one special character.
7	MSG07	Inline, under the field	The password confirmation differs from the password	Passwords do not match.
8	MSG08	Toast, red	A requested record does not exist or has been soft-deleted	Post not found. / Comment not found. / Proposal not found.
9	MSG09	Toast, green	A record has been saved	Profile updated successfully. / Skill added successfully. / Portfolio created successfully.

Table 46: Application Messages List